

小黄搞AI-AI算法手撕代码

目录

传统机器学习与经典算法

- K-Nearest Neighbors (KNN) 算法
- 支持向量机 (SVM) 分类器
- 逻辑回归 (Logistic Regression)
- 决策树 (Decision Tree) 算法
- 朴素贝叶斯 (Naive Bayes) 分类器
- K-means聚类算法
- 主成分分析 (PCA)
- 梯度提升树 (GBDT)

深度学习结构篇

- 全连接神经网络 (MLP)
- 卷积神经网络 (CNN)
- 循环神经网络 (RNN)
- 图神经网络 (GNN)
- Batch Normalization

大模型结构篇

- 自注意力机制 (Self-Attention)
- 层归一化 (Layer Normalization)
- 位置编码 (Positional Encoding)
- Transformer模型
- Multi-Head Attention (MHA)
- Grouped-Query Attention (GQA)
- ROPE (Rotary Position Embedding)
- 前馈神经网络 (Feed-Forward Neural Network)

- RMSNorm
- ReLU
- SwiGLU

强化学习与策略优化

- Q-learning算法
- REINFORCE算法
- 深度Q网络 (DQN)
- PPO
- DPO
- 蒙特卡洛树搜索 (MCTS)
- A*搜索算法
- 标准梯度下降 (Gradient Descent) 算法
- Adam优化器
- 学习率调度 (Learning Rate Scheduling)
- L2正则化
- RMSProp优化算法
- 交叉熵损失函数

参数调优与微调

- Prompt Tuning
- Prefix Tuning
- LoRA (Low-Rank Adaptation)
- AdaLoRA (Adaptive LoRA)
- QLoRA
- Dense模型与MOE (Mixture of Experts)

目录

传统机器学习与经典算法

深度学习结构篇

大模型结构篇

强化学习与策略优化

参数调优与微调

传统机器学习与经典算法

K-Nearest Neighbors (KNN) 算法

支持向量机 (SVM) 分类器

逻辑回归 (Logistic Regression)

决策树 (Decision Tree) 算法

朴素贝叶斯 (Naive Bayes) 分类器

K-means聚类算法

主成分分析 (PCA)

梯度提升树 (GBDT)

线性回归的随机梯度下降 (stochastic gradient descent, SGD)

反向传播(backward propagation, BP)

深度学习结构篇

传统机器学习与经典算法

K-Nearest Neighbors (KNN) 算法

 下面是一个简单实现K-Nearest Neighbors (KNN) 算法的Python代码。**K近邻算法**通过计算测试点与训练点之间的距离（通常使用欧氏距离），然后选取最近的k个训练样本，并通过这些样本的标签进行投票，返回出现频率最高的标签作为预测结果。

```
1 import numpy as np
2 from collections import Counter
3
4 class KNN:
5     def __init__(self, k=3):
6         self.k = k
7
```

```

8     def fit(self, X_train, y_train):
9         self.X_train = X_train  # 保存训练数据的特征
10        self.y_train = y_train  # 保存训练数据的标签
11
12        def predict(self, X_test):
13            predictions = [self._predict(x) for x in X_test]  # 对每个测试样本进行预
14            return np.array(predictions)
15
16        def _predict(self, x):
17            # 计算测试样本x与所有训练样本之间的距离
18            distances = [np.linalg.norm(x - x_train) for x_train in self.X_train]
19            # 按距离排序, 并返回前k个邻居的索引
20            k_indices = np.argsort(distances)[:self.k]
21            # 获取k个邻居的标签
22            k_nearest_labels = [self.y_train[i] for i in k_indices]
23            # 返回出现频率最高的标签 (即最常见的标签)
24            most_common = Counter(k_nearest_labels).most_common(1)
25            return most_common[0][0]
26
27        # 示例用法:
28        if __name__ == "__main__":
29            # 数据集: 特征和标签
30            X_train = np.array([[1, 2], [2, 3], [3, 4], [5, 6], [6, 7], [7, 8], [8,
31            9]])
32            y_train = np.array([0, 0, 0, 1, 1, 1, 1])
33
34            X_test = np.array([[2, 2], [6, 6]])
35
36            # 初始化KNN, k=3
37            knn = KNN(k=3)
38            knn.fit(X_train, y_train)
39            predictions = knn.predict(X_test)
40
41            # 打印预测结果
42            print("预测标签:", predictions)

```



Output:

预测标签: [0 1]

这意味着程序预测第一个测试点所属类别为0, 第二个测试点为1。



说明

- **KNN算法类创建:** `KNN` 类负责存储训练数据，并通过计算测试点与训练点之间的欧氏距离来进行预测。
- `fit` 方法: 存储训练数据中的特征和标签。
- `predict` 方法: 在测试数据中逐个预测每个样本的类标签。
- `_predict` 方法: 这是一个内部方法，用以计算输入样本与训练样本的距离，并选择最近的 `k` 个点，然后通过这些点的标签来确定预测标签。

支持向量机 (SVM) 分类器

```
1  import numpy as np
2
3  class SVM:
4      def __init__(self, learning_rate=0.001, regularization_strength=0.01,
5          n_iter=1000):
6          self.learning_rate = learning_rate # 学习率
7          self.regularization_strength = regularization_strength # 正则化强度
8          self.n_iter = n_iter # 迭代次数
9
10         def fit(self, X_train, y_train):
11             n_samples, n_features = X_train.shape
12             self.w = np.zeros(n_features) # 初始化权重
13             self.b = 0 # 初始化偏置
14
15             # 训练过程
16             for _ in range(self.n_iter):
17                 for idx, x_i in enumerate(X_train):
18                     condition = y_train[idx] * (np.dot(x_i, self.w) - self.b) >= 1
19                     if condition:
20                         self.w -= self.learning_rate * (2 *
21 self.regularization_strength * self.w)
22                     else:
23                         self.w -= self.learning_rate * (2 *
24 self.regularization_strength * self.w - np.dot(x_i, y_train[idx]))
25                         self.b -= self.learning_rate * y_train[idx]
26
27         def predict(self, X_test):
28             predictions = [np.sign(np.dot(x, self.w) - self.b) for x in X_test]
29             # 使用训练得到的w和b进行预测
30             return np.array(predictions)
```

逻辑回归 (Logistic Regression)

```

1  import numpy as np
2
3  class LogisticRegression:
4      def __init__(self, learning_rate=0.01, n_iter=1000):
5          self.learning_rate = learning_rate # 学习率
6          self.n_iter = n_iter # 迭代次数
7
8      def sigmoid(self, z):
9          return 1 / (1 + np.exp(-z)) # Sigmoid激活函数
10
11     def fit(self, X_train, y_train):
12         n_samples, n_features = X_train.shape
13         self.weights = np.zeros(n_features) # 初始化权重
14         self.bias = 0 # 初始化偏置
15
16         for _ in range(self.n_iter):
17             linear_model = np.dot(X_train, self.weights) + self.bias # 线性模型
18             y_pred = self.sigmoid(linear_model) # 预测结果
19
20             # 计算梯度
21             dw = (1 / n_samples) * np.dot(X_train.T, (y_pred - y_train)) # 权重梯度
22             db = (1 / n_samples) * np.sum(y_pred - y_train) # 偏置梯度
23
24             # 更新参数
25             self.weights -= self.learning_rate * dw
26             self.bias -= self.learning_rate * db
27
28     def predict(self, X_test):
29         linear_model = np.dot(X_test, self.weights) + self.bias # 线性模型
30         y_pred = self.sigmoid(linear_model) #
31

```

决策树（Decision Tree）算法

决策树是一种常见的分类和回归方法，它通过不断地将数据集分割成更小的子集，最终形成树状结构。每个节点表示一个特征或属性，分支表示该特征的不同取值，叶子节点表示分类标签（或回归值）。常见的决策树算法包括CART（分类和回归树），它使用信息增益或基尼指数来选择分割特征。

```

1  import numpy as np
2

```

```

3  # 计算数据集的基尼指数
4  def gini_impurity(y):
5      # 计算类别的频率
6      classes = np.unique(y)
7      gini = 1
8      for c in classes:
9          p = np.sum(y == c) / len(y)
10         gini -= p ** 2
11     return gini
12
13 # 找到最佳的特征和分割点
14 def best_split(X, y):
15     best_gini = float('inf')
16     best_split = None
17     m, n = X.shape
18     for feature_index in range(n):
19         values = np.unique(X[:, feature_index])
20         for value in values:
21             left_mask = X[:, feature_index] <= value
22             right_mask = ~left_mask
23             left_y, right_y = y[left_mask], y[right_mask]
24
25             # 计算基尼指数
26             gini = (len(left_y) * gini_impurity(left_y) + len(right_y) *
27                    gini_impurity(right_y)) / len(y)
28             if gini < best_gini:
29                 best_gini = gini
30                 best_split = (feature_index, value)
31     return best_split
32
33 # 构建决策树
34 class DecisionTree:
35     def __init__(self, max_depth=None):
36         self.max_depth = max_depth
37         self.tree = None
38
39     def fit(self, X, y):
40         self.tree = self._build_tree(X, y, depth=0)
41
42     def _build_tree(self, X, y, depth):
43         # 停止条件：最大深度或者纯度达到
44         if len(np.unique(y)) == 1 or (self.max_depth and depth ==
45         self.max_depth):
46             return np.unique(y)[0]
47
48         # 找到最佳分割
49         feature_index, value = best_split(X, y)

```

```

48
49     left_mask = X[:, feature_index] <= value
50     right_mask = ~left_mask
51     left_y, right_y = y[left_mask], y[right_mask]
52     left_X, right_X = X[left_mask], X[right_mask]
53
54     # 递归构建左右子树
55     left_tree = self._build_tree(left_X, left_y, depth + 1)
56     right_tree = self._build_tree(right_X, right_y, depth + 1)
57
58     return (feature_index, value, left_tree, right_tree)
59
60     def predict(self, X):
61         return np.array([self._predict_one(x, self.tree) for x in X])
62
63     def _predict_one(self, x, tree):
64         if not isinstance(tree, tuple):
65             return tree # 叶子节点
66
67         feature_index, value, left_tree, right_tree = tree
68         if x[feature_index] <= value:
69             return self._predict_one(x, left_tree)
70         else:
71             return self._predict_one(x, right_tree)
72
73     # 测试
74     from sklearn.datasets import load_iris
75     X, y = load_iris(return_X_y=True)
76     tree = DecisionTree(max_depth=3)
77     tree.fit(X, y)
78     predictions = tree.predict(X)

```

朴素贝叶斯（Naive Bayes）分类器

朴素贝叶斯分类器基于贝叶斯定理，假设特征之间是独立的（这是“朴素”部分），并通过计算每个类别的条件概率来进行分类。它通常用于文本分类、垃圾邮件识别等问题。

```

1     import numpy as np
2
3     class NaiveBayesClassifier:
4     def fit(self, X, y):
5         # 获取所有类及其样本数量
6         self.classes, class_counts = np.unique(y, return_counts=True)
7         # 计算先验概率（各类出现的概率）
8         self.class_priors = class_counts / len(y)

```

```

9         self.n_features = X.shape[1]
10
11         # 使用拉普拉斯平滑法计算似然度
12         self.likelihoods = {}
13         for c in self.classes:
14             # 获取属于当前类别的样本
15             X_c = X[y == c]
16             # 计算条件概率 $P(X|C)$ ，加1平滑处理
17             self.likelihoods[c] = (X_c.sum(axis=0) + 1) / (X_c.shape[0] + 2)
18
19         def predict(self, X):
20             log_probs = np.zeros((X.shape[0], len(self.classes)))
21
22             # 对每个类别，计算其对测试数据的对数概率
23             for idx, c in enumerate(self.classes):
24                 # 计算类的对数先验概率
25                 log_priors = np.log(self.class_priors[idx])
26                 # 计算条件概率，X为特征，likelihoods为条件概率，使用对数计算
27                 conditional_probs = X @ np.log(self.likelihoods[c]) + (1 - X) @
28                 np.log(1 - self.likelihoods[c])
29                 log_probs[:, idx] = log_priors + conditional_probs
30
31             # 返回对数概率最大的类别作为预测结果
32             return self.classes[np.argmax(log_probs, axis=1)]
33
34         # 示例用法：
35         # 假设有训练数据`X_train`和标签`y_train`
36         nb = NaiveBayesClassifier()
37         nb.fit(X_train, y_train)
38         predictions = nb.predict(X_test)
39
40         # 打印预测结果
41         print("预测结果:", predictions)

```



注意:

- `X_train` 和 `X_test` 是 numpy 数组，行是样本，列是特征。
- `y_train` 是训练样本的标签，如果是二分类问题，其值为0或1。
- 这个实现假设输入特征是二值的。如果输入是连续值，需要做离散化或基于概率密度函数的处理。
- 以上代码使用了拉普拉斯平滑，以避免概率为零的情况。

K-means聚类算法

K-means是一种常用的无监督学习聚类算法，它通过最小化样本到各自簇的距离平方和，来将数据分成K个簇。算法通过迭代的方式更新簇的中心，直到簇中心不再发生变化为止。

```
1 class KMeans:
2     def __init__(self, k=3, max_iters=100, tol=1e-4):
3         self.k = k
4         self.max_iters = max_iters
5         self.tol = tol
6         self.centroids = None
7
8     def fit(self, X):
9         m, n = X.shape
10        # 随机初始化K个簇中心
11        self.centroids = X[np.random.choice(m, self.k, replace=False)]
12
13        for _ in range(self.max_iters):
14            # 为每个样本分配最近的簇中心
15            labels = np.array([self._assign_label(x) for x in X])
16            new_centroids = np.array([X[labels == i].mean(axis=0) for i in
17range(self.k)])
18
19            # 如果簇中心没有变化，停止迭代
20            if np.all(np.abs(new_centroids - self.centroids) < self.tol):
21                break
22            self.centroids = new_centroids
23
24        def _assign_label(self, x):
25            distances = [np.linalg.norm(x - centroid) for centroid in
26self.centroids]
27            return np.argmin(distances)
28
29        def predict(self, X):
30            return np.array([self._assign_label(x) for x in X])
31
32        # 测试
33        from sklearn.datasets import make_blobs
34        X, _ = make_blobs(n_samples=100, centers=3, random_state=42)
35        kmeans = KMeans(k=3)
36        kmeans.fit(X)
37        predictions = kmeans.predict(X)
```

主成分分析 (PCA)

PCA (Principal Component Analysis) 是一种常见的数据降维方法，它通过线性变换将数据转换为一组新的变量（主成分）。这些主成分按方差大小排序，前几个主成分可以保留数据中的大部分信息，从而达到降维的效果。

```
1  class PCA:
2      def __init__(self, n_components=2):
3          self.n_components = n_components
4          self.components = None
5          self.mean = None
6
7      def fit(self, X):
8          # 数据中心化
9          self.mean = np.mean(X, axis=0)
10         X_centered = X - self.mean
11
12         # 计算协方差矩阵
13         cov_matrix = np.cov(X_centered, rowvar=False)
14
15         # 计算特征值和特征向量
16         eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
17
18         # 按照特征值降序排列
19         sorted_indices = np.argsort(eigenvalues)[::-1]
20         eigenvectors = eigenvectors[:, sorted_indices]
21
22         # 选择前n个主成分
23         self.components = eigenvectors[:, :self.n_components]
24
25     def transform(self, X):
26         X_centered = X - self.mean
27         return np.dot(X_centered, self.components)
28
29 # 测试
30 from sklearn.datasets import make_blobs
31 X, _ = make_blobs(n_samples=100, centers=3, random_state=42)
32 pca = PCA(n_components=2)
33 pca.fit(X)
34 X_pca = pca.transform(X)
35
```

梯度提升树 (GBDT)

梯度提升树 (GBDT) 是一种集成学习方法, 它通过结合多棵弱决策树来形成强大的预测模型。GBDT通过迭代的方式逐步改进模型, 每一棵树都是在前一棵树的基础上减少误差的。

```
1 class GBDT:
2     def __init__(self, n_estimators=100, learning_rate=0.1):
3         self.n_estimators = n_estimators
4         self.learning_rate = learning_rate
5         self.trees = []
6
7     def fit(self, X, y):
8         y_pred = np.zeros_like(y, dtype=float)
9
10        for _ in range(self.n_estimators):
11            residuals = y - y_pred
12            tree = DecisionTree(max_depth=3)
13            tree.fit(X, residuals) # 训练树来拟合残差
14            self.trees.append(tree)
15            y_pred += self.learning_rate * tree.predict(X)
16
17    def predict(self, X):
18        y_pred = np.zeros(X.shape[0], dtype=float)
19        for tree in self.trees:
20            y_pred += self.learning_rate * tree.predict(X)
21        return y_pred
22
23 # 测试
24 from sklearn.datasets import make_regression
25 X, y = make_regression(n_samples=100, n_features=2, noise=0.1)
26 gbdt = GBDT(n_estimators=50, learning_rate=0.1)
27 gbdt.fit(X, y)
28 predictions = gbdt.predict(X)
29
```

线性回归的随机梯度下降 (stochastic gradient descent, SGD)

代码块

```
1 # 数据加载
2 from sklearn.datasets import fetch_california_housing
3 from sklearn.model_selection import train_test_split
4
5 X, Y = fetch_california_housing(return_X_y=True)
6 X.shape, Y.shape # (20640, 8), (20640, )
7
8 # 数据预处理
```

```

9  ones = np.ones(shape=(X.shape[0], 1))
10 X = np.hstack([X, ones])
11 validate_size = 0.2
12 X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
13 test_size=validate_size, shuffle=True)
14
15 # batch 函数
16 def get_batch(batchsize: int, X: np.ndarray, Y: np.ndarray):
17     assert 0 <= X.shape[0] % batchsize, f'{X.shape[0]} % {batchsize} != 0'
18     batchnum = X.shape[0] // batchsize
19     X_new = X.reshape((batchnum, batchsize, X.shape[1]))
20     Y_new = Y.reshape((batchnum, batchsize, ))
21
22     for i in range(batchnum):
23         yield X_new[i, :, :], Y_new[i, :]
24
25 # 损失函数
26 def mse(X: np.ndarray, Y: np.ndarray, W: np.ndarray):
27     return 0.5 * np.mean(np.square(X@W-Y))
28
29 def diff_mse(X: np.ndarray, Y: np.ndarray, W: np.ndarray):
30     return X.T@(X@W-Y) / X.shape[0]
31
32 # 模型训练
33 lr = 0.001 # 学习率
34 num_epochs = 1000 # 训练周期
35 batch_size = 64 # | 每个batch包含的样本数
36 validate_every = 4 # 多少个周期进行一次检验
37 def train(num_epochs: int, batch_size: int, validate_every: int, W0:
38 np.ndarray, X_train: np.ndarray, Y_train: np.ndarray, X_test: np.ndarray,
39 Y_test: np.ndarray):
40     loop = tqdm(range(num_epochs))
41     loss_train = []
42     loss_validate = []
43     W = W0
44     # 遍历epoch
45     for epoch in loop:
46         loss_train_epoch = 0
47         # 遍历batch
48         for x_batch, y_batch in get_batch(batch_size, X_train, Y_train):
49             loss_batch = mse(X=x_batch, Y=y_batch, W=W)
50             loss_train_epoch += loss_batch * x_batch.shape[0] / X_train.shape[0]
51             grad = diff_mse(X=x_batch, Y=y_batch, W=W)
52             W = W - lr * grad
53
54     loss_train.append(loss_train_epoch)
55     loop.set_description(f'Epoch: {epoch}, loss: {loss_train_epoch}')

```

```

53
54     if 0 == epoch%validate_every:
55         loss_validate_epoch = mse(X=X_test, Y=Y_test, W=W)
56         loss_validate.append(loss_validate_epoch)
57         print('=====Validate=====')
58         print(f'Epoch: {epoch}, train loss: {loss_train_epoch}, val loss:
{loss_validate_epoch}')
59         print('=====')
60         plot_loss(np.array(loss_train), np.array(loss_validate), validate_every)
61
62 # 程序运行
63 W0 = np.random.random(size=(X.shape[1], )) # 初始权重
64 train(num_epochs=num_epochs, batch_size=batch_size,
validate_every=validate_every, W0=W0, X_train=X_train, Y_train=Y_train,
X_test=X_test, Y_test=Y_test)

```

反向传播(backward propagation, BP)

BP算法就是反向传播，要输入的数据经过一个前向传播会得到一个输出，但是由于权重的原因，所以其输出会和你想要的输出有差距，这个时候就需要进行反向传播，利用梯度下降，对所有的权重进行更新，这样的话在进行前向传播就会发现其输出和你想要的输出越来越接近了。

代码块

```

1 # 生成权重以及偏执项layers_dim代表每层的神经元个数，
2 #比如[2,3,1]代表一个三层的网络，输入为2层，中间为3层输出为1层
3 def init_parameters(layers_dim):
4
5     L = len(layers_dim)
6     parameters = {}
7     for i in range(1,L):
8         parameters["w"+str(i)] = np.random.random([layers_dim[i],layers_dim[i-
1]])
9         parameters["b"+str(i)] = np.zeros((layers_dim[i],1))
10    return parameters
11
12 def sigmoid(z):
13     return 1.0/(1.0+np.exp(-z))
14
15 # sigmoid的导函数
16 def sigmoid_prime(z):
17     return sigmoid(z) * (1-sigmoid(z))
18
19 # 前向传播，需要用到一个输入x以及所有的权重以及偏执项，都在parameters这个字典里面存储
20 # 最后返回会返回一个caches里面包含的 是各层的a和z, a[layers]就是最终的输出
21 def forward(x,parameters):

```

```

22     a = []
23     z = []
24     caches = {}
25     a.append(x)
26     z.append(x)
27     layers = len(parameters)//2
28     # 前面都要用sigmoid
29     for i in range(1,layers):
30         z_temp =parameters["w"+str(i)].dot(x) + parameters["b"+str(i)]
31         z.append(z_temp)
32         a.append(sigmoid(z_temp))
33     # 最后一层不用sigmoid
34     z_temp = parameters["w"+str(layers)].dot(a[layers-1]) +
parameters["b"+str(layers)]
35     z.append(z_temp)
36     a.append(z_temp)
37
38     caches["z"] = z
39     caches["a"] = a
40     return caches,a[layers]
41
42 # 反向传播, parameters里面存储的是所有的各层的权重以及偏执, caches里面存储各层的a和z
43 # al是经过反向传播后最后一层的输出, y代表真实值
44 # 返回的grades代表着误差对所有的w以及b的导数
45 def backward(parameters,caches,al,y):
46     layers = len(parameters)//2
47     grades = {}
48     m = y.shape[1]
49     # 假设最后一层不经历激活函数
50     # 就是按照上面的图片中的公式写的
51     grades["dz"+str(layers)] = al - y
52     grades["dw"+str(layers)] = grades["dz"+str(layers)].dot(caches["a"][layers-
1]).T) /m
53     grades["db"+str(layers)] = np.sum(grades["dz"+str(layers)],axis =
1,keepdims = True) /m
54     # 前面全部都是sigmoid激活
55     for i in reversed(range(1,layers)):
56         grades["dz"+str(i)] =
parameters["w"+str(i+1)].T.dot(grades["dz"+str(i+1)]) *
sigmoid_prime(caches["z"][i])
57         grades["dw"+str(i)] = grades["dz"+str(i)].dot(caches["a"][i-1].T)/m
58         grades["db"+str(i)] = np.sum(grades["dz"+str(i)],axis = 1,keepdims =
True) /m
59     return grades
60
61 # 就是把其所有的权重以及偏执都更新一下
62 def update_grades(parameters,grades,learning_rate):

```

```

63     layers = len(parameters)//2
64     for i in range(1,layers+1):
65         parameters["w"+str(i)] -= learning_rate * grades["dw"+str(i)]
66         parameters["b"+str(i)] -= learning_rate * grades["db"+str(i)]
67     return parameters
68 # 计算误差值
69 def compute_loss(al,y):
70     return np.mean(np.square(al-y))
71
72 # 加载数据
73 def load_data():
74     """
75     加载数据集
76     """
77     x = np.arange(0.0,1.0,0.01)
78     y = 20* np.sin(2*np.pi*x)
79     # 数据可视化
80     plt.scatter(x,y)
81     return x,y
82 #进行测试
83 x,y = load_data()
84 x = x.reshape(1,100)
85 y = y.reshape(1,100)
86 plt.scatter(x,y)
87 parameters = init_parameters([1,25,1])
88 al = 0
89 for i in range(4000):
90     caches,al = forward(x, parameters)
91     grades = backward(parameters, caches, al, y)
92     parameters = update_grades(parameters, grades, learning_rate= 0.3)
93     if i %100 ==0:
94         print(compute_loss(al, y))
95 plt.scatter(x,al)
96 plt.show()

```

深度学习结构篇

全连接神经网络（MLP）

全连接神经网络（Multilayer Perceptron, MLP）是最基本的神经网络结构。它由多个层次组成，每一层的每个神经元与上一层的所有神经元都有连接，因此称为“全连接”。MLP通常用于分类和回归任务。

算法原理：

- **输入层：**接收数据输入。

- **隐藏层**：至少一层，使用激活函数进行非线性变换。
- **输出层**：最后输出预测值。
- **前向传播**：通过每一层的权重矩阵与输入数据进行乘积运算，并加上偏置项，最后通过激活函数计算输出。
- **反向传播**：通过误差函数计算误差，反向传播误差来调整权重和偏置

```
1  import numpy as np
2
3  class MLP:
4      def __init__(self, input_size, hidden_size, output_size):
5          self.W1 = np.random.randn(input_size, hidden_size)
6          self.b1 = np.zeros((1, hidden_size))
7          self.W2 = np.random.randn(hidden_size, output_size)
8          self.b2 = np.zeros((1, output_size))
9
10     def sigmoid(self, z):
11         return 1 / (1 + np.exp(-z))
12
13     def sigmoid_derivative(self, z):
14         return z * (1 - z)
15
16     def forward(self, X):
17         self.z1 = np.dot(X, self.W1) + self.b1
18         self.a1 = self.sigmoid(self.z1)
19         self.z2 = np.dot(self.a1, self.W2) + self.b2
20         self.a2 = self.sigmoid(self.z2)
21         return self.a2
22
23     def backward(self, X, y, output):
24         output_error = y - output
25         output_delta = output_error * self.sigmoid_derivative(output)
26
27         hidden_error = output_delta.dot(self.W2.T)
28         hidden_delta = hidden_error * self.sigmoid_derivative(self.a1)
29
30         self.W2 += self.a1.T.dot(output_delta)
31         self.b2 += np.sum(output_delta, axis=0, keepdims=True)
32         self.W1 += X.T.dot(hidden_delta)
33         self.b1 += np.sum(hidden_delta, axis=0)
34
35     def train(self, X, y, epochs=10000, lr=0.1):
36         for epoch in range(epochs):
37             output = self.forward(X)
38             self.backward(X, y, output)
```

```

39
40 # 示例用法
41 if __name__ == "__main__":
42     # 训练集: 4个样本, 2个特征
43     X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
44     # 标签
45     y = np.array([[0], [1], [1], [0]]) # XOR
46
47     nn = MLP(input_size=2, hidden_size=2, output_size=1)
48     nn.train(X, y)
49
50     print("预测结果:")
51     print(nn.forward(X))

```

🌱 说明

- 这是一个简单的MLP用于解决XOR问题。
- `input_size` 是输入特征的数量; `hidden_size` 是隐藏层的神经元数量; `output_size` 是输出的维度。
- `train` 方法用于训练模型, `epochs` 是训练轮数, `lr` 是学习率 (在这个实现中未使用)。

卷积神经网络 (CNN)

卷积神经网络 (Convolutional Neural Network, CNN) 是深度学习中用于图像处理的经典神经网络结构。CNN通过卷积层提取局部特征, 池化层减少计算量和特征维度, 从而适合处理大规模的图像数据。

算法原理:

- **卷积层**: 通过卷积操作提取图像的局部特征。
- **池化层**: 通过池化操作进行降维, 减少计算量和防止过拟合。
- **全连接层**: 将卷积和池化层提取的特征映射展平, 进行分类任务

```

1 # 导入必要的库
2 import tensorflow as tf
3 from tensorflow.keras import layers, models
4 import matplotlib.pyplot as plt
5
6 # 加载MNIST数据集
7 mnist = tf.keras.datasets.mnist
8 (train_images, train_labels), (test_images, test_labels) = mnist.load_data()

```

```
9
10 # 预处理数据
11 train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') /
255
12 test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255
13
14 # 编码标签
15 train_labels = tf.keras.utils.to_categorical(train_labels)
16 test_labels = tf.keras.utils.to_categorical(test_labels)
17
18 # 构建CNN模型
19 model = models.Sequential()
20 model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28,
1)))
21 model.add(layers.MaxPooling2D((2, 2)))
22 model.add(layers.Conv2D(64, (3, 3), activation='relu'))
23 model.add(layers.MaxPooling2D((2, 2)))
24 model.add(layers.Conv2D(64, (3, 3), activation='relu'))
25
26 model.add(layers.Flatten())
27 model.add(layers.Dense(64, activation='relu'))
28 model.add(layers.Dense(10, activation='softmax'))
29
30 # 打印模型结构
31 model.summary()
32
33 # 编译模型
34 model.compile(optimizer='adam',
35               loss='categorical_crossentropy',
36               metrics=['accuracy'])
37
38 # 训练模型
39 history = model.fit(train_images, train_labels, epochs=5,
40                    validation_data=(test_images, test_labels))
41
42 # 评估模型
43 test_loss, test_acc = model.evaluate(test_images, test_labels)
44 print(f'Test accuracy: {test_acc:.4f}')
45
46 # 绘制训练和验证的准确率
47 plt.plot(history.history['accuracy'], label='accuracy')
48 plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
49 plt.xlabel('Epoch')
50 plt.ylabel('Accuracy')
51 plt.ylim([0, 1])
52 plt.legend(loc='lower right')
53 plt.show()
```

🌱 说明

1. **数据准备：**从Keras加载了MNIST数据集，并将图片重新调整为28x28x1的形状，将像素值缩放到0到1之间。
2. **模型结构：**
 - 三个卷积层，每个卷积层后接一个池化层。
 - 最后是平坦化层和两个全连接层，其中最后的密集层使用softmax激活函数以输出10个类别的概率。
3. **训练和评估：**
 - 使用Adam优化器和交叉熵损失函数进行编译。
 - 训练模型5个周期，并在测试集上评估模型的性能。
 - 绘制训练过程中的准确率变化。

循环神经网络（RNN）

循环神经网络（Recurrent Neural Network，RNN）适用于处理序列数据，如时间序列分析、语音识别、自然语言处理等。RNN的特点是能够在序列中共享参数，并通过其隐藏层传递信息。

算法原理：

- **隐藏层循环：**每个时间步的输入不仅依赖于当前的输入，还依赖于上一个时间步的隐藏状态。
- **前向传播：**输入当前时间步的信号，同时传递上一个时间步的隐藏状态。

```
1  # 导入所需库
2  import numpy as np
3  import tensorflow as tf
4  from tensorflow.keras.models import Sequential
5  from tensorflow.keras.layers import SimpleRNN, Dense
6  import matplotlib.pyplot as plt
7
8  # 生成正弦波数据
9  x = np.linspace(0, 100, 1000)
10 y = np.sin(x)
11
12 # 创建输入和输出序列
13 def create_dataset(y, time_step=10):
14     dataX, dataY = [], []
15     for i in range(len(y) - time_step - 1):
16         a = y[i:(i + time_step)]
17         dataX.append(a)
```

```

18         dataY.append(y[i + time_step])
19     return np.array(dataX), np.array(dataY)
20
21     time_step = 10
22     dataX, dataY = create_dataset(y, time_step)
23
24     # 重塑输入数据以符合RNN的输入格式 (samples, time steps, features)
25     dataX = dataX.reshape((dataX.shape[0], dataX.shape[1], 1))
26
27     # 构建RNN模型
28     model = Sequential()
29     model.add(SimpleRNN(50, activation='relu', input_shape=(time_step, 1)))
30     model.add(Dense(1))
31
32     # 编译模型
33     model.compile(optimizer='adam', loss='mse')
34
35     # 训练模型
36     model.fit(dataX, dataY, epochs=100, batch_size=64, verbose=1)
37
38     # 使用模型进行预测
39     train_predict = model.predict(dataX)
40
41     # 绘制结果
42     plt.figure(figsize=(10,6))
43     plt.plot(y, label='True Sine Wave')
44     plt.plot(np.arange(time_step, len(train_predict) + time_step), train_predict,
45              label='RNN Predictions')
46     plt.xlabel('Index')
47     plt.ylabel('Value')
48     plt.title('RNN Sine Wave Prediction')
49     plt.legend()
50     plt.show()

```

说明

a. 数据生成:

- 在这个例子中，我们生成了一个正弦波序列数据 `y`，并将其分成输入序列 `dataX` 和预测目标 `dataY`。
- 使用 `create_dataset` 函数根据给定的时间步长创建这两个序列。

b. 数据预处理:

- `dataX` 被重塑为3D数组 `(samples, time steps, features)`，以适合RNN的输入格式。

c. 模型构建:

- 使用 `Sequential` 模型和 `SimpleRNN` 层创建一个简单的RNN模型。
- RNN层具有50个神经元, 并使用ReLU激活函数。

d. 模型训练:

- 模型使用均方误差损失和Adam优化器进行编译和训练。

e. 结果可视化:

- 使用matplotlib绘制真实的正弦波和RNN的预测结果以进行比较。

图神经网络 (GNN)

图神经网络 (Graph Neural Network, GNN) 是一类专门处理图数据结构的神经网络模型。它能够通过图结构中节点和边的信息进行节点分类、图分类等任务。

算法原理:

- **消息传递机制:** 通过节点之间的信息传递来更新节点的状态。
- **邻接矩阵:** 定义图的结构, 决定哪些节点是相邻的。
- **聚合函数:** 聚合邻居节点的信息来更新当前节点的状态。

```
1 import torch
2 import torch.nn.functional as F
3 from torch_geometric.nn import GCNConv
4 from torch_geometric.data import Data
5
6 class GCN(torch.nn.Module):
7     def __init__(self, num_node_features, hidden_channels, num_classes):
8         super(GCN, self).__init__()
9         # 定义两层GCN
10        self.conv1 = GCNConv(num_node_features, hidden_channels)
11        self.conv2 = GCNConv(hidden_channels, num_classes)
12
13    def forward(self, x, edge_index):
14        # 第一层卷积和激活函数
15        x = self.conv1(x, edge_index)
16        x = F.relu(x)
17
18        # 第二层卷积
19        x = self.conv2(x, edge_index)
20
21        return F.log_softmax(x, dim=1)
22
23 # 假设我们有一个简单的图数据
```

```

24 edge_index = torch.tensor([[0, 1, 1, 2],
25                             [1, 0, 2, 1]], dtype=torch.long) # 4条边
26 x = torch.tensor([[1], [0], [1]], dtype=torch.float) # 3个节点特征
27
28 # 创建一个图数据对象
29 data = Data(x=x, edge_index=edge_index)
30
31 # 模型参数
32 num_node_features = data.num_node_features
33 hidden_channels = 16
34 num_classes = 2
35
36 # 初始化模型
37 model = GCN(num_node_features, hidden_channels, num_classes)
38
39 # 向前传播 (示例)
40 out = model(data.x, data.edge_index)
41 print(out)

```

🌱 说明

- **数据输入:** 我们使用 `torch_geometric.data.Data` 来定义图结构，其中包括节点特征 `x` 和边的信息 `edge_index`。
- **模型结构:** `GCNConv` 是PyTorch Geometric中的卷积层，`forward` 函数中定义了在上图进行的两次卷积操作。第一层卷积激活使用ReLU函数。
- **输出:** 最终输出是每个节点的分类分数，通过 `log_softmax` 可以得到对数概率分布。

Batch Normalization

批标准化 (Batch Normalization, BN) 是一种提高神经网络训练速度和稳定性的技术。它通过对每一层的输入进行标准化处理，使得数据的分布更加稳定。

算法原理:

- **标准化:** 对每一层的输入进行均值为0、方差为1的标准化。
- **学习参数:** 引入可学习的缩放因子和偏置项，允许网络重新调整标准化后的数据分布。

```

1 class MyBN:
2     def __init__(self, momentum=0.01, eps=1e-5, feat_dim=2):
3         """
4         初始化参数值
5         :param momentum: 动量，用于计算每个batch均值和方差的滑动均值

```

```

6         :param eps: 防止分母为0
7         :param feat_dim: 特征维度
8         """
9         # 均值和方差的滑动均值
10        self._running_mean = np.zeros(shape=(feat_dim, ))
11        self._running_var = np.ones((shape=(feat_dim, ))
12        # 更新self._running_xxx时的动量
13        self._momentum = momentum
14        # 防止分母计算为0
15        self._eps = eps
16        # 对应Batch Norm中需要更新的beta和gamma, 采用pytorch文档中的初始化值
17        self._beta = np.zeros(shape=(feat_dim, ))
18        self._gamma = np.ones(shape=(feat_dim, ))
19
20    def batch_norm(self, x):
21        """
22        BN向传播
23        :param x: 数据
24        :return: BN输出
25        """
26        if self.training:
27            x_mean = x.mean(axis=0)
28            x_var = x.var(axis=0)
29            # 对应running_mean的更新公式
30            self._running_mean = (1-self._momentum)*x_mean +
self._momentum*self._running_mean
31            self._running_var = (1-self._momentum)*x_var +
self._momentum*self._running_var
32            # 对应论文中计算BN的公式
33            x_hat = (x-x_mean)/np.sqrt(x_var+self._eps)
34        else:
35            x_hat = (x-
self._running_mean)/np.sqrt(self._running_var+self._eps)
36        return self._gamma*x_hat + self._beta

```

🌿 说明

- `batch_norm_forward` : 实现了 Batch Normalization 的前向传播。
- `batch_norm_backward` : 实现了 Batch Normalization 的反向传播。
- `eps` : 是一个小的值, 用于避免分母为零的情况。

手撕单头注意力机制 (ScaledDotProductAttention)

输入是query和 key-value, 注意力机制首先计算query与每个key的关联性(compatibility), 每个关联性作为每个value的权重(weight), 各个权重与value的乘积相加得到输出。

代码块

```
1 class ScaledDotProductAttention(nn.Module):
2     """ Scaled Dot-Product Attention """
3
4     def __init__(self, scale):
5         super().__init__()
6
7         self.scale = scale
8         self.softmax = nn.Softmax(dim=2)
9
10    def forward(self, q, k, v, mask=None):
11        u = torch.bmm(q, k.transpose(1, 2)) # 1.Matmul
12        u = u / self.scale # 2.Scale
13
14        if mask is not None:
15            u = u.masked_fill(mask, -np.inf) # 3.Mask
16
17        attn = self.softmax(u) # 4.Softmax
18        output = torch.bmm(attn, v) # 5.Output
19
20        return attn, output
21
22
23    if __name__ == "__main__":
24        n_q, n_k, n_v = 2, 4, 4
25        d_q, d_k, d_v = 128, 128, 64
26
27        q = torch.randn(batch, n_q, d_q)
28        k = torch.randn(batch, n_k, d_k)
29        v = torch.randn(batch, n_v, d_v)
30        mask = torch.zeros(batch, n_q, n_k).bool()
31
32        attention = ScaledDotProductAttention(scale=np.power(d_k, 0.5))
33        attn, output = attention(q, k, v, mask=mask)
34
35        print(attn)
36        print(output)
```

自注意力机制 (Self-Attention)

以下是一个简单的自注意力机制（Self-Attention）在Python中的实现。这里我们使用了NumPy库来进行矩阵运算。这只是一个最基础的自注意力机制实现，真实世界中的实现可能要复杂得多，尤其是在使用深度学习框架（如TensorFlow或PyTorch）时。

```
1  import numpy as np
2
3  def softmax(x):
4      """计算每组得分的softmax值"""
5      e_x = np.exp(x - np.max(x, axis=-1, keepdims=True)) # 防止溢出, 减去最大值
6      return e_x / np.sum(e_x, axis=-1, keepdims=True) # 归一化
7
8  def self_attention(Q, K, V, mask=None):
9      """
10     实现自注意力机制
11
12     Q, K, V: 查询矩阵、键矩阵和值矩阵
13     mask: 可选的掩码, 用于忽略某些位置 (例如在Transformer中)
14
15     返回:
16     Attention: 应用自注意力机制后的结果
17     """
18     # 计算缩放点积注意力
19     dk = K.shape[-1] # 键的维度
20     scores = np.matmul(Q, K.transpose(-2, -1)) / np.sqrt(dk) # 计算Q与K的点积,
    并进行缩放
21
22     if mask is not None:
23         scores += (mask * -1e9) # 如果给定了mask, 应用mask
24
25     attention_weights = softmax(scores) # 对分数进行softmax计算得到注意力权重
26     output = np.matmul(attention_weights, V) # 根据注意力权重加权值矩阵V
27     return output
28
29 # 示例矩阵
30 np.random.seed(0)
31 Q = np.random.rand(3, 4) # 查询矩阵: 3个查询, 维度为4
32 K = np.random.rand(3, 4) # 键矩阵: 3个键, 维度为4
33 V = np.random.rand(3, 4) # 值矩阵: 3个值, 维度为4
34
35 # 计算自注意力
36 attention_output = self_attention(Q, K, V)
37 print("注意力输出:\n", attention_output)
38
```

说明

a. 输入矩阵:

- Q (Query)、K (Key)、V (Value) 都是随机生成的示例矩阵。现实中，这些矩阵来源于输入序列经过线性变换后得到。

b. 缩放点积注意力:

- 首先计算 QK^T ，然后缩放 $\frac{1}{\sqrt{d_k}}$ ，其中 d_k 是键向量的维度。
- 应用Softmax函数来获得注意力权重，确保权重在0到1之间并且总和为1。

c. 输出计算:

- 将注意力权重与V矩阵相乘得到最终的输出。

层归一化 (Layer Normalization)



Layer Norm 计算原理

Layer Norm 对每个样本的所有特征维度进行归一化，使其均值为 0，方差为 1，保持不同特征间的相对比例。

数学公式:

$$\mu = E(X) = \frac{1}{H} \sum_{i=1}^H x_i$$

$$\sigma = \sqrt{\text{Var}(X)} = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2 + \epsilon}$$

$$y = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

其中:

- H 为特征维度 (embedding 维度)。
- γ 和 β 是可学习的缩放和偏移参数。

下面是一个使用Python实现的层归一化 (Layer Normalization) 的代码示例。使用NumPy库进行计算。层归一化是一种正则化技术，用于神经网络中，以改善训练的稳定性。

```
1 import torch
2 import torch.nn as nn
3
```

```

4 class LayerNorm(nn.Module):
5     def __init__(self, d_model, eps=1e-6):
6         """
7         初始化 LayerNorm 模块。
8
9         参数:
10        d_model: 输入张量的最后一个维度的大小。
11        eps: 防止除零错误的一个小常数。
12        """
13        super(LayerNorm, self).__init__()
14        self.gamma = nn.Parameter(torch.ones(d_model)) # 缩放参数
15        self.beta = nn.Parameter(torch.zeros(d_model)) # 平移参数
16        self.eps = eps
17
18    def forward(self, x):
19        """
20        前向传播函数。
21
22        参数:
23        x: 输入张量, 形状为 [batch_size, seq_len, d_model]。
24
25        返回:
26        归一化后的张量, 形状同输入张量。
27        """
28        mean = x.mean(-1, keepdim=True) # 计算均值
29        std = x.std(-1, keepdim=True) # 计算标准差
30        x_norm = (x - mean) / (std + self.eps) # 标准化
31        return self.gamma * x_norm + self.beta # 缩放和平移
32
33 # 测试 LayerNorm
34 x = torch.randn(2, 3, 4) # 示例输入张量
35 layer_norm = LayerNorm(4)
36 output = layer_norm(x)
37 print("LayerNorm Output:")
38 print(output)
39

```

层归一化 (RMSNorm)



- **核心思想**: 去掉均值计算, 只进行方差归一化 (使用均方根 (RMS) 而不是标准差来进行归一化), 加速计算。
- **公式**:

$$\text{RMS}(x) = \sqrt{\frac{1}{H} \sum_{i=1}^H x_i^2}$$

$$x = \frac{x}{\text{RMS}(x)} \cdot \gamma$$

- 优点：
 - 计算更高效，适用于大规模模型。
 - 在很多 NLP 任务中，效果与 Layer Norm 相近。
- 缺点：
 - 由于去除了均值计算，对某些分布的输入可能不稳定。

代码块

```
1 class RMSNorm(nn.Module):
2     def __init__(self, d_model, eps=1e-6):
3         """
4         初始化 RMSNorm 模块。
5
6         参数:
7         d_model: 输入张量的最后一个维度的大小。
8         eps: 防止除零错误的一个小常数。
9         """
10        super(RMSNorm, self).__init__()
11        self.gamma = nn.Parameter(torch.ones(d_model)) # 缩放参数
12        self.eps = eps
13
14        def forward(self, x):
15            """
16            前向传播函数。
17
18            参数:
19            x: 输入张量, 形状为 [batch_size, seq_len, d_model]。
20
21            返回:
22            归一化后的张量, 形状同输入张量。
23            """
24            rms = torch.sqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) # 计算均方
根
25            x_norm = x / rms # 标准化
26            return self.gamma * x_norm # 缩放
27
28 # 测试 RMSNorm
29 x = torch.randn(2, 3, 4) # 示例输入张量
30 rms_norm = RMSNorm(4)
```

```
31 output = rms_norm(x)
32 print("RMSNorm Output:")
33 print(output)
```

位置编码 (Positional Encoding)

位置编码 (Positional Encoding) 是Transformer模型中用于表示序列中元素的相对位置信息的重要部分。下面是一个简单的Python实现，用于计算位置编码：

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def get_positional_encoding(seq_len, d_model):
5     """
6     计算位置编码。
7
8     参数:
9     seq_len -- 序列的长度
10    d_model -- 嵌入的维度大小
11
12    返回:
13    pos_enc -- 位置编码矩阵, 形状为 (seq_len, d_model)
14    """
15    pos_enc = np.zeros((seq_len, d_model))
16    for pos in range(seq_len):
17        for i in range(0, d_model, 2):
18            pos_enc[pos, i] = np.sin(pos / (10000 ** (i / d_model)))
19            if i + 1 < d_model:
20                pos_enc[pos, i + 1] = np.cos(pos / (10000 ** ((i + 1) /
21                d_model)))
22    return pos_enc
23
24 # 示例: 生成序列长度为50, 嵌入维度为512的位置信息
25 seq_len = 50
26 d_model = 512
27 pos_encoding = get_positional_encoding(seq_len, d_model)
28
29 # 可视化位置编码
30 plt.figure(figsize=(10, 10))
31 plt.imshow(pos_encoding, cmap='viridis')
32 plt.title('Positional Encoding')
33 plt.xlabel('Embedding Dimension')
34 plt.ylabel('Position in Sequence')
35 plt.colorbar()
36 plt.show()
```

此代码实现了位置编码矩阵，该矩阵通过对序列长度 `seq_len` 和嵌入维度 `d_model` 进行循环计算而得。偶数纬度使用 `sin` 函数编码，奇数纬度使用 `cos` 函数编码。随后，生成的矩阵可以被可视化以便更好地理解其行为。

One-Hot 代码实现

 OneHot编码是一种将词语转换为数字向量的方式，其中每个词都会被表示为一个向量，**在该向量中，只有一个位置是1，其他位置都是0**。这个向量的长度等于词汇表中词语的总数，每个位置对应一个特定的词

🔥 示例：

假设我们有一个词汇表：

代码块

```
1 vocab = ["apple", "banana", "cherry"]
```

One-Hot 编码后：

- "apple" → [1, 0, 0]
- "banana" → [0, 1, 0]
- "cherry" → [0, 0, 1]

🐸 代码实现：

代码块

```
1 # 1 定义词汇表 (Vocabulary)
2 vocab = ["apple", "banana", "cherry"] # 词汇表，包含 3 个单词
3
4 # 2 构造 "单词 -> 索引" 的映射字典
5 # 例如: {"apple": 0, "banana": 1, "cherry": 2}
6 word_to_index = {word: idx for idx, word in enumerate(vocab)}
7
8 # 3 定义 One-Hot 编码函数
9 def one_hot_encoding(word, vocab, word_to_index):
10     """
11     将单词转换为 One-Hot 编码
12
13     参数：
14     - word: 需要转换的单词 (字符串)
15     - vocab: 词汇表 (列表)
```

```

16     - word_to_index: 单词到索引的映射字典
17
18 返回:
19     - 一个与词汇表等长的 One-Hot 向量 (列表)
20     ""
21     # 创建一个长度等于词汇表大小的全 0 向量
22     encoding = [0] * len(vocab)
23
24     # 🔍 获取单词对应的索引
25     idx = word_to_index.get(word, -1) # 若单词不存在, 返回 -1
26
27     # 如果单词在词汇表中, 更新对应位置为 1
28     if idx != -1:
29         encoding[idx] = 1 # 例如 "banana" 在索引 1, 则 encoding = [0, 1, 0]
30
31     return encoding
32
33 # 4️⃣ 测试 One-Hot 编码
34 print("One-Hot 编码结果:")
35 for word in vocab:
36     print(f'{word}': {one_hot_encoding(word, vocab, word_to_index)}")
37

```

输出结果:

代码块

```

1  '🍏 apple': [1, 0, 0]
2  '🍌 banana': [0, 1, 0]
3  '🍒 cherry': [0, 0, 1]

```

CBOW (Continuous Bag-of-Words)

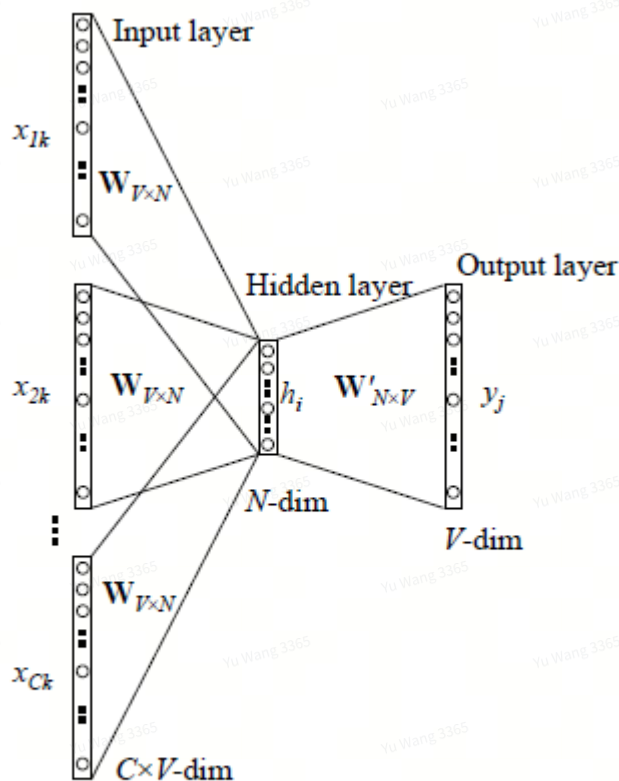


Figure 2: Continuous bag-of-words model

目标： 通过上下文词语（前后几个词）来预测当前目标词。

适用于： 词汇较小、数据量较大的情况。

例子：

输入（上下文）： "I am __ a student"

预测（目标词）： "not"

◆ CBOW 代码实现

代码块

```
1 import torch
2 import torch.nn as nn # 导入神经网络模块
3
4 class CBOW(nn.Module):
5     def __init__(self, voc_size, embedding_size):
6         super(CBOW, self).__init__()
7         # 定义从 One-Hot 词向量到嵌入向量的映射
8         self.input_to_hidden = nn.Linear(voc_size, embedding_size, bias=False)
9
10        # 定义从嵌入向量到输出单词分布的映射
11        self.hidden_to_output = nn.Linear(embedding_size, voc_size,
12        bias=False)
13
14    def forward(self, X):
15        """
```

```

14 前向传播:
15 X: [num_context_words, voc_size] # 输入为多个上下文词的 One-Hot 表示
16 """"
17 embeddings = self.input_to_hidden(X) # 生成嵌入向量 [num_context_words,
embedding_size]
18 hidden_layer = torch.mean(embeddings, dim=0) # 对多个嵌入向量求均值
19 output_layer = self.hidden_to_output(hidden_layer.unsqueeze(0)) # 计算
最终输出
20 return output_layer
21
22 # 设定参数
23 voc_size = 10 # 词汇表大小
24 embedding_size = 2 # 设定嵌入层的大小
25
26 # 创建 CBOW 模型
27 cbow_model = CBOW(voc_size, embedding_size)
28 print("CBOW 模型结构: ", cbow_model)

```

Skip-gram

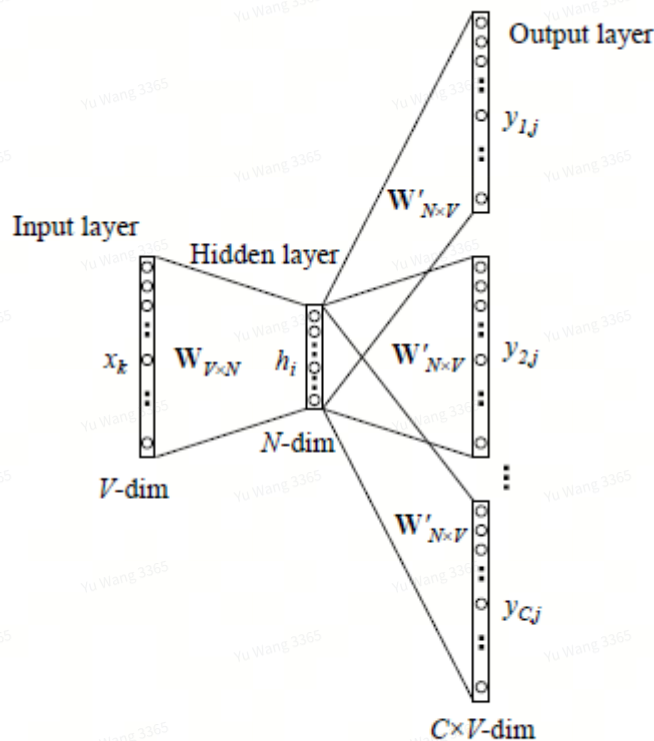


Figure 3: The skip-gram model.

目标： 通过单词预测其上下文，即与 CBOW 任务相反。

适用于： 词汇较大、数据量较小的情况。

例子：

输入（当前词）： "not"

预测 (上下文): "I am __ a student"

◆ Skip-gram 代码实现

代码块

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 # 设定设备
6 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
7
8 class Skip_gram(nn.Module):
9     def __init__(self, voc_size, embedding_size):
10         super(Skip_gram, self).__init__()
11         # W: 从 One-Hot 词向量映射到嵌入空间
12         self.W = nn.Parameter(torch.randn(voc_size, embedding_size).to(device))
13         # V: 输出层的参数
14         self.V = nn.Parameter(torch.randn(embedding_size, voc_size).to(device))
15
16     def forward(self, X):
17         """
18         前向传播:
19         X: [batch_size, voc_size] # 输入为多个单词的 One-Hot 编码
20         """
21         hidden_layer = torch.matmul(X, self.W) # 计算隐藏层 [batch_size, embedding_size]
22         output_layer = torch.matmul(hidden_layer, self.V) # 计算输出层 [batch_size, voc_size]
23         return output_layer
24
25 # 设定参数
26 voc_size = 10 # 词汇表大小
27 embedding_size = 2 # 设定嵌入层的大小
28
29 # 实例化模型
30 model = Skip_gram(voc_size, embedding_size).to(device)
31 criterion = nn.CrossEntropyLoss().to(device) # 交叉熵损失函数
32 optimizer = optim.Adam(model.parameters(), lr=1e-3) # Adam 优化器
33
```

Transformer模型

实现一个完整的Transformer模型需要处理多个部分,包括多头自注意力机制、前馈神经网络、嵌入层、位置编码等。下面是一个简化版的Transformer模型实现,其中不包括训练过程和超参数调节。

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 # 缩放点积注意力 (Scaled Dot-Product Attention)
7 def scaled_dot_product_attention(query, key, value, mask=None):
8     d_k = query.size(-1)
9     scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k) # 计算注意力得分
10    if mask is not None:
11        scores = scores.masked_fill(mask == 0, -1e9) # 如果有mask, 则将不必要的位置填充为负无穷
12    weights = F.softmax(scores, dim=-1) # 对得分进行softmax归一化, 得到注意力权重
13    output = torch.matmul(weights, value) # 用注意力权重加权值
14    return output, weights
15
16 # 多头注意力模块 (Multi-Head Attention)
17 class MultiHeadAttention(nn.Module):
18     def __init__(self, d_model, num_heads):
19         super(MultiHeadAttention, self).__init__()
20         assert d_model % num_heads == 0 # 确保模型的维度能被头数整除
21         self.d_k = d_model // num_heads # 每个头的维度
22         self.num_heads = num_heads
23
24         self.q_linear = nn.Linear(d_model, d_model) # 查询 (Query) 的线性变换
25         self.k_linear = nn.Linear(d_model, d_model) # 键 (Key) 的线性变换
26         self.v_linear = nn.Linear(d_model, d_model) # 值 (Value) 的线性变换
27         self.out_linear = nn.Linear(d_model, d_model) # 输出的线性变换
28
29     def forward(self, query, key, value, mask=None):
30         batch_size = query.size(0)
31
32         # 进行线性变换
33         query = self.q_linear(query).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
34         key = self.k_linear(key).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
35         value = self.v_linear(value).view(batch_size, -1, self.num_heads, self.d_k).transpose(1, 2)
36
37         # 应用注意力机制
38         x, attn_weights = scaled_dot_product_attention(query, key, value, mask)
39
40         # 连接多头结果并应用最终的线性层
41         x = x.transpose(1, 2).contiguous().view(batch_size, -1, self.d_model)

```

```

42         x = self.out_linear(x)
43
44     return x, attn_weights
45
46 # 位置前馈网络 (Position-wise Feed-Forward Network)
47 class FeedForward(nn.Module):
48     def __init__(self, d_model, d_ff):
49         super(FeedForward, self).__init__()
50         self.linear1 = nn.Linear(d_model, d_ff) # 第一层线性变换
51         self.linear2 = nn.Linear(d_ff, d_model) # 第二层线性变换
52
53     def forward(self, x):
54         x = self.linear1(x) # 通过第一层
55         x = F.relu(x) # 激活函数ReLU
56         x = self.linear2(x) # 通过第二层
57         return x
58
59 # 位置编码 (Positional Encoding)
60 class PositionalEncoding(nn.Module):
61     def __init__(self, d_model, max_len=5000):
62         super(PositionalEncoding, self).__init__()
63         pe = torch.zeros(max_len, d_model) # 初始化位置编码矩阵
64         position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
65         # 位置矩阵
66         div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-
67             math.log(10000.0) / d_model)) # 分母项
68         pe[:, 0::2] = torch.sin(position * div_term) # 偶数位置的sin编码
69         pe[:, 1::2] = torch.cos(position * div_term) # 奇数位置的cos编码
70         pe = pe.unsqueeze(0).transpose(0, 1) # 形状调整
71         self.register_buffer('pe', pe) # 注册为buffer, 不会作为模型参数更新
72
73     def forward(self, x):
74         x = x + self.pe[:x.size(0), :] # 将位置编码加到输入的embedding中
75         return x
76
77 # Transformer编码器层 (Transformer Encoder Layer)
78 class TransformerEncoderLayer(nn.Module):
79     def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
80         super(TransformerEncoderLayer, self).__init__()
81         self.self_attn = MultiHeadAttention(d_model, num_heads) # 多头自注意力
82         self.ff = FeedForward(d_model, d_ff) # 前馈网络
83         self.norm1 = nn.LayerNorm(d_model) # 层归一化1
84         self.norm2 = nn.LayerNorm(d_model) # 层归一化2
85         self.dropout = nn.Dropout(dropout) # Dropout层
86
87     def forward(self, x, mask=None):
88         # 自注意力

```

```

87         attn_output, _ = self.self_attn(x, x, x, mask)
88         x = x + self.dropout(attn_output) # 残差连接
89         x = self.norm1(x) # 第一层归一化
90         # 前馈网络
91         ff_output = self.ff(x)
92         x = x + self.dropout(ff_output) # 残差连接
93         x = self.norm2(x) # 第二层归一化
94         return x
95
96     # Transformer编码器 (Transformer Encoder)
97     class TransformerEncoder(nn.Module):
98         def __init__(self, d_model, num_heads, d_ff, num_layers, vocab_size,
99             max_len):
100             super(TransformerEncoder, self).__init__()
101             self.embedding = nn.Embedding(vocab_size, d_model) # 词嵌入层
102             self.pos_encoder = PositionalEncoding(d_model, max_len) # 位置编码层
103             self.layers = nn.ModuleList([TransformerEncoderLayer(d_model,
104                 num_heads, d_ff) for _ in range(num_layers)]) # 编码器层
105             self.d_model = d_model
106
107         def forward(self, src, mask=None):
108             src = self.embedding(src) * math.sqrt(self.d_model) # 词嵌入和缩放
109             src = self.pos_encoder(src) # 添加位置编码
110             for layer in self.layers:
111                 src = layer(src, mask) # 逐层传递输入
112             return src

```

🌱 说明

1. **Multi-Head Attention & Scaled Dot-Product Attention**: 多头注意力机制可以让模型从不同的表示子空间中关注不同的位置。而缩放点积注意力是其核心计算步骤。
2. **Feed Forward Network**: 每个位置的向量都经过一个前馈网络，处理相应位置的信息。
3. **Positional Encoding**: 用于给定序列中的每个位置编码位置相关的信息，以保留序列的顺序信息。
4. **Layer Normalization & Dropout**: 分别用于改善模型的训练稳定性和防止过拟合。
5. **Transformer Encoder Layer**: 每一层由一个多头注意力和一个前馈网络组成。
6. **Transformer Encoder**: 由多个编码器层和嵌入、位置编码组成，最终实现对输入序列的编码转换。

在实际应用中，你可以通过定义 `src_mask` 来处理序列中实际的数据部分和填充（即 padding）部分。这段代码省略了 decoder 部分等组件的实现，关注点在于理解 Transformer

的核心结构。在生产环境中，还需对模型进行更多的优化，包括更复杂的训练过程、参数微调、以及错误和异常处理。

Multi-Head Attention (MHA)

下面是一个实现多头注意力机制 (Multi-Head Attention, MHA) 的 Python 代码示例。为了实现这一功能，首先我们需要定义基本的注意力机制，然后基于此构建多头注意力。

```
1  import torch
2  from torch import nn
3  class MutiHeadAttention(torch.nn.Module):
4      def __init__(self, hidden_size, num_heads):
5          super(MutiHeadAttention, self).__init__()
6          self.num_heads = num_heads
7          self.head_dim = hidden_size // num_heads
8
9          ## 初始化Q、K、V投影矩阵
10         self.q_linear = nn.Linear(hidden_size, hidden_size)
11         self.k_linear = nn.Linear(hidden_size, hidden_size)
12         self.v_linear = nn.Linear(hidden_size, hidden_size)
13         ## 输出线性层
14         self.o_linear = nn.Linear(hidden_size, hidden_size)
15
16         def forward(self, hidden_state, attention_mask=None):
17             batch_size = hidden_state.size()[0]
18
19             query = self.q_linear(hidden_state)
20             key = self.k_linear(hidden_state)
21             value = self.v_linear(hidden_state)
22
23             query = self.split_head(query)
24             key = self.split_head(key)
25             value = self.split_head(value)
26
27             ## 计算注意力分数
28             attention_scores = torch.matmul(query, key.transpose(-1, -2)) /
torch.sqrt(torch.tensor(self.head_dim))
29
30             if attention_mask != None:
31                 attention_scores += attention_mask * -1e9
32             ## 对注意力分数进行归一化
33             attention_probs = torch.softmax(attention_scores, dim=-1)
34
35             output = torch.matmul(attention_probs, value)
36             ## 对注意力输出进行拼接
```

```

37         output = output.transpose(1, 2).contiguous().view(batch_size, -1,
self.head_dim * self.num_heads)
38
39         output = self.o_linear(output)
40         return output
41
42     def split_head(self, x):
43         batch_size = x.size()[0]
44         return x.view(batch_size, -1, self.num_heads,
self.head_dim).transpose(1,2)

```

🌿 说明

MultiHeadAttention 类: 继承了 `nn.Module`，负责初始化多头注意力的组件，包括线性投影层和输出层。

forward 方法: 实现前向传播。分别计算 queries, keys, values 的线性变换，并使用缩放点积注意力得到最终输出。

torch.einsum: 用来实现张量之间的Einstein求和约定，计算过程更直观。

注意事项

`embed_size` 必须可以被 `num_heads` 整除，以便将数据拆分为多个头。

`torch.sqrt` 和 `torch.softmax` 用于对注意力得分进行缩放和归一化。

可选的 `mask` 可以用于处理填充的序列。

Grouped-Query Attention (GQA)

Grouped-Query Attention (GQA) 是一种用于改进自注意力机制的技术，尤其在减少计算复杂度方面表现出色。下面是一个使用Python和PyTorch实现GQA的示例代码。

```

1  import torch
2  import torch.nn as nn
3
4  class GroupedQueryAttention(nn.Module):
5      def __init__(self, embed_dim, num_heads, num_groups):
6          super(GroupedQueryAttention, self).__init__()
7          assert embed_dim % num_heads == 0, "Embedding dimension must be
divisible by the number of heads."
8          assert num_heads % num_groups == 0, "Number of heads must be
divisible by the number of groups."
9
10         self.embed_dim = embed_dim

```

```

11         self.num_heads = num_heads
12         self.num_groups = num_groups
13         self.head_dim = embed_dim // num_heads
14         self.group_heads = num_heads // num_groups
15
16         self.q_proj = nn.Linear(embed_dim, embed_dim)
17         self.k_proj = nn.Linear(embed_dim, embed_dim)
18         self.v_proj = nn.Linear(embed_dim, embed_dim)
19         self.out_proj = nn.Linear(embed_dim, embed_dim)
20
21     def forward(self, query, key, value, mask=None):
22         batch_size, seq_length, _ = query.size()
23
24         q = self.q_proj(query).view(batch_size, seq_length, self.num_heads,
self.head_dim)
25         k = self.k_proj(key).view(batch_size, seq_length, self.num_heads,
self.head_dim)
26         v = self.v_proj(value).view(batch_size, seq_length, self.num_heads,
self.head_dim)
27
28         q = q.permute(0, 2, 1, 3) # [batch_size, num_heads, seq_length,
head_dim]
29         k = k.permute(0, 2, 1, 3)
30         v = v.permute(0, 2, 1, 3)
31
32         # Reshape into groups
33         q = q.view(batch_size, self.num_groups, self.group_heads, seq_length,
self.head_dim)
34         k = k.view(batch_size, self.num_groups, self.group_heads, seq_length,
self.head_dim)
35         v = v.view(batch_size, self.num_groups, self.group_heads, seq_length,
self.head_dim)
36
37         # Compute attention scores
38         scores = torch.einsum('bghld,bghmd->bghlm', q, k) / (self.head_dim **
0.5)
39
40         if mask is not None:
41             scores = scores.masked_fill(mask == 0, float("-inf"))
42
43         attn_weights = torch.nn.functional.softmax(scores, dim=-1)
44
45         context = torch.einsum('bghlm,bghmd->bghld', attn_weights, v)
46
47         context = context.view(batch_size, self.num_heads, seq_length,
self.head_dim)

```

```

48         context = context.permute(0, 2, 1, 3).contiguous().view(batch_size,
seq_length, self.embed_dim)
49
50         return self.out_proj(context)
51
52     # Usage
53     embed_dim = 64
54     num_heads = 8
55     num_groups = 2
56     batch_size = 2
57     seq_length = 10
58
59     gqa = GroupedQueryAttention(embed_dim, num_heads, num_groups)
60     query = torch.rand(batch_size, seq_length, embed_dim)
61     key = torch.rand(batch_size, seq_length, embed_dim)
62     value = torch.rand(batch_size, seq_length, embed_dim)
63
64     output = gqa(query, key, value)
65     print(output.shape)

```

说明

这个实现的 `GroupedQueryAttention` 类进行前向传播计算，并输出形状为 `(batch_size, seq_length, embed_dim)` 的张量。

好的，下面是一个简单的手撕版Multi-Query Attention (MQA)的Python实现，并且附带注释和解释。

Multi-Query Attention (MQA)



在传统的自注意力机制中，每个查询 (Query) 都会与键 (Key) 和值 (Value) 进行计算，从而获得最终的注意力加权输出。而在Multi-Query Attention (MQA)中，一个查询会共享多个键值对。这种机制通常用于提高计算效率和降低内存消耗，尤其在Transformer模型中可以节省大量的资源。

```

1  import torch
2  import torch.nn.functional as F
3
4  class MultiQueryAttention(torch.nn.Module):
5      def __init__(self, embed_dim, num_heads, num_queries):
6          super(MultiQueryAttention, self).__init__()
7          # 初始化注意力层的各个组件
8          self.num_heads = num_heads

```

```

9         self.num_queries = num_queries
10        self.embed_dim = embed_dim
11
12        # 查询、键和值的线性变换
13        self.query_proj = torch.nn.Linear(embed_dim, embed_dim)
14        self.key_proj = torch.nn.Linear(embed_dim, embed_dim)
15        self.value_proj = torch.nn.Linear(embed_dim, embed_dim)
16
17        # 输出线性变换
18        self.out_proj = torch.nn.Linear(embed_dim, embed_dim)
19
20        def forward(self, x):
21            """
22            x: 输入的张量 (batch_size, seq_len, embed_dim)
23            """
24            batch_size, seq_len, _ = x.size()
25
26            # 线性变换得到查询、键和值
27            queries = self.query_proj(x) # (batch_size, seq_len, embed_dim)
28            keys = self.key_proj(x) # (batch_size, seq_len, embed_dim)
29            values = self.value_proj(x) # (batch_size, seq_len, embed_dim)
30
31            # 重塑查询张量, 使得每个查询共享多个查询向量
32            queries = queries.unsqueeze(1).expand(-1, self.num_queries, -1, -1)
33            # (batch_size, num_queries, seq_len, embed_dim)
34
35            # 计算注意力分数 (Q*K^T)
36            queries = queries.view(batch_size * self.num_queries, seq_len,
37            self.embed_dim) # (batch_size * num_queries, seq_len, embed_dim)
38            keys = keys.view(batch_size, seq_len, self.num_heads, self.embed_dim
39            // self.num_heads) # (batch_size, seq_len, num_heads, embed_dim // num_heads)
40            values = values.view(batch_size, seq_len, self.num_heads,
41            self.embed_dim // self.num_heads) # (batch_size, seq_len, num_heads,
42            embed_dim // num_heads)
43
44            # 计算Q和K的点积, 并应用缩放
45            attention_scores = torch.matmul(queries, keys.transpose(-2, -1)) /
46            (self.embed_dim ** 0.5) # (batch_size * num_queries, seq_len, seq_len)
47
48            attention_probs = F.softmax(attention_scores, dim=-1) # (batch_size
49            * num_queries, seq_len, seq_len)
50
51            # 计算加权平均值
52            weighted_values = torch.matmul(attention_probs, values) #
53            (batch_size * num_queries, seq_len, embed_dim)
54
55            # 输出线性变换

```

```

47         output = self.out_proj(weighted_values.view(batch_size *
self.num_queries, seq_len, self.embed_dim)) # (batch_size * num_queries,
seq_len, embed_dim)
48
49         # 将输出重塑回原来的形状
50         output = output.view(batch_size, self.num_queries, seq_len,
self.embed_dim) # (batch_size, num_queries, seq_len, embed_dim)
51
52         return output
53
54 # 测试用例
55 if __name__ == "__main__":
56     # 假设输入的张量是 (batch_size, seq_len, embed_dim) = (2, 10, 64)
57     x = torch.randn(2, 10, 64) # batch_size = 2, seq_len = 10, embed_dim = 64
58     mqa = MultiQueryAttention(embed_dim=64, num_heads=8, num_queries=4) #
num_queries = 4
59     output = mqa(x)
60     print(output.shape) # 输出应为 (batch_size, num_queries, seq_len,
embed_dim) = (2, 4, 10, 64)

```



初始化:

- 定义了一个 `MultiQueryAttention` 类继承自 `torch.nn.Module`，用于实现 MQA。
- `embed_dim` 是输入向量的维度，`num_heads` 是多头注意力的头数，`num_queries` 是每个查询共享的查询数。

线性变换:

- `query_proj`、`key_proj` 和 `value_proj` 分别将输入的嵌入 (`x`) 通过线性变换得到查询、键和值。

扩展查询:

- `queries = queries.unsqueeze(1).expand(-1, self.num_queries, -1, -1)` 这一行代码将查询张量扩展，使得每个查询都可扩展成一个查询向量。

计算注意力分数:

- 将查询和键做点积来计算注意力分数，并且进行缩放（除以 `sqrt(embed_dim)`）。

计算注意力概率:

- 使用 `F.softmax` 来归一化注意力分数，得到注意力概率。

加权求和:

- 使用 `attention_probs` 来加权求和值张量，最终得到每个查询的输出。

输出线性变换:

- 最终的加权输出通过 `out_proj` 进行线性变换, 得到最终的输出结果。

ROPE (Rotary Position Embedding)

下面是一个Python实现的ROPE (Rotary Position Embedding) 代码。ROPE是一种用于引入位置信息的技术, 通过将位置信息应用于上下文中的每一个标记的方式来增强Transformer模型的性能。

```
1  import numpy as np
2
3  class ROPE:
4      def __init__(self, dim):
5          self.dim = dim
6          self.theta = 0.1 # 这个值可以根据任务需求进行调整
7
8          # 计算频率
9          inv_freq = 1.0 / (10000 ** (np.arange(0, dim, 2) / dim))
10         self.inv_freq = inv_freq
11
12         def get_positional_encoding(self, seq_len):
13             # 计算位置
14             pos = np.arange(seq_len)
15             angle_rates = pos[:, None] * self.inv_freq[None, :]
16             pos_enc = np.zeros((seq_len, self.dim))
17             pos_enc[:, 0::2] = np.sin(angle_rates)
18             pos_enc[:, 1::2] = np.cos(angle_rates)
19             return pos_enc
20
21         def apply_rotary_pos_emb(self, x, seq_len):
22             pos_enc = self.get_positional_encoding(seq_len)
23             # 应用旋转位置嵌入到输入x
24             return (x * np.cos(pos_enc)) + (self.rotate_half(x) * np.sin(pos_enc))
25
26         def rotate_half(self, x):
27             x1 = x[:, :, ::2]
28             x2 = x[:, :, 1::2]
29             # 执行半旋转
30             x = np.stack((-x2, x1), axis=-1).reshape(x.shape)
31             return x
32
33         # 示例用法
34         seq_len = 10 # 输入序列长度
35         dim = 16      # 向量的维度
36
37         rope = ROPE(dim=dim)
```

```

38 input_vectors = np.random.rand(1, seq_len, dim) # 模拟输入向量 (batch_size=1)
39
40 # 应用ROPE
41 output_vectors = rope.apply_rotary_pos_emb(input_vectors, seq_len)
42 print("输出向量形状:", output_vectors.shape)
43 print(output_vectors)

```

🌱 说明

- **ROPE** 类: 实现ROPE的关键组件, 包括计算旋转位置编码和将其应用于输入向量的功能。
- **theta**: 控制每个位置之间的旋转幅度。
- **get_positional_encoding**: 计算正弦和余弦位置编码。
- **apply_rotary_pos_emb**: 将旋转位置编码应用于输入向量。
- **rotate_half**: 用于旋转输入向量的半部分。
- **apply_rotary_pos_emb** 函数为输入序列中的每一个向量应用旋转位置嵌入。在Transformer模型的不同上下文中, 可以频繁地调用这种编码方式来加强模型的性能。

前馈神经网络 (Feed-Forward Neural Network)

下面是一个简单的前馈神经网络 (Feed-Forward Neural Network) 的实现。这个网络有一个输入层、一个隐藏层和一个输出层。它使用sigmoid激活函数, 并通过梯度下降进行训练。使用均方误差 (MSE) 作为损失函数。

```

1 import numpy as np
2
3 # Sigmoid函数及其导数
4 def sigmoid(x):
5     return 1 / (1 + np.exp(-x))
6
7 def sigmoid_derivative(x):
8     return x * (1 - x)
9
10 # 神经网络类
11 class NeuralNetwork:
12     def __init__(self, input_size, hidden_size, output_size,
13 learning_rate=0.1):
14         # 初始化权重
15         self.input_size = input_size
16         self.hidden_size = hidden_size
17         self.output_size = output_size
18         self.learning_rate = learning_rate

```

```

18
19     # 设置权重和偏置
20     self.weights_input_hidden = np.random.rand(self.input_size,
self.hidden_size)
21     self.weights_hidden_output = np.random.rand(self.hidden_size,
self.output_size)
22     self.bias_hidden = np.random.rand(self.hidden_size)
23     self.bias_output = np.random.rand(self.output_size)
24
25     def feedforward(self, X):
26         # 输入层到隐藏层
27         self.hidden_input = np.dot(X, self.weights_input_hidden) +
self.bias_hidden
28         self.hidden_output = sigmoid(self.hidden_input)
29
30         # 隐藏层到输出层
31         self.final_input = np.dot(self.hidden_output,
self.weights_hidden_output) + self.bias_output
32         self.final_output = sigmoid(self.final_input)
33
34         return self.final_output
35
36     def backpropagate(self, X, y, output):
37         # 计算误差
38         output_error = y - output
39         output_delta = output_error * sigmoid_derivative(output)
40
41         hidden_error = output_delta.dot(self.weights_hidden_output.T)
42         hidden_delta = hidden_error * sigmoid_derivative(self.hidden_output)
43
44         # 更新权重和偏置
45         self.weights_hidden_output += self.hidden_output.T.dot(output_delta)
* self.learning_rate
46         self.bias_output += np.sum(output_delta, axis=0) * self.learning_rate
47
48         self.weights_input_hidden += X.T.dot(hidden_delta) *
self.learning_rate
49         self.bias_hidden += np.sum(hidden_delta, axis=0) * self.learning_rate
50
51     def train(self, X, y, epochs=10000):
52         for _ in range(epochs):
53             # 前向传播
54             output = self.feedforward(X)
55
56             # 反向传播
57             self.backpropagate(X, y, output)
58

```

```

59 # 示例数据
60 X = np.array([[0, 0],
61               [0, 1],
62               [1, 0],
63               [1, 1]])
64 y = np.array([[0], [1], [1], [0]])
65
66 # 初始化神经网络
67 nn = NeuralNetwork(input_size=2, hidden_size=3, output_size=1)
68
69 # 训练神经网络
70 nn.train(X, y, epochs=10000)
71
72 # 查看训练结果
73 output = nn.feedforward(X)
74 print("预测结果: ")
75 print(output)

```

RMSNorm

RMSNorm (Root Mean Square Layer Normalization) 是另一种常见的层归一化方法，类似于 LayerNorm，但在计算标准化时略有不同。

```

1 import numpy as np
2
3 class RMSNorm:
4     def __init__(self, dim, epsilon=1e-8):
5         self.epsilon = epsilon
6         self.scale = np.ones(dim)
7
8     def __call__(self, x):
9         # Compute the mean square of each feature
10        mean_square = np.mean(x**2, axis=-1, keepdims=True)
11        # Compute RMS
12        rms = np.sqrt(mean_square + self.epsilon)
13        # Normalize
14        x_normalized = x / rms
15        # Scale
16        output = self.scale * x_normalized
17        return output
18
19 # Example usage
20 x = np.array([[1.0, 2.0, 3.0],
21               [4.0, 5.0, 6.0]])
22

```

```

23 rms_norm = RMSNorm(dim=x.shape[-1])
24 output = rms_norm(x)
25 print(output)

```

1. **RMSNorm类的初始化**: 创建RMSNorm类时, 需要指定输入的特征维度 (`dim`) 和一个小的数值 (`epsilon`) 以避免除零错误。
 2. **计算RMS**: 在调用该类时, 首先计算每个输入向量的均方 (mean square), 然后得出其平方根作为均方根 (RMS)。
 3. **归一化**: 每个输入特征除以其对应的RMS, 得到归一化的输出。
 4. **缩放**: 将结果乘以一个可训练的参数 `scale`, 这个参数可以在实际的训练过程中更新。
- 这样, 就对输入进行了RMSNorm层归一化, 并且代码是简单易懂的。

ReLU

下面是使用 Python 实现的 ReLU (Rectified Linear Unit) 函数。ReLU 是一种常用的激活函数, 在神经网络中应用广泛。以下代码展示了如何定义和应用 ReLU 函数:

```

1  import numpy as np
2
3  def relu(x):
4      """
5      Apply the ReLU activation function to a numpy array or a single scalar
6      value.
7
8      The ReLU function is defined as  $f(x) = \max(0, x)$ .
9
10     Parameters:
11     x (np.ndarray or float): Input data.
12
13     Returns:
14     np.ndarray or float: ReLU applied element-wise if input is an array,
15                          or the output of ReLU if a scalar is provided.
16     """
17     return np.maximum(0, x)
18
19 # Example usage:
20 x_array = np.array([-2, -1, 0, 1, 2, 3])
21 x_scalar = -0.5
22
23 relu_array = relu(x_array)
24 relu_scalar = relu(x_scalar)

```

24

```
25 print("ReLU applied to array: ", relu_array)
```

```
26 print("ReLU applied to scalar: ", relu_scalar)
```

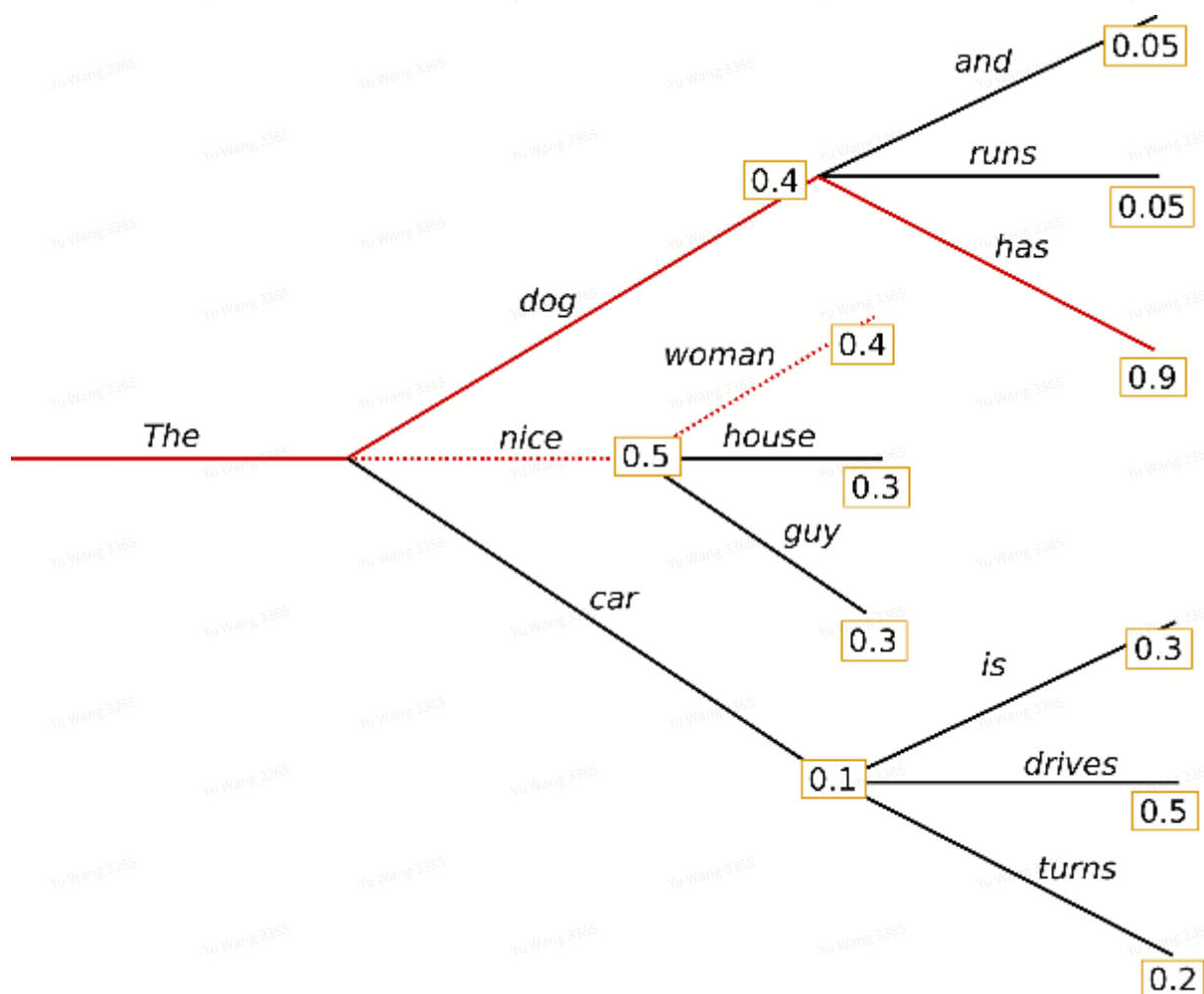
🌱 说明

- `relu` 函数接受一个 numpy 数组或单一的浮点数作为输入。
- 使用 `np.maximum(0, x)` 可以在数组的每个元素或者单一数值上执行 ReLU 操作。
- ReLU 的定义是：当输入 x 大于 0 时输出 x ，否则输出 0。

beamsearch 算法

在NLP翻译或对话任务中，在句子解码阶段，经常用到一种搜索算法beam search。这个算法有时候在大厂面试中，甚至可能会被要求手写实现。这里就从beam search的原理出发，最后手写实现一个beam search。

beam search维护一个大小为 k 的候选序列集合，每一步从每个候选序列的概率分布中选择概率最高的 k 个单词，然后保留总概率最高的 k 个候选序列。这种方法可以平衡生成的质量和多样性，但是可能会导致生成的文本过于保守和不自然



```

1 def pred(input):
2     batch, seq_len = input.shape
3     generate = torch.randn(size=(batch, 1, 10))
4     return generate
5
6 def beam_search(input_ids, max_length, num_beams):
7     batch = input_ids.shape[0]
8
9     # 输入扩展: 效果是把input_ids复制成 batch_size * beam_size的个数
10    expand_size = num_beams
11    expanded_return_idx = (
12        torch.arange(input_ids.shape[0]).view(-1, 1).repeat(1,
13        expand_size).view(-1).to(input_ids.device)
14    )
15    input_ids = input_ids.index_select(0, expanded_return_idx)
16    print(input_ids)
17
18    batch_beam_size, cur_len = input_ids.shape
19    beam_scores = torch.zeros(size=
20    (batch, num_beams), dtype=torch.float, device=input_ids.device)
21    beam_scores[:, 1:] = -1e9
22
23    beam_scores = beam_scores.view(size=(batch * num_beams,))
24    next_tokens = torch.zeros(size=
25    (batch, num_beams), dtype=torch.long, device=input_ids.device)
26    next_indices = torch.zeros(size=
27    (batch, num_beams), dtype=torch.long, device=input_ids.device)
28
29    while cur_len < max_length:
30
31        logits = pred(input_ids) # batch, seq_len, vocab
32        next_token_logits = logits[:, -1, :] # 当前时刻的输出
33
34        # 归一化
35        next_token_scores = F.log_softmax(next_token_logits, dim=-1) #
36        (batch_size * num_beams, vocab_size)
37
38        # 求概率
39        next_token_scores = next_token_scores + beam_scores[:,
40        None].expand_as(next_token_scores) # 当前概率+先前概率
41
42        # reshape for beam search
43        vocab_size = next_token_scores.shape[-1]
44        next_token_scores = next_token_scores.view(batch, num_beams *
45        vocab_size)
46
47        # 当前时刻的token 得分, token_id
48        next_token_scores, next_tokens = torch.topk(

```

```

41         next_token_scores, num_beams, dim=1, largest=True, sorted=True
42     )
43
44     next_indices = next_tokens // vocab_size #对应的beam_id
45     next_tokens = next_tokens % vocab_size #对应的indices
46
47     #束搜索核心
48     def process(input_ids, next_scores, next_tokens, next_indices):
49         batch_size=3
50         group_size=3
51         next_beam_scores = torch.zeros((batch_size, num_beams),
dtype=next_scores.dtype)
52         next_beam_tokens = torch.zeros((batch_size, num_beams),
dtype=next_tokens.dtype)
53         next_beam_indices = torch.zeros((batch_size, num_beams),
dtype=next_indices.dtype)
54
55         for batch_idx in range(batch_size):
56             beam_idx=0
57             for beam_token_rank, (next_token, next_score, next_index) in
enumerate(
58                 zip(next_tokens[batch_idx], next_scores[batch_idx],
next_indices[batch_idx])
59             ):
60                 batch_beam_idx=batch_idx*num_beams+next_index
61
62                 next_beam_scores[batch_idx, beam_idx] = next_score #当
前路径得分
63                 next_beam_tokens[batch_idx, beam_idx] = next_token #当
前时刻的token
64                 next_beam_indices[batch_idx, beam_idx] = batch_beam_idx #
先前对应的id
65
66                 beam_idx += 1
67
68         return next_beam_scores.view(-1), next_beam_tokens.view(-1),
next_beam_indices.view(-1)
69
70     beam_scores, beam_next_tokens,
beam_idx=process(input_ids, next_token_scores, next_tokens, next_indices)
71
72     # 更新输入, 找到对应的beam_idx, 选择的tokens, 拼接为新的输入 #
(batch*beam, seq_len)
73     input_ids = torch.cat([input_ids[beam_idx, :],
beam_next_tokens.unsqueeze(-1)], dim=-1)
74     cur_len = cur_len + 1
75     #输出

```

```

76     return input_ids, beam_scores
77
78
79 if __name__ == '__main__':
80     input_ids=torch.randint(0,100,size=(3,1))
81     print(input_ids)
82     input_ids,beam_scores=beam_search(input_ids,max_length=10,num_beams=3)
83     print(input_ids)

```

SwiGLU

下面是一个SwiGLU的简单实现。SwiGLU是一种神经网络的激活函数，在某些任务中表现优于传统的ReLU激活函数。

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  class SwiGLU(nn.Module):
6      def __init__(self, input_dim, hidden_dim=None):
7          super(SwiGLU, self).__init__()
8
9          # Default to double the input dimension for hidden_dim if not provided
10         if hidden_dim is None:
11             hidden_dim = input_dim * 2
12
13         self.linear1 = nn.Linear(input_dim, hidden_dim)
14         self.linear2 = nn.Linear(hidden_dim, input_dim)
15
16     def forward(self, x):
17         # GLU:  $x * \text{sigmoid}(x)$ 
18         return F.silu(self.linear1(x)) * self.linear2(x)
19
20 # Example usage
21 if __name__ == "__main__":
22     input_dim = 64
23     hidden_dim = 128 # Can be double the input_dim or any other design choice
24
25     swiglu_layer = SwiGLU(input_dim, hidden_dim)
26
27     # Random input tensor of shape (batch_size, input_dim)
28     x = torch.rand(32, input_dim)
29
30     # Forward pass
31     output = swiglu_layer(x)

```

BPE实现代码



BPE实现代码主要有两个函数：一个专门统计vocabulary，另一个负责合并字符串。

统计词频很暴力，就是遍历vocabulary里每一个元素，利用像两个元素的滑动窗口，挨个组合并且记录频次。找到频次最高的之后进行合并并且输出新的vocabulary。

统计词频

代码块

```
1 import re, collections
2 text = "The aims for this subject is for students to develop an
understanding of the main algorithms used in naturallanguage processing,
for use in a diverse range of applications including text classification,
machine translation, and question answering. Topics to be covered include
part-of-speech tagging, n-gram language modelling, syntactic parsing and
deep learning. The programming language used is Python, see for more
information on its use in the workshops, assignments and installation at
home."
3 # text = 'low '*5 +'lower '*2+'newest '*6 +'widest '*3
4 '''
5 先统计词频
6 '''
7 def get_vocab(text):
8
9     # 初始化为 0
10    vocab = collections.defaultdict(int)
11    # 去头去尾再根据空格split
12    for word in text.strip().split():
13        #note: we use the special token </w> (instead of underscore in the
lecture) to denote the end of a word
14        # 给list中每个元素增加空格，并在最后增加结束符号，同时统计单词出现次数
15        vocab[' '.join(list(word)) + ' </w>'] += 1
16    return vocab
17 print(get_vocab(text))
```

统计相邻字符对频率

代码块

```
1  '''''
```

```

2 这个函数遍历词汇表中的所有单词，并计算彼此相邻的一对标记。
3 EXAMPLE:
4 word = 'T h e <\w>'
5 这个单词可以两两组合成: [('T', 'h'), ('h', 'e'), ('e', '<\w>')]
6 输入:
7 vocab: Dict[str, int] # vocab统计了词语出现的词频
8 输出:
9 pairs: Dict[Tuple[str, str], int] # 字母对, pairs统计了单词对出现的频率
10 """
11 def get_stats(vocab):
12     pairs = collections.defaultdict(int)
13
14     for word, freq in vocab.items():
15
16         # 遍历每一个word里面的symbol, 去凑所有的相邻两个内容
17         symbols = word.split()
18         for i in range(len(symbols)-1):
19             pairs[(symbols[i], symbols[i+1])] += freq
20
21     return pairs

```

合并高频字符对

代码块

```

1 """
2 EXAMPLE:
3 word = 'T h e <\w>'
4 pair = ('e', '<\w>')
5 word_after_merge = 'T h e<\w>'
6 输入:
7 pair: Tuple[str, str] # 需要合并的字符对
8 v_in: Dict[str, int] # 合并前的vocab
9 输出:
10 v_out: Dict[str, int] # 合并后的vocab
11 注意:
12 当合并word 'Th e<\w>'中的字符对 ('h', 'e')时, 'Th'和'e<\w>'字符对不能被合并。
13 """
14 def merge_vocab(pair, v_in):
15     v_out = {}
16     # 把pair拆开, 然后用空格合并起来, 然后用\把空格转义
17     bigram = re.escape(' '.join(pair))
18     # 自定义一个正则规则, (?<!\S)h\ e(?:!\S) 只有前面、后面不是非空白字符(\S)(意思前后得是没东西的), 才匹配h\ e, 这样就可以把Th\ e<\w>排除在外
19     p = re.compile(r'(?<!\S)' + bigram + r'(?!\\S)')
20

```

```

21     for v in v_in:
22         # 遍历当前的vocabulary, 找到匹配正则的v时, 才用合并的pair去替换变成新的
pair new, 如果没有匹配上, 那就保持原来的。
23         # 比如pair当前是'h'和'e', 然后遍历vocabulary, 找到符合前后都没有东西只
有'h\ e'的时候就把他们并在一起变成'he'
24         new = p.sub(''.join(pair),v)
25         # 然后新的合并的数量就是当前vocabulary里面pair对应的数量
26         v_out[new] = v_in[v]
27     return v_out
28 def get_tokens(vocab):
29     tokens = collections.defaultdict(int)
30     for word, freq in vocab.items():
31         word_tokens = word.split()
32         for token in word_tokens:
33             tokens[token] += freq
34     return tokens
35 vocab = get_vocab(text)
36 print("Vocab =", vocab)
37 print('=====')
38 print('Tokens Before BPE')
39 tokens = get_tokens(vocab)
40 print('Tokens: {}'.format(tokens))
41 print('Number of tokens: {}'.format(len(tokens)))
42 print('=====')
43 #about 100 merges we start to see common words
44 num_merges = 100
45 for i in range(num_merges):
46     pairs = get_stats(vocab)
47     if not pairs:
48         break
49
50 # vocabulary里面pair出现次数最高的作为最先合并的pair
51 best = max(pairs, key=pairs.get)
52
53 # 先给他合并了再说, 当然这里不操作也没什么, 到merge_vocab里面都一样
54 new_token = ''.join(best)
55 vocab = merge_vocab(best, vocab)
56 print('Iter: {}'.format(i))
57 print('Best pair: {}'.format(best))
58 # add new token to the vocab
59 tokens[new_token] = pairs[best]
60 # deduct frequency for tokens have been merged
61 tokens[best[0]] -= pairs[best]
62 tokens[best[1]] -= pairs[best]
63 print('Tokens: {}'.format(tokens))
64 print('Number of tokens: {}'.format(len(tokens)))
65 print('=====')

```

分词示例, 应用上面代码产生的vocabulary

代码块

```

1  def get_tokens_from_vocab(vocab):
2      tokens_frequencies = collections.defaultdict(int)
3      vocab_tokenization = {}
4      for word, freq in vocab.items():
5          # 看vocabulary里面的token频率, 相当于上面的code中的tokens去除freq为0的
6          word_tokens = word.split()
7          for token in word_tokens:
8              tokens_frequencies[token] += freq
9          # vocab和其对应的tokens
10         vocab_tokenization['.'.join(word_tokens)] = word_tokens
11     return tokens_frequencies, vocab_tokenization
12
13 def measure_token_length(token):
14     # 如果token最后四个元素是 < / w >
15     if token[-4:] == '</w>':
16         # 那就返回除了最后四个之外的长度再加上1(结尾)
17         return len(token[:-4]) + 1
18     else:
19         # 如果这个token里面没有结尾就直接返回当前长度
20         return len(token)
21
22 # 如果vocabulary里面找不到要拆分的词, 就根据已有的token现拆
23 def tokenize_word(string, sorted_tokens, unknown_token='</u>'):
24
25     # base case, 没词进来了, 那拆的结果就是空的
26     if string == '':
27         return []
28     # 已有的sorted tokens没有了, 那就真的没这个词了
29     if sorted_tokens == []:
30         return [unknown_token] * len(string)
31
32     # 记录拆分结果
33     string_tokens = []
34
35     # iterate over all tokens to find match
36     for i in range(len(sorted_tokens)):
37         token = sorted_tokens[i]
38
39         # 自定义一个正则, 然后要把token里面包含句号的变成[.]
40         token_reg = re.escape(token.replace('.', '[.]'))

```

```

41
42     # 在当前string里面遍历, 找到每一个match token的开始和结束位置, 比如
string=good, 然后token是o, 输出[(2,2),(3,3)]?
43     matched_positions = [(m.start(0), m.end(0)) for m in
re.finditer(token_reg, string)]
44     # if no match found in the string, go to next token
45     if len(matched_positions) == 0:
46         continue
47     # 因为要拆分这个词, 匹配上的token把这个word拆开了, 那就要拿到除了match部分之
外的substring, 所以这里要拿match的start
48     substring_end_positions = [matched_position[0] for matched_position
in matched_positions]
49     substring_start_position = 0
50
51
52     # 如果有匹配成功的话, 就会进入这个循环
53     for substring_end_position in substring_end_positions:
54         # slice for sub-word
55         substring =
string[substring_start_position:substring_end_position]
56         # tokenize this sub-word with tokens remaining 接着用substring匹
配剩余的sorted token, 因为刚就匹配了一个
57         string_tokens += tokenize_word(string=substring,
sorted_tokens=sorted_tokens[i+1:], unknown_token=unknown_token)
58         # 先把sorted token里面匹配上的记下来
59         string_tokens += [token]
60         substring_start_position = substring_end_position + len(token)
61         # tokenize the remaining string 去除前头的substring, 去除已经匹配上的,
后面还剩下substring_start_pos到结束的一段substring没看
62         remaining_substring = string[substring_start_position:]
63         # 接着匹配
64         string_tokens += tokenize_word(string=remaining_substring,
sorted_tokens=sorted_tokens[i+1:], unknown_token=unknown_token)
65         break
66     else:
67         # return list of unknown token if no match is found for the string
68         string_tokens = [unknown_token] * len(string)
69
70     return string_tokens
71
72 """
73 该函数生成一个所有token的列表, 按其长度 (第一键) 和频率 (第二键) 排序。
74
75 EXAMPLE:
76     token frequency dictionary before sorting: {'natural': 3, 'language':2,
'processing': 4, 'lecture': 4}
77     sorted tokens: ['processing', 'language', 'lecture', 'natural']

```

```

78
79 INPUT:
80 token_frequencies: Dict[str, int] # Counter for token frequency
81
82 OUTPUT:
83 sorted_token: List[str] # Tokens sorted by length and frequency
84
85 """
86 def sort_tokens(tokens_frequencies):
87     # 对 token_frequencies里面的东西, 先进行长度排序, 再进行频次, sorted是从低到高
    所以要reverse
88     sorted_tokens_tuple = sorted(tokens_frequencies.items(), key=lambda
    item:(measure_token_length(item[0]),item[1]), reverse=True)
89
90     # 然后只要tokens不要频次
91     sorted_tokens = [token for (token, freq) in sorted_tokens_tuple]
92
93     return sorted_tokens
94
95 #display the vocab
96 tokens_frequencies, vocab_tokenization = get_tokens_from_vocab(vocab)
97
98 #sort tokens by length and frequency
99 sorted_tokens = sort_tokens(tokens_frequencies)
100 print("Tokens =", sorted_tokens, "\n")
101
102 #print("vocab tokenization: ", vocab_tokenization)
103
104 sentence_1 = 'I like natural language processing!'
105 sentence_2 = 'I like natural languaaage processing!'
106 sentence_list = [sentence_1, sentence_2]
107
108 for sentence in sentence_list:
109
110     print('=====')
111     print("Sentence =", sentence)
112
113     for word in sentence.split():
114         word = word + "</w>"
115
116         print('Tokenizing word: {}'.format(word))
117         if word in vocab_tokenization:
118             print(vocab_tokenization[word])
119         else:
120             print(tokenize_word(string=word, sorted_tokens=sorted_tokens,
            unknown_token='</u>'))

```

BBPE代码

```
1 import torch
2 import torch.nn as nn
3 import regex as re
4 import json
5 from collections import Counter
6 from concurrent.futures import ThreadPoolExecutor
7
8 def bytes_to_unicode()::
9     """
10     这个函数的作用是创建一个从字节到 Unicode 字符的映射表。
11     由于 BPE (字节对编码) 是在 Unicode 字符串上操作的, 但我们仍然希望能
12     处理原始字节数据, 因此我们需要一个可逆的字节到 Unicode 映射。
13
14     这里的逻辑是:
15     - 直接保留 ASCII 可打印字符 (即 '!' 到 '~' 以及 '¡' 到 'ÿ' 这些常见的字符)。
16     - 对于其他字节, 我们用扩展的 Unicode 码位来表示。
17     """
18     bs = (
19         list(range(ord("!"), ord("~") + 1)) +
20         list(range(ord("¡"), ord("¬") + 1)) +
21         list(range(ord("®"), ord("ÿ") + 1))
22     )
23
24     cs = bs[:]
25     n = 0
26     for b in range(2 ** 8): # 遍历所有 256 个字节
27         if b not in bs:
28             bs.append(b)
29             cs.append(2 ** 8 + n) # 使用 Unicode 扩展区的码点
30             n += 1
31
32     cs = [chr(n) for n in cs] # 转换成 Unicode 字符
33     return dict(zip(bs, cs)) # 返回字节到 Unicode 的映射
34
35 class BBPETokenizer(nn.Module):
36     def __init__(self, vocab_path: str, merges_path: str):
37         super().__init__()
38
39         # 读取词汇表 (即 token 到索引的映射)
40         with open(vocab_path, "r", encoding="utf-8") as f:
41             vocab = json.load(f)
42
43         # 读取 BPE 的合并规则
44         with open(merges_path, "r", encoding="utf-8") as f:
```

```

45         merges = f.read()
46
47     # 处理合并规则，转换为元组列表
48     merges = [tuple(merge_str.split()) for merge_str in merges.split("\n")
49     [:-1]]
50
51     # 编码器 (token -> 索引) 和解码器 (索引 -> token)
52     self.encoder = vocab
53     self.decoder = {v: k for k, v in self.encoder.items()}
54
55     # 字节到 Unicode 的转换
56     self.byte_encoder = bytes_to_unicode()
57     self.byte_decoder = {v: k for k, v in self.byte_encoder.items()}
58
59     # BPE 合并规则的优先级字典
60     self.bbpe_ranks = dict(zip(merges, range(len(merges))))
61
62     # 缓存已经处理过的 token，避免重复计算
63     self.cache = {}
64
65     # 预处理：定义正则表达式模式，拆分文本为基本 token
66     self.pat = re.compile(r"""
67         's|'t|'re|'ve|'m|'ll|'d|  # 常见的缩写，例如 it's, you're
68         \ ?\p{L}+|\ ?\p{N}+|  # 单词和数字，可以带可选空格
69         \ ?[^\s\p{L}\p{N}]+|  # 其他符号，比如标点符号
70         \s+(?!\\S)|  # 仅由空格组成的 token（行尾空格）
71         \s+  # 其他空格
72     """, re.X)
73
74     def forward(self, text):
75         """
76         这个函数允许你像 PyTorch 模型一样直接调用 BBPETokenizer 来进行编码。
77         支持单个字符串和字符串列表（批量处理）。
78         """
79         if isinstance(text, list):
80             tokens = self.encode_batch(text)
81             tokens = [token for row in tokens for token in row]
82         else:
83             tokens = self.encode(text)
84         return torch.tensor(tokens)
85
86     def bbpe(self, token):
87         """
88         执行 BPE 合并操作，应用合并规则来优化 token 表示。
89         """
90         if token in self.cache:
91             return self.cache[token]

```

```

91     chars = list(token)
92     for pair in self.bbpe_ranks.keys():
93         i = 0
94         while i < len(chars) - 1:
95             if chars[i] == pair[0] and chars[i + 1] == pair[1]:
96                 chars = chars[:i] + ["{}.".join(pair)] + chars[i + 2:]
97             else:
98                 i += 1
99
100     self.cache[token] = chars
101     return chars
102
103
104 def encode(self, text: str) -> list[int]:
105     """
106     将字符串转换为 BPE token ID 序列。
107     """
108     bbpe_tokens_id = []
109     for token in re.findall(self.pat, text):
110         token = "".join(self.byte_encoder[b] for b in token.encode("utf-
111 8"))
112         bbpe_tokens_id.extend(self.encoder[bpe_token] for bpe_token in
113 self.bbpe(token))
114     return bbpe_tokens_id
115
116
117 def decode(self, tokens) -> str:
118     """
119     反向转换 token ID 序列，恢复原始文本。
120     """
121     if isinstance(tokens, torch.Tensor):
122         tokens = tokens.tolist()
123     text = "".join([self.decoder[token] for token in tokens])
124     text = bytearray([self.byte_decoder[c] for c in text]).decode("utf-
125 8", errors="replace")
126     return text
127
128
129 @staticmethod
130 def train_tokenizer(data, vocab_size, vocab_outfile=None,
131 merges_outfile=None):
132     """
133     训练 BPE 词汇表。
134     """
135     if vocab_size < 256:
136         raise ValueError("vocab_size 必须大于 256")
137
138     byte_encoder = bytes_to_unicode()

```

```

133     pat_str = r"'s|'t|'re|'ve|'m|'ll|'d' ?[\p{L}]+| ?[\p{N}]+| ?
    [^\s\p{L}\p{N}]+\s+(?!S)|\s+"
134     split_words = [[byte_encoder[b] for b in token.encode("utf-8")] for
    token in re.findall(pat_str, data)]
135
136     vocab = set(byte_encoder.values())
137     merges = []
138
139     while len(vocab) < vocab_size:
140         pair_freq = Counter()
141         for split_word in split_words:
142             pair_freq.update(zip(split_word[:-1], split_word[1:]))
143             most_common_pair = pair_freq.most_common(1)[0][0]
144
145             new_token = most_common_pair[0] + most_common_pair[1]
146             vocab.add(new_token)
147             merges.append(most_common_pair)
148
149             new_split_words = []
150             for split_word in split_words:
151                 new_split_words.append([new_token if (split_word[i],
    split_word[i+1]) == most_common_pair else split_word[i] for i in
    range(len(split_word))])
152             split_words = new_split_words
153
154     if merges_outfile:
155         with open(merges_outfile, "w", encoding="utf-8") as f:
156             for merge in merges:
157                 f.write(merge[0] + " " + merge[1] + "\n")
158     if vocab_outfile:
159         with open(vocab_outfile, "w", encoding="utf-8") as f:
160             json.dump({v: i for i, v in enumerate(sorted(vocab))}, f,
    ensure_ascii=False)
161

```

WordPiece代码

WordPiece和BPE的主要区别就是计算 `score` 的部分，BPE计算 `freq`，WordPiece计算 `score`

代码块

```

1  from collections import defaultdict
2
3  def compute_pair_scores(splits):
4      """
5      计算当前所有子词对的得分，衡量哪些字符或子词应该被合并。

```

这里我们使用两个字典：

- `letter\_freqs`：统计所有单个字母或子词的频率
- `pair\_freqs`：统计相邻子词对的出现次数

公式：

- 每个子词对的分数 = (该对子词的出现次数) / (构成该对子词的两个部分各自的频率乘积)
- 这个计算方式类似于点互信息 (PMI)，可以衡量两个子词的相关性。

"""

```
letter_freqs = defaultdict(int) # 记录单个字母/子词的频率
pair_freqs = defaultdict(int) # 记录相邻子词对的频率
```

```
for word, freq in word_freqs.items():
    split = splits[word] # 该单词目前的子词分解形式

    if len(split) == 1:
        # 该单词只有一个子词 (没有可以合并的对子词)
        letter_freqs[split[0]] += freq
        continue # 跳过后续的对子词计算

    for i in range(len(split) - 1):
        pair = (split[i], split[i + 1]) # 获取相邻的两个子词
        letter_freqs[split[i]] += freq # 统计单个子词出现频率
        pair_freqs[pair] += freq # 统计对子词的出现次数

    # 统计最后一个子词的出现频率
    letter_freqs[split[-1]] += freq

    # 计算每个子词对的得分
    scores = {
        pair: freq / (letter_freqs[pair[0]] * letter_freqs[pair[1]])
        for pair, freq in pair_freqs.items()
    }

    return scores # 返回对子词的评分字典
```

```
def merge_pair(a, b, splits):
```

"""

将最高得分的对子词 (a, b) 合并成一个新子词，并更新 `splits`。

这里的逻辑：

- 遍历所有单词的子词拆分情况
- 找到 (a, b) 组合的子词对，把它们合并成新的子词
- 如果 `b` 以 "##" 开头，表示这是一个子词的后半部分，合并时去掉 "##"

例如：

```
splits = {'hello': ['h', 'e', 'l', 'l', 'o']}
```

```
merge_pair('l', 'l', splits)
```

```
结果: {'hello': ['h', 'e', 'll', 'o']}
```

```
"""
```

```
for word in word_freqs:
```

```
    split = splits[word] # 获取该单词当前的子词拆分
```

```
    if len(split) == 1:
```

```
        continue # 只有一个子词的情况，跳过
```

```
    i = 0
```

```
    while i < len(split) - 1:
```

```
        if split[i] == a and split[i + 1] == b:
```

```
            # 如果找到目标对子词 (a, b)，合并成一个新的子词
```

```
            merge = a + b[2:] if b.startswith("##") else a + b
```

```
            split = split[:i] + [merge] + split[i + 2:] # 替换合并后的子词
```

```
        else:
```

```
            i += 1 # 继续往后检查
```

```
    splits[word] = split # 更新 `splits` 词典
```

```
return splits
```

```
# 目标词汇表大小
```

```
vocab_size = 70
```

```
while len(vocab) < vocab_size:
```

```
    # 计算当前的子词对得分
```

```
    scores = compute_pair_scores(splits)
```

```
    # 找到得分最高的子词对
```

```
    best_pair, max_score = "", None
```

```
    for pair, score in scores.items():
```

```
        if max_score is None or max_score < score:
```

```
            best_pair = pair
```

```
            max_score = score
```

```
    # 将得分最高的对子词合并
```

```
    splits = merge_pair(*best_pair, splits)
```

```
    # 生成新的子词，并添加到词汇表
```

```
    new_token = (
```

```
        best_pair[0] + best_pair[1][2:] # 处理 `##` 前缀的情况
```

```
        if best_pair[1].startswith("##")
```

```
100         else best_pair[0] + best_pair[1]
101     )
102     vocab.append(new_token) # 词汇表扩充
```

ULM分词代码

```
1  from transformers import AutoTokenizer
2  from collections import defaultdict
3  from math import log
4  import copy
5
6  # 加载 XLNet 的分词器
7  tokenizer = AutoTokenizer.from_pretrained("xlnet-base-cased")
8
9  # 统计单词频率
10 word_freqs = defaultdict(int) # 用于存储单词的出现次数
11
12 for text in corpus:
13     # 使用 tokenizer 进行预分词（分词后带有偏移量信息）
14     words_with_offsets =
15     tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str(text)
16
17     # 只保留分词后的字符串（忽略偏移量信息）
18     new_words = [word for word, offset in words_with_offsets]
19
20     # 统计每个单词出现的次数
21     for word in new_words:
22         word_freqs[word] += 1
23
24 # 统计字符和子词的频率
25 char_freqs = defaultdict(int) # 记录单个字符的频率
26 subwords_freqs = defaultdict(int) # 记录子词（长度至少为2）的频率
27
28 for word, freq in word_freqs.items():
29     for i in range(len(word)):
30         char_freqs[word[i]] += freq # 统计每个字符的频率
31
32     # 遍历当前单词内所有可能的子词（长度至少为 2）
33     for j in range(i + 2, len(word) + 1):
34         subwords_freqs[word[i:j]] += freq # 统计子词频率
35
36 # 按频率降序排序子词
37 sorted_subwords = sorted(subwords_freqs.items(), key=lambda x: x[1],
38                           reverse=True)
```

```

38 # 生成初始 token 词汇表:
39 # - 先加入所有单字符
40 # - 再加入最常见的子词, 直到达到 300 个 token
41 token_freqs = list(char_freqs.items()) + sorted_subwords[:300 -
len(char_freqs)]
42 token_freqs = {token: freq for token, freq in token_freqs}
43
44 # 计算每个 token 的对数概率 (负对数频率), 用于编码计算
45 total_sum = sum(freq for token, freq in token_freqs.items())
46 model = {token: -log(freq / total_sum) for token, freq in token_freqs.items()}
47
48
49 def encode_word(word, model):
50     """
51     使用动态规划将单词编码成子词序列, 目标是最小化得分 (负对数概率)。
52
53     过程:
54     1. `best_segmentations[i]` 记录到第 i 个字符时的最佳切分方式及其得分
55     2. 遍历 `word` 的所有可能子词, 并尝试更新最佳分割方案
56     3. 反向回溯, 找到最终的最佳子词序列
57     """
58
59     # 初始化动态规划表
60     best_segmentations = [{"start": 0, "score": 1}] + [
61         {"start": None, "score": None} for _ in range(len(word))
62     ]
63
64     # 遍历起始位置 start_idx
65     for start_idx in range(len(word)):
66         best_score_at_start = best_segmentations[start_idx]["score"]
67
68         # 生成所有可能的子词
69         for end_idx in range(start_idx + 1, len(word) + 1):
70             token = word[start_idx:end_idx]
71
72             # 如果 token 在 model 里, 并且当前的起点有有效得分
73             if token in model and best_score_at_start is not None:
74                 score = model[token] + best_score_at_start
75
76                 # 更新最优分割方式
77                 if (
78                     best_segmentations[end_idx]["score"] is None
79                     or best_segmentations[end_idx]["score"] > score
80                 ):
81                     best_segmentations[end_idx] = {"start": start_idx,
82 "score": score}

```

```

83     # 反向回溯，得到最终的分词结果
84     segmentation = best_segmentations[-1]
85     if segmentation["score"] is None:
86         return ["<unk>"], None # 若无法分割，返回 <unk>
87
88     score = segmentation["score"]
89     start = segmentation["start"]
90     end = len(word)
91     tokens = []
92
93     while start != 0:
94         tokens.insert(0, word[start:end]) # 插入当前子词
95         next_start = best_segmentations[start]["start"]
96         end = start
97         start = next_start
98
99     tokens.insert(0, word[start:end])
100    return tokens, score
101
102
103    def compute_loss(model):
104        """
105        计算当前模型的总损失，即所有单词的分词得分之和。
106
107        - 目标是最小化该损失，从而使分词结果更符合数据分布
108        """
109        loss = 0
110        for word, freq in word_freqs.items():
111            _, word_loss = encode_word(word, model)
112            loss += freq * word_loss # 频率加权的损失
113        return loss
114
115
116    def compute_scores(model):
117        """
118        计算移除每个 token 后的损失变化情况，用于评估哪些 token 影响较小，可以移除。
119        """
120        scores = {}
121        model_loss = compute_loss(model)
122
123        for token, score in model.items():
124            if len(token) == 1:
125                continue # 保留所有单字符 token
126
127            model_without_token = copy.deepcopy(model)
128            _ = model_without_token.pop(token) # 移除当前 token
129

```

```

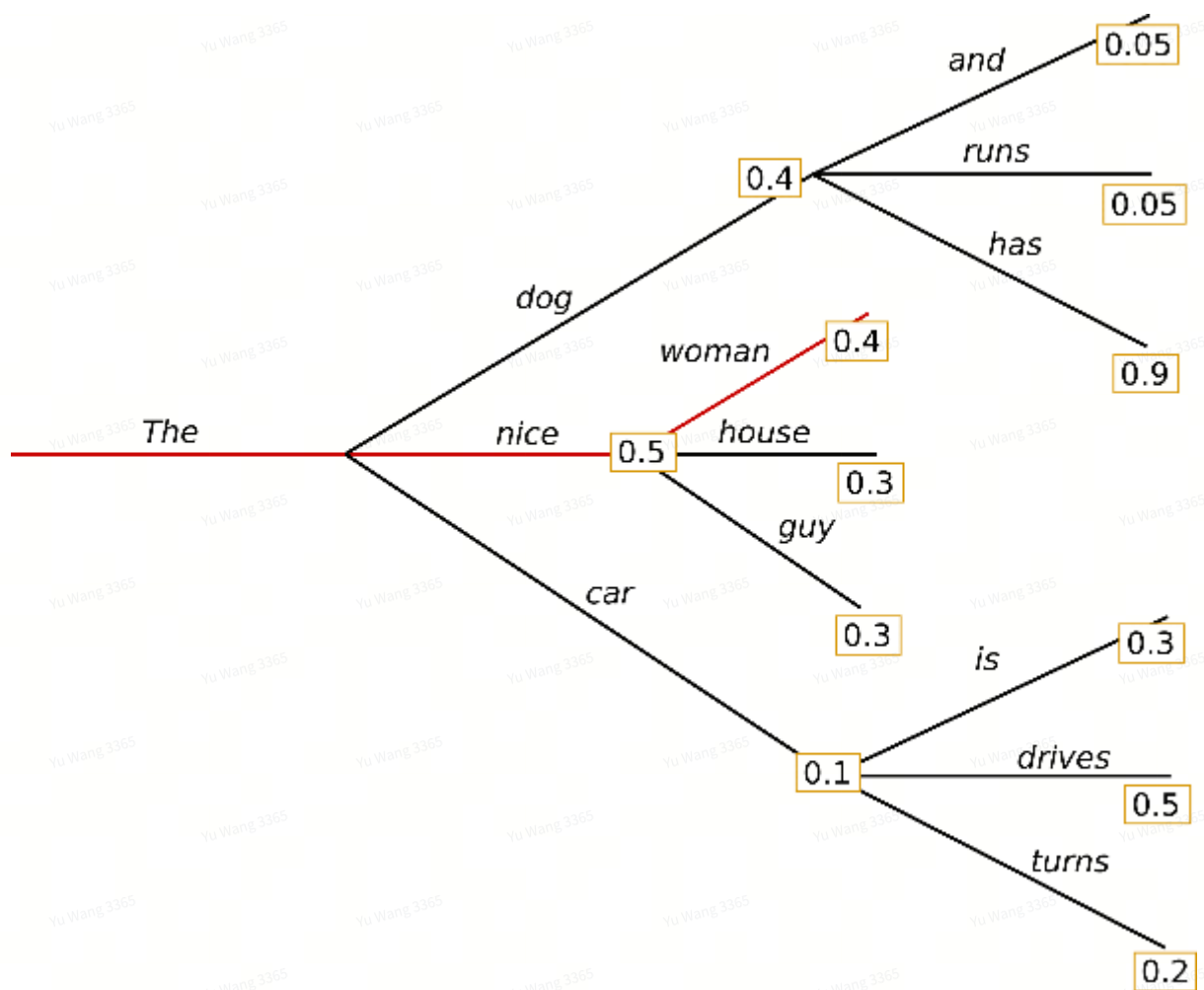
130     scores[token] = compute_loss(model_without_token) - model_loss # 计算
    损失变化
131     return scores
132
133
134     # 设定每次移除的 token 比例
135     percent_to_remove = 0.1
136
137     # 逐步减少 token, 直到模型大小 <= 100
138     while len(model) > 100:
139         scores = compute_scores(model)
140
141         # 按分数升序排列 (分数越低, 表示移除后影响越小)
142         sorted_scores = sorted(scores.items(), key=lambda x: x[1])
143
144         # 移除影响最小的若干 token
145         for i in range(int(len(model) * percent_to_remove)):
146             _ = token_freqs.pop(sorted_scores[i][0])
147
148         # 重新计算新的 token 频率
149         total_sum = sum(freq for token, freq in token_freqs.items())
150         model = {token: -log(freq / total_sum) for token, freq in
    token_freqs.items()}
151

```

不同的分词算法的比较要明确（面试时候就能非常有条理）：

对比项	ULM	BPE	WordPiece
合并策略	先初始化大词表，再删除影响最小的 token	按 频率最高 选择合并 token 对	按 互信息最大 选择合并 token 对
子词概率建模	基于 Unigram 语言模型，可以输出带概率的多个分词方式	无，只基于合并频率	有，但基于 token 互信息
计算复杂度	较高（需要反复删除 token 并计算 Loss 变化）	较低（仅基于 token 频率）	中等（需要计算 token 互信息）
OOV 处理	较强，能适应不同的分词方式	一般，可能会切分成过多子词	较好，但对拼写错误敏感
适用于	低资源语言、多分词候选任务	通用 NLP 任务	BERT、T5 等预训练模型

Greedy Search



贪心的每次选择概率最大的token（词），简单高效，但是可能会导致生成的文本过于单调和重复

代码块

```

1  import torch
2  import torch.nn.functional as F
3
4  def greedy_search(logits, max_len=20, eos_token_id=2):
5      batch_size = logits.size(0)
6      seq_len = logits.size(1)
7      vocab_size = logits.size(2)
8
9      # 初始化生成的序列，存储每个时间步选择的词索引
10     generated_sequence = torch.zeros(batch_size, max_len, dtype=torch.long)
11     # 记录当前生成的序列，初始化为开始 token（一般为0或者某个特殊的起始符号）
12     current_input = torch.zeros(batch_size, dtype=torch.long) # 起始token id
13     for t in range(max_len):
14         # 获取当前步骤的 logits
15         current_logits = logits[:, t, :]
16         # 计算每个词的概率分布（通过softmax）
17         probs = F.softmax(current_logits, dim=-1)
18         # 选择概率最高的词索引（greedy）
19         next_token = torch.argmax(probs, dim=-1)
20         # 将选择的词加入到生成的序列中

```

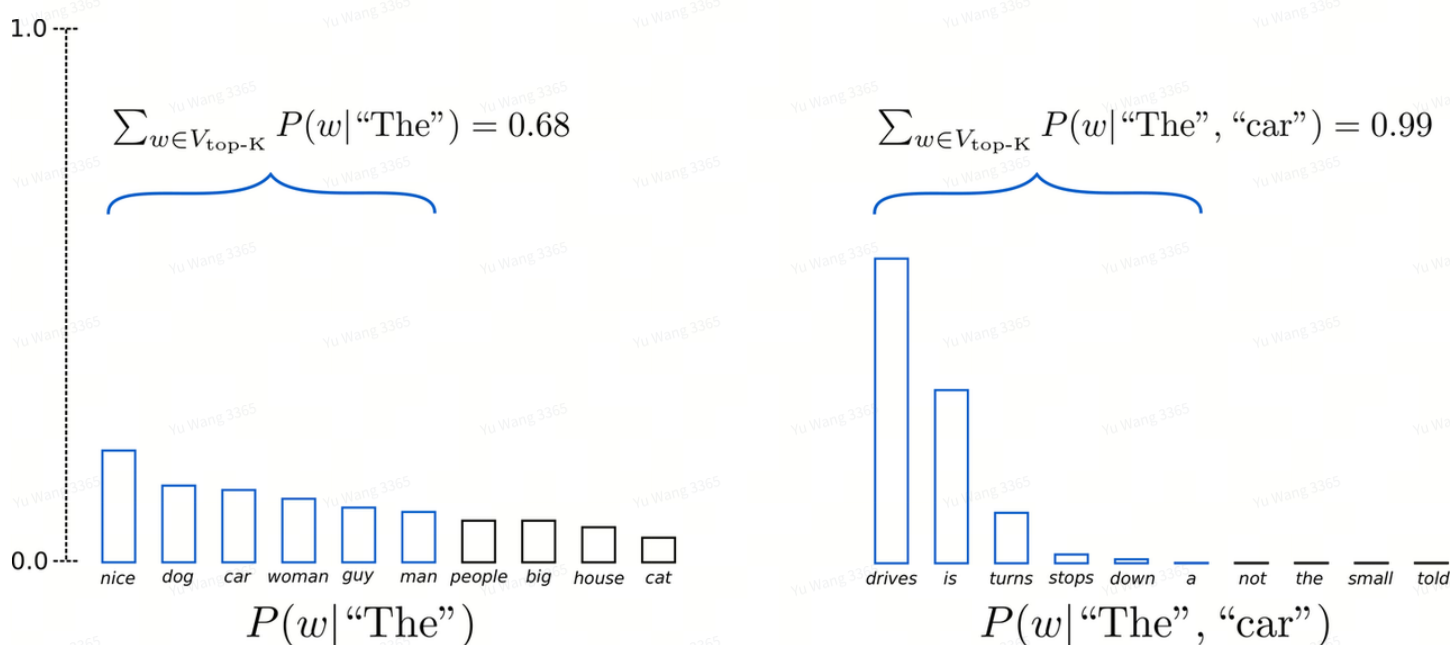
```

21 generated_sequence[:, t] = next_token
22 # 检查是否遇到结束标记, 若遇到则停止生成
23 if (next_token == eos_token_id).all():
24     break
25 # 更新当前输入 (下一个时间步的输入)
26 current_input = next_token
27 return generated_sequence

```

Top-K Sampling

从排名前 k 的 token 中进行抽样, 允许其他分数或概率较高的 token 也有机会被选中。在很多情况下, 这种抽样带来的随机性有助于提高生成质量。但是将采样池限制为固定大小 K 可能会在分布比较尖锐的时候产生胡言乱语, 而在分布比较平坦的时候限制模型的创造力



代码块

```

1  #@torch.inference_mode()
2  @torch.no_grad()
3  def generate(self, idx,
4               eos, max_new_tokens, temperature=1.0, top_k=None):
5      for _ in range(max_new_tokens):
6          # if the sequence context is growing too long we must crop it at
          block_size
7          idx_cond = idx if idx.size(1) <= self.params.max_seq_len else
idx[:, -self.params.max_seq_len:]
8          # forward the model to get the logits for the index in the sequence
9          logits = self(idx_cond) ##=====
10         logits = logits[:, -1, :] # crop to just the final time step
11         if temperature == 0.0:
12             # "sample" the single most likely index
13             _, idx_next = torch.topk(logits, k=1, dim=-1)

```

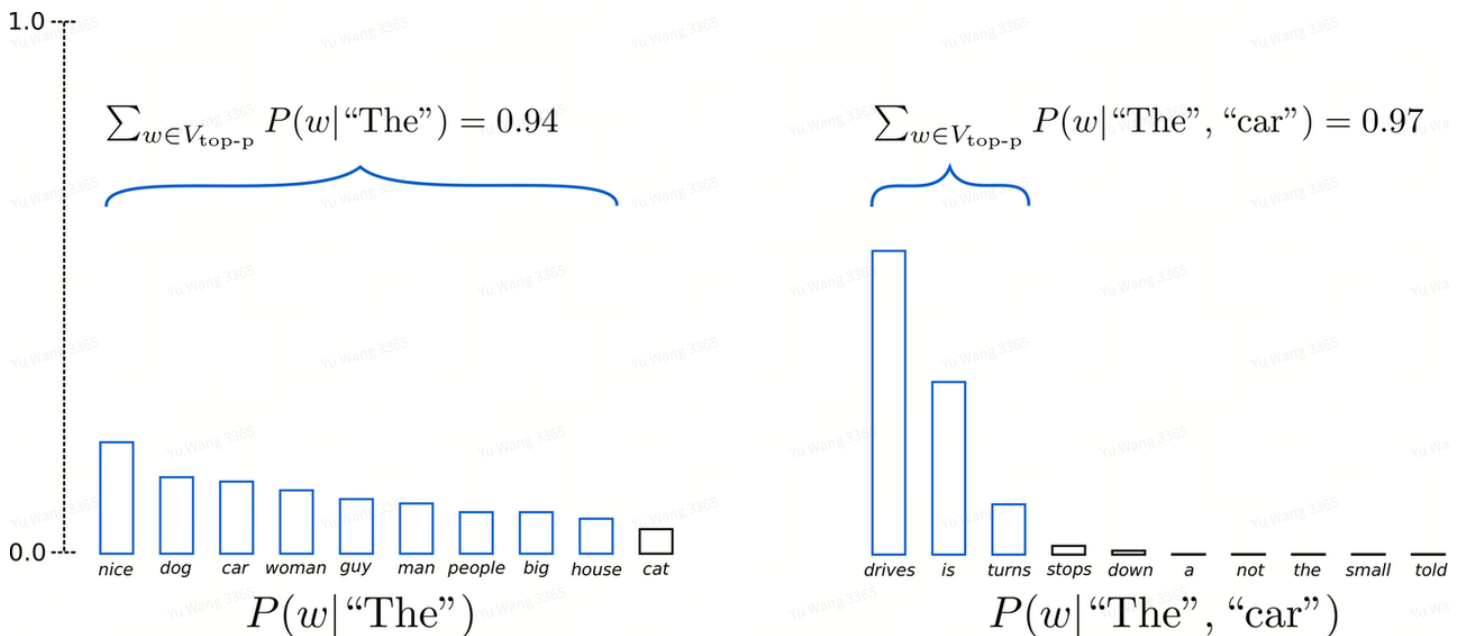
```

14         else:
15             # pluck the logits at the final step and scale by desired
temperature
16             logits = logits / temperature
17             # optionally crop the logits to only the top k options
18             if top_k is not None:
19                 v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
20                 logits[logits < v[:, [-1]]] = -float('Inf')
21             # apply softmax to convert logits to (normalized) probabilities
22             probs = F.softmax(logits, dim=-1)
23             idx_next = torch.multinomial(probs, num_samples=1)
24             # append sampled index to the running sequence and continue
25             idx = torch.cat((idx, idx_next), dim=1)
26             if idx_next==eos:
27                 break
28         return idx

```

Top-P Sampling

在累积概率超过设定值P的最小单词集中采样



大模型训练&推理

手撕flash-attention



FlashAttention是一种优化的注意力机制实现，旨在提高大模型中的注意力计算效率，特别是降低了内存占用和加速计算。它通过将常规的注意力计算步骤重组为块状矩阵运算，并通

过减少中间结果的存储来减少内存的占用。FlashAttention使用了部分矩阵乘法和片段(chunked) 计算, 以优化内存使用, 特别适用于长序列的计算。

1. FlashAttention的基本步骤

在常规的注意力计算中, 我们有:

Query、Key 和 Value 矩阵的计算

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v$$

注意力分数的计算

$$\text{Attention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

FlashAttention将这个过程优化为分块操作, 通过减少矩阵乘法中间结果的存储需求来减少内存占用。

2. 手撕FlashAttention代码

代码块

```
1  import torch
2  import torch.nn.functional as F
3
4  class FlashAttention(torch.nn.Module):
5      def __init__(self, embed_dim, num_heads, scale=None):
6          super(FlashAttention, self).__init__()
7          self.embed_dim = embed_dim
8          self.num_heads = num_heads
9          self.head_dim = embed_dim // num_heads
10         self.scale = scale or self.head_dim ** -0.5
11
12         # 初始化 Query, Key, Value 的线性映射
13         self.q_proj = torch.nn.Linear(embed_dim, embed_dim)
14         self.k_proj = torch.nn.Linear(embed_dim, embed_dim)
15         self.v_proj = torch.nn.Linear(embed_dim, embed_dim)
16         self.out_proj = torch.nn.Linear(embed_dim, embed_dim)
17
18     def forward(self, x):
19         batch_size, seq_len, embed_dim = x.size()
20
21         # 计算 Q, K, V 矩阵
22         Q = self.q_proj(x).view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)
23         K = self.k_proj(x).view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)
24         V = self.v_proj(x).view(batch_size, seq_len, self.num_heads,
self.head_dim).transpose(1, 2)
25
```

```

26     # 初始化输出
27     out = torch.zeros_like(Q)
28     block_size = 64 # 分块大小, 可调整以优化性能
29
30     for i in range(0, seq_len, block_size):
31         # 取当前块的 Key 和 Value
32         K_block = K[:, :, i:i + block_size]
33         V_block = V[:, :, i:i + block_size]
34
35         # 初始化块内的注意力得分和 softmax 缓存
36         attention_scores = torch.zeros(batch_size, self.num_heads,
seq_len, block_size, device=x.device)
37         max_scores = torch.full((batch_size, self.num_heads, seq_len, 1),
float('-inf'), device=x.device)
38         softmax_sum = torch.zeros(batch_size, self.num_heads, seq_len, 1,
device=x.device)
39
40         for j in range(0, seq_len, block_size):
41             # 取 Query 当前块
42             Q_block = Q[:, :, j:j + block_size]
43
44             # 计算 Q_block 和 K_block 的相似度, 并进行缩放
45             attention_chunk = torch.matmul(Q_block, K_block.transpose(-1,
-2)) * self.scale
46
47             # 使用局部 softmax 减少内存开销
48             max_scores = torch.maximum(max_scores,
attention_chunk.max(dim=-1, keepdim=True)[0])
49             exp_scores = torch.exp(attention_chunk - max_scores)
50             softmax_sum += exp_scores.sum(dim=-1, keepdim=True)
51
52             # 计算注意力输出
53             attention_scores[:, :, j:j + block_size] = exp_scores /
softmax_sum
54
55             # 累积输出
56             out[:, :, i:i + block_size] = torch.matmul(attention_scores,
V_block)
57
58             # 通过线性层返回最终输出
59             out = out.transpose(1, 2).contiguous().view(batch_size, seq_len,
embed_dim)
60         return self.out_proj(out)
61
62     # 示例输入
63     x = torch.randn(2, 128, 512) # batch_size = 2, seq_len = 128, embed_dim = 512
64     flash_attention = FlashAttention(embed_dim=512, num_heads=8)

```

```
65 output = flash_attention(x)
66 print(output)
```

3. 逐行解释

1. `__init__` 方法:

- `embed_dim`: 输入嵌入的维度 (例如 512)。
- `num_heads`: 多头注意力机制的头数。
- `scale`: 缩放因子, 通常为 $1/\sqrt{d_k}$, 即 `head_dim` 的倒数。

初始化中, 我们使用线性层 `q_proj`, `k_proj`, `v_proj` 来生成 Query、Key、Value 矩阵, 同时 `out_proj` 负责最后的输出投影。

2. `forward` 方法:

- 首先, 将输入 `x` 通过线性层生成 Query、Key 和 Value 矩阵, 并重塑为 `(batch_size, num_heads, seq_len, head_dim)` 的形状。
- **分块处理**: 通过 `for` 循环将 Key 和 Value 矩阵按块分割, 每次处理一个大小为 `block_size` 的子块。
 - 对每个 Query 块和 Key 块进行矩阵乘法, 计算注意力得分。
 - 对得分进行局部的 `softmax` 操作, 使用块内最大值减少浮点数溢出的可能。
- 使用得到的注意力得分与对应的 Value 块进行矩阵乘法, 得到最终的输出。

3. 分块计算: `FlashAttention` 的关键在于分块处理, 使得在处理长序列时, 减少了整个注意力矩阵的显式存储需求, 每次只计算并存储一个块的注意力得分, 从而减少内存消耗。

强化学习与策略优化

Q-learning 算法

下面是一个使用 Python 实现 Q-learning 算法的例子。这段代码假设我们在一个简单的网格环境 (例如迷宫) 中。为了简化起见, 环境的细节是抽象的。

```
1 import numpy as np
2 import random
3
4 class QLearningAgent:
5     def __init__(self, n_states, n_actions, learning_rate=0.1,
6                   discount_factor=0.9, exploration_rate=0.1):
7         self.n_states = n_states
8         self.n_actions = n_actions
```

```

8         self.learning_rate = learning_rate
9         self.discount_factor = discount_factor
10        self.exploration_rate = exploration_rate
11        self.q_table = np.zeros((n_states, n_actions))
12
13    def choose_action(self, state):
14        # Epsilon-greedy strategy for action selection
15        if random.uniform(0, 1) < self.exploration_rate:
16            return random.choice(range(self.n_actions))
17        else:
18            return np.argmax(self.q_table[state])
19
20    def update(self, state, action, reward, next_state):
21        # Q-learning update rule
22        best_next_action = np.argmax(self.q_table[next_state])
23        td_target = reward + self.discount_factor * self.q_table[next_state,
best_next_action]
24        td_delta = td_target - self.q_table[state, action]
25        self.q_table[state, action] += self.learning_rate * td_delta
26
27    # Example usage:
28    def train_agent(n_episodes=100):
29        n_states = 5 # Example state space size
30        n_actions = 2 # Example action space size
31        agent = QLearningAgent(n_states, n_actions)
32
33        for episode in range(n_episodes):
34            state = np.random.randint(0, n_states) # Randomly initialize the
starting state
35            done = False
36            while not done:
37                action = agent.choose_action(state)
38                # Example transition dynamics
39                next_state = (state + 1) % n_states
40                reward = 1 if next_state == n_states - 1 else 0
41                done = next_state == n_states - 1
42
43                agent.update(state, action, reward, next_state)
44                state = next_state
45
46            return agent.q_table
47
48    # Train the agent and print the resulting Q-table
49    q_table = train_agent()
50    print("Trained Q-Table:")
51    print(q_table)

```

🌱 说明

- **状态和动作空间:** 代码中定义了一个简单的状态和动作空间，该例子中状态和动作数量分别为5和2。这个配置可以根据你的具体问题进行调整。
- **选择动作:** 使用 ϵ -贪婪策略 (epsilon-greedy) 来选择动作，其中 `exploration_rate` 控制探索的概率。
- **更新Q表:** 根据Q-learning的更新规则，采用 `learning_rate` (学习率)和 `discount_factor` (折扣因子)来更新Q值。
- **训练循环:** 在每个episode中，代理不断选择动作、观察结果，并更新Q-table，直到达到终止状态。

REINFORCE算法

下面是一个使用Python实现的REINFORCE算法的示例。REINFORCE是一种策略梯度方法，用于训练政策的无模型强化学习算法。

```
1  import numpy as np
2  import gym
3  import torch
4  import torch.nn as nn
5  import torch.optim as optim
6
7  # 定义策略网络
8  class PolicyNetwork(nn.Module):
9      def __init__(self, state_dim, action_dim, hidden_dim=128):
10         super(PolicyNetwork, self).__init__()
11         self.fc1 = nn.Linear(state_dim, hidden_dim)
12         self.fc2 = nn.Linear(hidden_dim, action_dim)
13         self.softmax = nn.Softmax(dim=1)
14
15     def forward(self, x):
16         x = torch.relu(self.fc1(x))
17         x = self.fc2(x)
18         x = self.softmax(x)
19         return x
20
21 # REINFORCE算法
22 def reinforce(env, policy, optimizer, num_episodes=1000, gamma=0.99):
23     policy.train() # 进入训练模式
24
25     for episode in range(num_episodes):
26         state = env.reset()
```

```

27     rewards = []
28     log_probs = []
29
30     # 生成一条完整的轨迹
31     done = False
32     while not done:
33         state_tensor = torch.FloatTensor(state).unsqueeze(0)
34         action_probs = policy(state_tensor)
35         action =
np.random.choice(len(action_probs.squeeze().detach().numpy()),
p=action_probs.squeeze().detach().numpy())
36
37         log_prob = torch.log(action_probs.squeeze(0)[action])
38         log_probs.append(log_prob)
39
40         next_state, reward, done, _ = env.step(action)
41         rewards.append(reward)
42         state = next_state
43
44     # 计算奖励的折扣总和
45     discounted_rewards = []
46     R = 0
47     for r in reversed(rewards):
48         R = r + gamma * R
49         discounted_rewards.insert(0, R)
50
51     discounted_rewards = torch.tensor(discounted_rewards)
52     # 标准化回报
53     discounted_rewards = (discounted_rewards - discounted_rewards.mean())
/ (discounted_rewards.std() + 1e-9)
54
55     # 计算损失
56     loss = 0
57     for log_prob, Gt in zip(log_probs, discounted_rewards):
58         loss -= log_prob * Gt
59
60     # 反向传播更新策略网络
61     optimizer.zero_grad()
62     loss.backward()
63     optimizer.step()
64
65     # 每个回合的输出
66     total_reward = sum(rewards)
67     print(f"Episode {episode+1}: Total Reward = {total_reward}")
68
69     # 环境和算法参数
70     env = gym.make('CartPole-v1')

```

```

71 state_dim = env.observation_space.shape[0]
72 action_dim = env.action_space.n
73
74 # 实例化策略网络与优化器
75 policy = PolicyNetwork(state_dim, action_dim)
76 optimizer = optim.Adam(policy.parameters(), lr=0.01)
77
78 # 执行REINFORCE算法
79 reinforce(env, policy, optimizer, num_episodes=1000)
80
81 # 关闭训练环境
82 env.close()

```

🌱 说明

- **策略网络**：一个简单的全连接神经网络，通过softmax输出每个动作的概率。
- **REINFORCE算法**：这个实现中为每个回合计算累计折扣奖励和策略损失，并对政策网络进行梯度更新。
- **环境**：在OpenAI Gym库中的 `CartPole-v1` 环境中运行。
- **Hyperparameters**：学习率为0.01，总共1000个训练回合，折扣因子（gamma）为0.99。

深度Q网络（DQN）

实现深度Q网络（DQN）是一个复杂的任务，但是我可以为您提供一个基本的示例代码，帮助您了解DQN的工作原理。以下是一个简单的DQN实现，使用了PyTorch库来创建深度神经网络，并与OpenAI Gym集成进行环境交互。

```

1  import gym
2  import torch
3  import torch.nn as nn
4  import torch.optim as optim
5  import random
6  import numpy as np
7  from collections import deque
8
9  # 超参数定义
10 GAMMA = 0.99
11 BATCH_SIZE = 64
12 REPLAY_MEMORY_SIZE = 10000
13 LEARNING_RATE = 0.001
14 EPSILON_START = 1.0

```

```

15 EPSILON_END = 0.1
16 EPSILON_DECAY = 0.995
17
18 # Q网络的定义
19 class QNetwork(nn.Module):
20     def __init__(self, state_size, action_size):
21         super(QNetwork, self).__init__()
22         self.fc1 = nn.Linear(state_size, 24)
23         self.fc2 = nn.Linear(24, 24)
24         self.fc3 = nn.Linear(24, action_size)
25
26     def forward(self, x):
27         x = torch.relu(self.fc1(x))
28         x = torch.relu(self.fc2(x))
29         return self.fc3(x)
30
31 class DQN:
32     def __init__(self, state_size, action_size):
33         self.state_size = state_size
34         self.action_size = action_size
35         self.memory = deque(maxlen=REPLAY_MEMORY_SIZE)
36         self.epsilon = EPSILON_START
37         self.q_network = QNetwork(state_size, action_size)
38         self.optimizer = optim.Adam(self.q_network.parameters(),
lr=LEARNING_RATE)
39
40     def remember(self, state, action, reward, next_state, done):
41         self.memory.append((state, action, reward, next_state, done))
42
43     def act(self, state):
44         if random.random() <= self.epsilon:
45             return random.randrange(self.action_size)
46         else:
47             with torch.no_grad():
48                 state_t = torch.FloatTensor(state).unsqueeze(0)
49                 q_values = self.q_network(state_t)
50                 return np.argmax(q_values.numpy())
51
52     def replay(self):
53         if len(self.memory) < BATCH_SIZE:
54             return
55
56         batch = random.sample(self.memory, BATCH_SIZE)
57         state, action, reward, next_state, done = zip(*batch)
58
59         state = torch.FloatTensor(state)
60         action = torch.LongTensor(action).unsqueeze(1)

```

```

61     reward = torch.FloatTensor(reward)
62     next_state = torch.FloatTensor(next_state)
63     done = torch.FloatTensor(done)
64
65     current_q = self.q_network(state).gather(1, action)
66     max_next_q = self.q_network(next_state).max(1)[0]
67     expected_q = reward + (GAMMA * max_next_q * (1 - done))
68
69     loss = nn.MSELoss()(current_q.squeeze(), expected_q)
70
71     self.optimizer.zero_grad()
72     loss.backward()
73     self.optimizer.step()
74
75     if self.epsilon > EPSILON_END:
76         self.epsilon *= EPSILON_DECAY
77
78 # 主程序
79 if __name__ == "__main__":
80     env = gym.make('CartPole-v0')
81     state_size = env.observation_space.shape[0]
82     action_size = env.action_space.n
83
84     dqn = DQN(state_size, action_size)
85     episodes = 1000
86
87     for episode in range(episodes):
88         state = env.reset()
89         total_reward = 0
90         for t in range(200):
91             action = dqn.act(state)
92             next_state, reward, done, _ = env.step(action)
93             total_reward += reward
94             dqn.remember(state, action, reward, next_state, done)
95             state = next_state
96             if done:
97                 print(f"Episode: {episode+1}/{episodes}, Score: {t+1},
Epsilon: {dqn.epsilon:.2}")
98                 break
99             dqn.replay()

```

说明

- **QNetwork:** 这是我们的神经网络，用于近似Q值。它由三层全连接层构成。
- **DQN类:** 这个类管理重放记忆、选择动作和更新网络。

- **remember**方法: 保存经验到重放记忆。
- **act**方法: 基于 ϵ -greedy策略选择动作。
- **replay**方法: 采样批量经验更新网络。

PPO

下面是一个基本示例, 使用 `PyTorch` 库。假设环境是 OpenAI Gym 提供的, 如CartPole-v1环境。

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.distributions import Categorical
5 import gym
6 import numpy as np
7
8 class ActorCritic(nn.Module):
9     def __init__(self, state_dim, action_dim):
10         super(ActorCritic, self).__init__()
11         self.actor = nn.Sequential(
12             nn.Linear(state_dim, 128),
13             nn.ReLU(),
14             nn.Linear(128, action_dim),
15             nn.Softmax(dim=-1),
16         )
17
18         self.critic = nn.Sequential(
19             nn.Linear(state_dim, 128),
20             nn.ReLU(),
21             nn.Linear(128, 1)
22         )
23
24     def forward(self, x):
25         action_probs = self.actor(x)
26         state_value = self.critic(x)
27         return action_probs, state_value
28
29 class PPO:
30     def __init__(self, state_dim, action_dim, lr=3e-4, gamma=0.99,
31 eps_clip=0.2, K_epochs=4):
32         self.policy = ActorCritic(state_dim, action_dim).to(device)
33         self.optimizer = optim.Adam(self.policy.parameters(), lr=lr)
34         self.policy_old = ActorCritic(state_dim, action_dim).to(device)
35         self.policy_old.load_state_dict(self.policy.state_dict())
```

```

35         self.MseLoss = nn.MSELoss()
36
37         self.gamma = gamma
38         self.eps_clip = eps_clip
39         self.K_epochs = K_epochs
40
41     def select_action(self, state):
42         with torch.no_grad():
43             state = torch.FloatTensor(state).to(device)
44             action_probs, _ = self.policy_old(state)
45             dist = Categorical(action_probs)
46             action = dist.sample()
47             return action.item(), dist.log_prob(action)
48
49     def update(self, memory):
50         # Convert list items to tensor
51         states = torch.FloatTensor(memory.states).to(device)
52         actions = torch.LongTensor(memory.actions).to(device)
53         logprobs = torch.FloatTensor(memory.logprobs).to(device)
54         rewards = memory.rewards
55         is_terminals = memory.is_terminals
56
57         # Monte Carlo estimate of rewards
58         returns = []
59         discounted_reward = 0
60         for reward, is_terminal in zip(reversed(rewards),
reversed(is_terminals)):
61             if is_terminal:
62                 discounted_reward = 0
63                 discounted_reward = reward + (self.gamma * discounted_reward)
64                 returns.insert(0, discounted_reward)
65
66         # Normalize the rewards
67         returns = torch.tensor(returns).to(device)
68         returns = (returns - returns.mean()) / (returns.std() + 1e-5)
69
70         for _ in range(self.K_epochs):
71             # Evaluating old actions and values
72             action_probs, state_values = self.policy(states)
73             dist = Categorical(action_probs)
74             new_logprobs = dist.log_prob(actions)
75             state_values = state_values.squeeze()
76
77             # Finding the ratio (pi_theta / pi_theta__old)
78             ratios = torch.exp(new_logprobs - logprobs.detach())
79
80             # Finding Surrogate Loss

```

```

81         advantages = returns - state_values.detach()
82         surr1 = ratios * advantages
83         surr2 = torch.clamp(ratios, 1-self.eps_clip, 1+self.eps_clip) *
advantages
84         loss = -torch.min(surr1, surr2) + 0.5*self.MseLoss(state_values,
returns)
85
86         # Taking gradient step
87         self.optimizer.zero_grad()
88         loss.mean().backward()
89         self.optimizer.step()
90
91         # Copy new weights into old policy
92         self.policy_old.load_state_dict(self.policy.state_dict())
93
94     class Memory:
95         def __init__(self):
96             self.actions = []
97             self.states = []
98             self.logprobs = []
99             self.rewards = []
100            self.is_terminals = []
101
102        def clear_memory(self):
103            del self.actions[:]
104            del self.states[:]
105            del self.logprobs[:]
106            del self.rewards[:]
107            del self.is_terminals[:]
108
109    # Device configuration
110    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
111
112    def main():
113        ##### Hyperparameters #####
114        env_name = "CartPole-v1"
115        render = False
116        solved_reward = 230          # stop training if avg_reward > solved_reward
117        log_interval = 20          # print avg reward in the interval
118        max_episodes = 3000        # max training episodes
119        max_timesteps = 300        # max timesteps in one episode
120
121        update_timestep = 2000     # update policy every n timesteps
122        action_std = 0.5           # constant std for action distribution
123
124        K_epochs = 4              # update policy for K epochs
125        eps_clip = 0.2            # clip parameter for PPO

```

```

126     gamma = 0.99                                # discount factor
127
128     lr = 0.002                                    # parameters for Adam optimizer
129     betas = (0.9, 0.999)
130     #####
131
132     env = gym.make(env_name)
133     state_dim = env.observation_space.shape[0]
134     action_dim = env.action_space.n
135
136     memory = Memory()
137     ppo = PPO(state_dim, action_dim, lr, gamma, eps_clip, K_epochs)
138     print(lr)
139
140     # logging variables
141     running_reward = 0
142     avg_length = 0
143     timestep = 0
144
145     # training loop
146     for i_episode in range(1, max_episodes+1):
147         state = env.reset()
148         for t in range(max_timesteps):
149             timestep += 1
150
151             # Running policy_old:
152             action, logprob = ppo.select_action(state)
153             state, reward, done, _ = env.step(action)
154
155             # Saving reward and is_terminals:
156             memory.states.append(state)
157             memory.actions.append(action)
158             memory.logprobs.append(logprob)
159             memory.rewards.append(reward)
160             memory.is_terminals.append(done)
161
162             # update if its time
163             if timestep % update_timestep == 0:
164                 ppo.update(memory)
165                 memory.clear_memory()
166                 timestep = 0
167
168             running_reward += reward
169             if render:
170                 env.render()
171             if done:
172                 break

```

```

173
174         avg_length += t
175
176         # logging
177         if i_episode % log_interval == 0:
178             avg_length = int(avg_length/log_interval)
179             running_reward = int((running_reward/log_interval))
180
181             print(f'Episode {i_episode}\tAvg reward: {running_reward}\tAvg
length: {avg_length}')
182             running_reward = 0
183             avg_length = 0
184
185 if __name__ == '__main__':
186     main()

```

🌱 说明

- 使用 `ActorCritic` 类作为策略网络，它包含一个 actor 网络和一个 critic 网络。
- PPO 使用探讨-价值方式来提供网络的更新。
- `Memory` 类用于存储与每个 episode 有关的信息，用于在更新网络权重时回顾与学习。
- `select_action` 方法用于从策略网络中采样动作。
- `update` 方法用于根据采样数据更新策略。

DPO

Direct Preference Optimization (DPO) 是一种强化学习中的偏好优化方法。DPO 的目标是直接优化模型参数，以便在不同的选择之间做出优先选择，而不依赖于传统的奖励信号。

下面是一个简化的 DPO 实现代码框架。假设我们有一个环境模型（环境交互）、一个策略网络（actor）和一个偏好模型（用于优化偏好）。

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import numpy as np
6
7 # 简单的环境模拟
8 class SimpleEnvironment:
9     def __init__(self):
10         self.state_space = 5 # 假设状态空间大小为5

```

```

11     self.action_space = 3 # 假设动作空间大小为3
12
13     def reset(self):
14         # 随机初始化一个状态
15         return np.random.randint(0, self.state_space)
16
17     def step(self, action):
18         # 随机生成下一个状态, 动作选择的奖励为随机的
19         next_state = np.random.randint(0, self.state_space)
20         reward = np.random.rand()
21         return next_state, reward
22
23 # 策略网络
24 class PolicyNetwork(nn.Module):
25     def __init__(self, state_size, action_size):
26         super(PolicyNetwork, self).__init__()
27         self.fc = nn.Linear(state_size, action_size)
28
29     def forward(self, state):
30         return F.softmax(self.fc(state), dim=-1)
31
32 # 偏好模型
33 class PreferenceModel(nn.Module):
34     def __init__(self):
35         super(PreferenceModel, self).__init__()
36         self.fc = nn.Linear(1, 1) # 简单的偏好模型
37
38     def forward(self, reward):
39         return self.fc(reward)
40
41 # DPO算法实现
42 class DPOAgent:
43     def __init__(self, state_size, action_size, preference_model):
44         self.policy_network = PolicyNetwork(state_size, action_size)
45         self.preference_model = preference_model
46         self.policy_optimizer = optim.Adam(self.policy_network.parameters(),
47 lr=0.001)
48         self.preference_optimizer =
49 optim.Adam(self.preference_model.parameters(), lr=0.001)
50         self.gamma = 0.99 # 折扣因子
51
52     def get_action(self, state):
53         state_tensor = torch.tensor(state, dtype=torch.float32)
54         action_probs = self.policy_network(state_tensor)
55         action = torch.multinomial(action_probs, 1).item() # 使用多项分布进行采
56         样
57         return action

```

```

55
56     def train(self, env, num_episodes=1000):
57         for episode in range(num_episodes):
58             state = env.reset()
59             done = False
60             episode_rewards = []
61             actions = []
62             rewards = []
63
64             while not done:
65                 action = self.get_action(state)
66                 next_state, reward = env.step(action)
67
68                 episode_rewards.append(reward)
69                 actions.append(action)
70                 rewards.append(reward)
71
72                 # 计算目标, 假设 DPO 的目标是优化偏好模型
73                 # 将奖励传入偏好模型
74                 preference = self.preference_model(torch.tensor([reward],
dtype=torch.float32))
75
76                 # 使用偏好模型预测奖励的偏好
77                 preference_loss = F.mse_loss(preference,
torch.tensor([reward], dtype=torch.float32))
78
79                 # 优化偏好模型
80                 self.preference_optimizer.zero_grad()
81                 preference_loss.backward()
82                 self.preference_optimizer.step()
83
84                 state = next_state
85
86                 # 假设每一回合结束时都根据偏好模型更新策略
87                 # 使用策略梯度方法更新策略网络
88                 if done:
89                     total_reward = sum(episode_rewards)
90                     policy_loss = -
torch.mean(torch.log(self.policy_network(torch.tensor(state,
dtype=torch.float32)))) * total_reward)
91
92                     self.policy_optimizer.zero_grad()
93                     policy_loss.backward()
94                     self.policy_optimizer.step()
95
96                 if episode % 100 == 0:

```

```

97         print(f"Episode {episode}: Policy loss -
{policy_loss.item()}, Preference loss - {preference_loss.item()}")
98
99     # 初始化环境和DPO代理
100     env = SimpleEnvironment()
101     preference_model = PreferenceModel()
102     dpo_agent = DPOAgent(state_size=env.state_space,
        action_size=env.action_space, preference_model=preference_model)
103
104     # 训练DPO代理
105     dpo_agent.train(env, num_episodes=1000)
106

```

🌱 说明

环境模拟 (`SimpleEnvironment`): 模拟一个简单的环境，它包含一个有限的状态空间和动作空间，并能够返回一个随机的奖励。 `reset()` 方法返回一个初始状态， `step()` 方法接收动作并返回下一个状态和奖励。

策略网络 (`PolicyNetwork`): 一个简单的神经网络模型，接受当前状态作为输入，输出一个概率分布，表示各个动作的选择概率。该模型使用softmax激活函数来产生动作的概率分布。

偏好模型 (`PreferenceModel`): 一个简单的线性模型，接受奖励作为输入并输出偏好评分。在实际DPO中，偏好模型用于根据奖励生成偏好信息，从而优化决策过程。

DPO代理 (`DPOAgent`): 代理类，其中包含了策略网络和偏好模型的训练过程。在每个回合中，代理根据当前策略网络来选择动作并与环境交互。然后，奖励会传递给偏好模型，偏好模型会输出偏好评分。接着，代理使用该偏好评分来更新策略网络的参数。

训练过程: 代理通过多个回合进行训练，每回合结束时都会计算总奖励并通过反向传播来更新策略网络。偏好模型使用均方误差损失进行训练，以优化其对奖励的偏好预测。

蒙特卡洛树搜索 (MCTS)

以下是一个简单的蒙特卡洛树搜索 (MCTS) 算法的实现代码。此实现用于一个回合制的两人游戏，并且假设游戏状态和动作被抽象为对象。为了简单起见，这个例子实现了一个假想的井字棋 (Tic-Tac-Toe) 游戏。

```

1     import math
2     import random
3
4     class MCTSNode:
5         def __init__(self, state, parent=None):

```

```

6         self.state = state
7         self.parent = parent
8         self.children = []
9         self.visits = 0
10        self.wins = 0
11
12        def is_fully_expanded(self):
13            return len(self.children) == len(self.state.get_legal_actions())
14
15        def best_child(self, c_param=1.4):
16            choices_weights = [
17                (child.wins / child.visits) + c_param * math.sqrt((2 *
math.log(self.visits) / child.visits))
18                for child in self.children
19            ]
20            return self.children[choices_weights.index(max(choices_weights))]
21
22        def expand(self):
23            for action in self.state.get_legal_actions():
24                if not any(child.state == self.state.do_action(action) for child
in self.children):
25                    new_state = self.state.do_action(action)
26                    child_node = MCTSNode(state=new_state, parent=self)
27                    self.children.append(child_node)
28                    return child_node
29            raise Exception("No actions to expand.")
30
31        def is_terminal_node(self):
32            return self.state.is_terminal()
33
34        def rollout(self):
35            current_state = self.state
36            while not current_state.is_terminal():
37                possible_actions = current_state.get_legal_actions()
38                action = random.choice(possible_actions)
39                current_state = current_state.do_action(action)
40            return current_state.reward()
41
42        def backpropagate(self, reward):
43            self.visits += 1
44            self.wins += reward
45            if self.parent:
46                self.parent.backpropagate(reward)
47
48    class MCTS:
49        def __init__(self, root):
50            self.root = root

```

```

51
52     def best_action(self, simulations_number=1000):
53         for _ in range(simulations_number):
54             v = self.tree_policy()
55             reward = v.rollout()
56             v.backpropagate(reward)
57         return self.root.best_child(c_param=0).state
58
59     def tree_policy(self):
60         current_node = self.root
61         while not current_node.is_terminal_node():
62             if not current_node.is_fully_expanded():
63                 return current_node.expand()
64             else:
65                 current_node = current_node.best_child()
66         return current_node
67
68     # Assumed Game State (stub for Tic-Tac-Toe)
69     class GameState:
70         def __init__(self, board, player_turn):
71             self.board = board
72             self.player_turn = player_turn
73
74         def get_legal_actions(self):
75             # Replace with actual logic for obtaining legal moves.
76             pass
77
78         def do_action(self, action):
79             # Replace with actual logic to update the game state with the action.
80             pass
81
82         def is_terminal(self):
83             # Replace with actual logic to check if the game is in a terminal
84             state.
85             pass
86
87         def reward(self):
88             # Replace with actual logic for calculating the reward. +1 for win,
89             -1 for loss, 0 for draw.
90             pass
91
92     # Example use case assuming a GameState initialization
93     initial_state = GameState(board=[[None]*3 for _ in range(3)], player_turn=1)
94     root = MCTSNode(state=initial_state)
95     mcts = MCTS(root=root)
96     best_next_state = mcts.best_action(simulations_number=1000)

```

```
96 # Output the result
97 print("Best action leads to state:")
98 print(best_next_state.board)
```

🌱 说明

这个例子假设 `GameState` 类里实现了以下方法：

- `get_legal_actions()`：返回当前状态可采取的合法动作列表。
- `do_action(action)`：接受一个动作并返回执行该动作后的新状态。
- `is_terminal()`：检查游戏是否已进入结束状态。
- `reward()`：返回当前状态的奖励值。

这些方法需要根据特定的游戏逻辑进行具体实现。以上代码提供了一个框架，通过它可以实现 MCTS 算法，用于在游戏中模拟和选择策略。

A*搜索算法

下面是用Python实现的A*搜索算法。A*搜索算法是一种启发式搜索算法，用于在图中查找从起始节点到目标节点的最短路径。该算法结合了广度优先搜索和贪心最佳优先搜索的思想，以保证找到最低成本的路径。

```
1 import heapq
2
3 def heuristic(a, b):
4     """估计函数，用于估计从a到b的距离"""
5     return abs(a[0] - b[0]) + abs(a[1] - b[1])
6
7 def a_star_search(grid, start, goal):
8     """实现A*搜索算法"""
9     neighbors = [(0, 1), (1, 0), (0, -1), (-1, 0)] # 4个可能的移动方向
10
11     # open_set是我们将要探索的节点，使用优先级队列
12     open_set = []
13     heapq.heappush(open_set, (0, start))
14
15     came_from = {} # 记录路径
16
17     # g_score记录从起点到某点的最小代价
18     g_score = {start: 0}
19
20     # f_score = g_score + heuristic
21     f_score = {start: heuristic(start, goal)}
```

```

22
23     while open_set:
24         current = heapq.heappop(open_set)[1]
25
26         if current == goal: # 找到了目标节点
27             # 重构路径
28             path = []
29             while current in came_from:
30                 path.append(current)
31                 current = came_from[current]
32             path.append(start) # 最后加上起始点
33             path.reverse() # 反转路径
34             return path
35
36         for dx, dy in neighbors: # 遍历邻居
37             neighbor = (current[0] + dx, current[1] + dy)
38             tentative_g_score = g_score[current] + 1 # 假设移动的代价为1
39
40             if 0 <= neighbor[0] < len(grid) and 0 <= neighbor[1] <
len(grid[0]) and grid[neighbor[0]][neighbor[1]] == 0:
41                 if neighbor not in g_score or tentative_g_score <
g_score[neighbor]:
42                     # 更新g_score和f_score
43                     came_from[neighbor] = current
44                     g_score[neighbor] = tentative_g_score
45                     f_score[neighbor] = tentative_g_score +
heuristic(neighbor, goal)
46                 if neighbor not in [i[1] for i in open_set]:
47                     heapq.heappush(open_set, (f_score[neighbor],
neighbor))
48
49         return None # 如果没有路径则返回None
50
51 # 用法示例
52 grid = [
53     [0, 1, 0, 0, 0],
54     [0, 1, 0, 1, 0],
55     [0, 0, 0, 1, 0],
56     [0, 1, 1, 1, 0],
57     [0, 0, 0, 0, 0]
58 ]
59 start = (0, 0)
60 goal = (4, 4)
61
62 path = a_star_search(grid, start, goal)
63 print("找到的路径:", path)

```

🌱 说明

- `grid` : 二维数组表示的地图，其中0表示可通行区域，1表示障碍。
- `heuristic` : 启发式函数，采用曼哈顿距离。
- `a_star_search` 函数实现了A*搜索：
 - `open_set` : 优先队列，保存将要探索的节点。
 - `came_from` : 用于记录路径。
 - `g_score` 和 `f_score` : 分别记录从起点到当前节点的代价和估计总代价。

标准梯度下降 (Gradient Descent) 算法

下面是一个基本的标准梯度下降 (Gradient Descent) 算法的Python实现。这段代码将为一个简单的线性回归模型找到最优参数，其中我们尝试最小化均方误差损失函数。

```
1  import numpy as np
2
3  # 样本数据 (X: 特征, y: 标签)
4  X = np.array([1, 2, 3, 4, 5])
5  y = np.array([2, 4, 6, 8, 10])
6
7  # 初始化参数
8  theta = 0.0 # 假设只有一个参数
9  learning_rate = 0.01
10 num_iterations = 1000
11
12 # 梯度下降算法
13 def gradient_descent(X, y, theta, learning_rate, num_iterations):
14     m = len(y) # 样本数量
15
16     for _ in range(num_iterations):
17         # 计算预测值
18         predictions = X * theta
19
20         # 计算损失函数的梯度
21         gradient = (1/m) * np.sum((predictions - y) * X)
22
23         # 更新参数
24         theta = theta - learning_rate * gradient
25
26         # 打印每次迭代的损失值, 帮助观察收敛情况
27         loss = (1/(2*m)) * np.sum((predictions - y)**2)
28         print(f'Iteration: {_,} Loss: {loss:.4f}, Theta: {theta:.4f}')
```

```
29
30     return theta
31
32 # 使用梯度下降算法找到最优参数
33 optimal_theta = gradient_descent(X, y, theta, learning_rate, num_iterations)
34 print(f'Optimal Theta: {optimal_theta:.4f}')
```

🌱 说明

- **样本数据 (X, y):** 传统上, X 代表特征, y 代表标签。在这个简单例子中, 我们使用一维数据集。
- **参数初始化 (theta):** 我们从一个任意值 `theta` 开始 (这里是0)。
- **学习率 (learning_rate):** 学习率决定了每次更新时移动的步长。太大可能会导致不收敛, 太小可能会导致收敛速度太慢。
- **迭代次数 (num_iterations):** 运行梯度下降的次数, 可以调整这个参数以确保收敛。
- **梯度下降流程:** 计算预测值, 计算梯度, 更新参数, 并打印损失以观察收敛情况。
- **目标:** 找到一个可以最小化损失函数的参数 `theta`。在这个例子中, 由于数据是简单的线性关系, 最优 `theta` 应该接近 2。

Adam优化器

下面是一个使用Python实现的Adam优化器代码示例。Adam (自适应矩估计) 是一种基于一阶和二阶矩估计的自适应学习率优化算法, 广泛应用于深度学习中。

```
1 class AdamOptimizer:
2     def __init__(self, params, lr=0.001, beta1=0.9, beta2=0.999, epsilon=1e-
3         8):
4         """
5         初始化Adam优化器
6
7         参数:
8         - params: 需要优化的参数列表
9         - lr (float): 学习率 (基础学习率)
10        - beta1 (float): 一阶动量估计的指数衰减率
11        - beta2 (float): 二阶动量估计的指数衰减率
12        - epsilon (float): 防止除零操作的小常数
13        """
14        self.params = params
15        self.lr = lr
16        self.beta1 = beta1
17        self.beta2 = beta2
```

```

17         self.epsilon = epsilon
18         self.m = [0 for _ in params] # 一阶矩估计
19         self.v = [0 for _ in params] # 二阶矩估计
20         self.t = 0 # 时间步
21
22     def step(self, grads):
23         """
24         更新参数
25
26         参数:
27         - grads: 参数的梯度列表
28         """
29         self.t += 1
30         lr_t = self.lr * (1 - self.beta2**self.t)**0.5 / (1 -
self.beta1**self.t)
31
32         for i in range(len(self.params)):
33             # 更新一阶和二阶矩估计
34             self.m[i] = self.beta1 * self.m[i] + (1 - self.beta1) * grads[i]
35             self.v[i] = self.beta2 * self.v[i] + (1 - self.beta2) *
(grads[i]**2)
36
37             # 计算偏移校正后的梯度并更新参数
38             m_hat = self.m[i] / (1 - self.beta1**self.t)
39             v_hat = self.v[i] / (1 - self.beta2**self.t)
40             self.params[i] -= lr_t * m_hat / (v_hat**0.5 + self.epsilon)
41
42     # 使用示例
43     # 初始化参数和梯度
44     params = [1.5, 2.5, 3.5]
45     grads = [0.01, 0.02, 0.03]
46
47     # 创建Adam优化器实例
48     adam = AdamOptimizer(params)
49
50     # 执行优化
51     print("参数更新前:", params)
52     adam.step(grads)
53     print("参数更新后:", params)

```

该代码定义了一个 `AdamOptimizer` 类，通过初始化时传入参数和优化器超参数（学习率、`beta1`、`beta2` 和 `epsilon`）来创建 Adam 优化器。`step` 方法用于更新每个参数的值。这个简单的示例展示了如何用 Adam 优化器更新参数。

学习率调度 (Learning Rate Scheduling)

下面是一个使用Python实现学习率调度的示例代码。这里我们使用的是PyTorch库，其中包含了学习率调度器的实现。我们将使用 `StepLR` 调度器作为示例。`StepLR` 在每个固定的epoch数之后，将学习率降低一定的比例。

```
1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  import torchvision.models as models
5
6  # 模型初始化 (以ResNet18为例)
7  model = models.resnet18(pretrained=False)
8
9  # 定义损失函数和优化器
10 criterion = nn.CrossEntropyLoss()
11 optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
12
13 # 定义学习率调度器
14 scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
15
16 # 示例训练过程
17 num_epochs = 30
18 for epoch in range(num_epochs):
19     # 模拟训练步骤
20     model.train()
21
22     # 假设这里有以下训练步骤
23     # inputs, labels = ...
24     # optimizer.zero_grad()
25     # outputs = model(inputs)
26     # loss = criterion(outputs, labels)
27     # loss.backward()
28     # optimizer.step()
29
30     # 更新学习率
31     scheduler.step()
32
33     # 输出当前学习率
34     print(f"Epoch {epoch+1}/{num_epochs}, Current LR: {scheduler.get_last_lr()}
35           [0]:.6f}")
```

说明

1. **模型初始化:** 创建了一个简单的ResNet18模型。
2. **优化器和损失函数:** 选择了SGD优化器和交叉熵损失函数。

3. **学习率调度器:** 使用 `StepLR` 来调整学习率。 `step_size=10` 表示每10个epoch学习率减小一次。 `gamma=0.1` 表示学习率每次减小到原来的0.1倍。
4. **训练过程:** 在每个epoch中进行训练（此处省略具体训练步骤），并在每个epoch末更新学习率。
5. **输出:** 在每个epoch结束时输出当前学习率。

L2正则化

下面是一个实现L2正则化的Python代码示例，使用NumPy库来进行矩阵运算。L2正则化有助于防止过拟合，它通过在损失函数中加入权重的平方和项来惩罚过大的权重。

```
1 import numpy as np
2
3 def compute_cost(X, y, theta, lambda_):
4     """
5     计算带有L2正则化项的线性回归成本函数。
6
7     参数:
8     X -- 输入特征矩阵 (m x n), m是样本数, n是特征数.
9     y -- 输出标签向量 (m x 1).
10    theta -- 权重参数向量 (n x 1).
11    lambda_ -- 正则化参数.
12
13    返回:
14    J -- 成本函数的值.
15    """
16    m = y.size
17    predictions = X.dot(theta)
18    cost = (1/(2*m)) * np.sum(np.square(predictions - y))
19    reg_term = (lambda_/(2*m)) * np.sum(np.square(theta[1:])) # 不包括偏置项
20    J = cost + reg_term
21    return J
22
23 def gradient_descent(X, y, theta, alpha, lambda_, num_iters):
24     """
25     带有L2正则化的梯度下降算法。
26
27     参数:
28     X -- 输入特征矩阵 (m x n).
29     y -- 输出标签向量 (m x 1).
30     theta -- 初始化权重参数向量 (n x 1).
31     alpha -- 学习率.
32     lambda_ -- 正则化参数.
```

```

33     num_iters -- 迭代次数.
34
35     返回:
36     theta -- 优化后的权重参数.
37     J_history -- 每次迭代的成本值.
38     """
39     m = y.size
40     J_history = []
41
42     for _ in range(num_iters):
43         predictions = X.dot(theta)
44         errors = predictions - y
45         gradient = (1/m) * X.T.dot(errors)
46         reg_term = (lambda_/m) * np.r_[[0]], theta[1:].reshape(-1, 1))
47         theta -= alpha * (gradient + reg_term)
48         J_history.append(compute_cost(X, y, theta, lambda_))
49
50     return theta, J_history
51
52 # 示例:
53
54 # 假设我们有以下训练集
55 X = np.array([[1, 2104], [1, 1416], [1, 1534], [1, 852]]) # 特征矩阵 (m x n)
56 y = np.array([[400], [232], [315], [178]]) # 标签向量 (m x 1)
57 theta = np.zeros((2, 1)) # 初始化权重参数 (n
58 x 1)
59
60 # 参数
61 alpha = 0.01 # 学习率
62 lambda_ = 1 # 正则化参数
63 num_iters = 1000 # 迭代次数
64
65 # 执行梯度下降
66 optimal_theta, J_history = gradient_descent(X, y, theta, alpha, lambda_,
67 num_iters)
68
69 # 输出结果
70 print("优化后的参数 (权重):", optimal_theta)
71 print("训练损失:", J_history[-1])

```

RMSProp优化算法

下面是 RMSProp 优化算法的 Python 实现。RMSProp 是一种适用于非平稳目标函数的自适应学习率方法，常用于训练神经网络。

```

1  import numpy as np
2
3  class RMSProp:
4      def __init__(self, learning_rate=0.001, beta=0.9, epsilon=1e-8):
5          self.learning_rate = learning_rate
6          self.beta = beta
7          self.epsilon = epsilon
8          self.s = None
9
10     def update(self, w, dw):
11         if self.s is None:
12             self.s = np.zeros_like(dw)
13
14         # Update the running average of squared gradients
15         self.s = self.beta * self.s + (1 - self.beta) * (dw ** 2)
16
17         # Update the weights
18         w -= self.learning_rate * dw / (np.sqrt(self.s) + self.epsilon)
19
20         return w
21
22     # Example Usage
23     def loss_fn(w):
24         # Example quadratic loss: (w - 3)^2
25         return (w - 3) ** 2
26
27     def gradient_fn(w):
28         # Gradient of the quadratic loss
29         return 2 * (w - 3)
30
31     # Initialize weight
32     w = np.array([0.0])
33
34     # Initialize RMSProp optimizer
35     optimizer = RMSProp(learning_rate=0.1)
36
37     # Training loop
38     for i in range(100):
39         # Compute gradient
40         dw = gradient_fn(w)
41
42         # Update weights using RMSProp optimizer
43         w = optimizer.update(w, dw)
44
45         # Print the loss every 10 iterations
46         if i % 10 == 0:
47             loss = loss_fn(w)

```

```
48         print(f"Iteration {i}: Loss = {loss}, Weight = {w}")
49
50     # Final result
51     print(f"Final optimized weight: {w}")
```

🌱 说明

- `RMSProp` 类实现了RMSProp优化算法，其中 `__init__` 方法负责初始化学习率、衰减因子和一个小的常量以防止除零误差。
- `update` 方法用于调整权重。它根据梯度 `dw` 更新权重 `w`，并维护梯度平方的指数移动平均 `self.s`。
- 示例部分定义了一个简单的二次损失函数及其梯度，并使用 `RMSProp` 优化器来更新初始权重。

交叉熵损失函数

下面是一个用于计算交叉熵损失函数的Python实现代码。这个函数可以处理单个样本或批量样本的数据。代码还包含一些基本的输入验证，以确保输入的正确性。

```
1  import numpy as np
2
3  def cross_entropy_loss(y_true, y_pred):
4      """
5      计算交叉熵损失函数。
6
7      参数:
8      - y_true: 真实的标签, 形状为 (n_samples, n_classes), 使用one-hot编码。
9      - y_pred: 预测的概率分布, 形状为 (n_samples, n_classes)。
10
11     返回:
12     - 平均交叉熵损失。
13     """
14     # 转换为numpy数组
15     y_true = np.array(y_true)
16     y_pred = np.array(y_pred)
17
18     # 检查输入的形状
19     if y_true.shape != y_pred.shape:
20         raise ValueError("真实标签和预测标签的形状必须一致。")
21
22     # 防止log(0)的计算, 使用一个很小的值epsilon进行调整
23     epsilon = 1e-15
```

```

24     y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
25
26     # 计算交叉熵损失
27     loss = -np.sum(y_true * np.log(y_pred), axis=1)
28     return np.mean(loss)
29
30 # 示例用法
31 y_true = [[1, 0, 0], [0, 1, 0], [0, 0, 1]] # One-hot编码的真实标签
32 y_pred = [[0.8, 0.1, 0.1], [0.1, 0.7, 0.2], [0.2, 0.2, 0.6]] # 模型预测的概率分布
33
34 loss = cross_entropy_loss(y_true, y_pred)
35 print(f"平均交叉熵损失: {loss}")

```

参数调优与微调

Prompt Tuning

Prompt Tuning 是一种通过优化提示（prompt）来适应预训练语言模型的新任务的方法。以下是一个简化的实现例子，旨在展示如何使用 Python 和 Hugging Face 的 Transformers 库来进行基本的 Prompt Tuning。

```

1  import torch
2  from transformers import GPT2Tokenizer, GPT2LMHeadModel
3  from torch.optim import Adam
4  from tqdm import tqdm
5
6  # 加载预训练的GPT-2模型和分词器
7  model_name = "gpt2"
8  tokenizer = GPT2Tokenizer.from_pretrained(model_name)
9  model = GPT2LMHeadModel.from_pretrained(model_name)
10
11 # 如果有GPU则使用GPU
12 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
13 model.to(device)
14
15 # 创建用于Prompt Tuning的提示词汇
16 prompt = "Translate English to French: "
17 prompt_tokens = tokenizer(prompt, return_tensors='pt').input_ids.to(device)
18
19 # 定义训练数据
20 train_data = [
21     ("Hello, how are you?", "Bonjour, comment ça va ?"),
22     ("What is your name?", "Quel est votre nom ?"),

```

```

23     # 添加更多数据...
24 ]
25
26 # 设置超参数
27 learning_rate = 1e-5
28 num_epochs = 3
29
30 # 将模型设置为训练模式
31 model.train()
32
33 # 使用Adam优化器
34 optimizer = Adam(model.parameters(), lr=learning_rate)
35
36 for epoch in range(num_epochs):
37     total_loss = 0
38     for source_text, target_text in tqdm(train_data, desc=f"Epoch {epoch+1}"):
39         # 准备输入和目标
40         input_text = prompt + source_text
41         target_text = prompt + target_text
42         input_ids = tokenizer(input_text,
43                               return_tensors='pt').input_ids.to(device)
44         target_ids = tokenizer(target_text,
45                               return_tensors='pt').input_ids.to(device)
46
47         # 前向传播
48         outputs = model(input_ids, labels=target_ids)
49         loss = outputs.loss
50
51         # 反向传播和优化
52         optimizer.zero_grad()
53         loss.backward()
54         optimizer.step()
55
56         total_loss += loss.item()
57
58     print(f"Epoch {epoch+1} Loss: {total_loss / len(train_data)}")
59
60 # 保存微调后的模型
61 model.save_pretrained("prompt-tuned-model")
62 tokenizer.save_pretrained("prompt-tuned-model")

```

说明

- **预训练模型：**我们使用 GPT-2 作为基础模型，因为这是一个流行的通用文本生成模型。

- **提示词汇**：我们在训练数据中增加了提示 "Translate English to French: "，它指导模型执行我们指定的任务。
- **数据集**：示例中，训练数据是简单的英文到法文句子翻译对。实际任务中你需要替换为自己的数据。
- **损失和优化**：使用 `CrossEntropyLoss` 来计算损失，并采用 Adam 优化器进行优化。
- **设备**：代码会自动检测是否有可用的 GPU 并自动选择设备。

Prefix Tuning

Prefix Tuning 是一种高效的参数调优方法，专门用于生成性任务中的大型语言模型，比如GPT-3。其主要思想是在输入序列前添加可训练的前缀，以微调模型的性能。下面是一个使用 Python 和 Hugging Face 的 Transformers 库实现 Prefix Tuning 的基本示例。

```
1 import torch
2 from torch import nn
3 from transformers import GPT2Tokenizer, GPT2LMHeadModel
4
5 class PrefixTuning(nn.Module):
6     def __init__(self, gpt2_model, prefix_length):
7         super(PrefixTuning, self).__init__()
8         self.gpt2 = gpt2_model
9         self.prefix_length = prefix_length
10
11         # Initialize learnable prefix embeddings
12         self.prefix_embeddings = nn.Parameter(torch.randn(prefix_length,
13 self.gpt2.config.n_embd))
14
15     def forward(self, input_ids, attention_mask=None):
16         batch_size = input_ids.size(0)
17
18         # Create a batch of prefix embeddings
19         prefix_embs =
20 self.prefix_embeddings.unsqueeze(0).expand(batch_size, -1, -1)
21
22         # Pass through the GPT-2 model
23         inputs_embs = self.gpt2.transformer.wte(input_ids) # Get the input
24 embeddings
25         inputs_embs = torch.cat((prefix_embs, inputs_embs), dim=1)
26
27         # Adjust attention mask to include prefixes
28         if attention_mask is not None:
29             prefix_mask = torch.ones(batch_size,
30 self.prefix_length).to(attention_mask.device)
```

```

27         attention_mask = torch.cat((prefix_mask, attention_mask), dim=1)
28
29     # Forward pass
30     outputs = self.gpt2(inputs_embeds=inputs_embeds,
31                          attention_mask=attention_mask)
32
33     return outputs
34
35 # Load pre-trained GPT-2 model and tokenizer
36 tokenizer = GPT2Tokenizer.from_pretrained("gpt2")
37 gpt2_model = GPT2LMHeadModel.from_pretrained("gpt2")
38
39 # Initialize Prefix Tuning model
40 prefix_length = 10
41 model = PrefixTuning(gpt2_model, prefix_length)
42
43 # Sample input
44 input_text = "Once upon a time"
45 input_ids = tokenizer.encode(input_text, return_tensors="pt")
46
47 # Forward pass through the Prefix Tuning model
48 outputs = model(input_ids)
49 logits = outputs.logits
50
51 # Generate text using the adjusted model
52 generated_ids = torch.argmax(logits, dim=-1)
53 generated_text = tokenizer.decode(generated_ids[0])
54
55 print(f"Input: {input_text}")
56 print(f"Generated Text: {generated_text}")

```

🌱 说明

- `MOELayer` 类实现了一个简单的专家混合层，该层由多个专家（Dense层）和一个门控网络组成。门控网络根据输入数据为每个专家分配权重。以上代码中，我们定义了一个新的 `PrefixTuning` 类：
- `model`：使用 Hugging Face 的 GPT-2 作为我们基础的预训练模型。
- `prefix_length`：我们要在输入序列前添加的 learnable 前缀嵌入的长度。
- `prefix_embeddings`：一个 `nn.Parameter`，用于存储可学习的前缀嵌入。

模型的前向传播将 GPT-2 的输入嵌入与前缀嵌入结合起来，然后进行推理。

这个示例代码仅作基本功能实现，并不进行训练。实际使用时需要调整具体的训练步骤（如优化器设置、损失计算等）。务必确保在具体任务中，数据经过适当处理和设定。

LoRA (Low-Rank Adaptation)

实现LoRA (Low-Rank Adaptation) 需要对神经网络模型进行低秩分解调整。我们可以利用PyTorch 来实现这个方法。LoRA 主要通过对模型的权重矩阵做低秩分解来减少参数量。以下是一个简单的实现示例，展示如何在一个全连接层上应用 LoRA。

```
1 import torch
2 import torch.nn as nn
3
4 class LoRALayer(nn.Module):
5     def __init__(self, in_features, out_features, rank=4, alpha=1.0):
6         super(LoRALayer, self).__init__()
7         self.in_features = in_features
8         self.out_features = out_features
9         self.rank = rank
10        self.alpha = alpha
11
12        # Initialize A and B matrix with low rank
13        self.A = nn.Parameter(torch.randn(in_features, rank) * 0.01)
14        self.B = nn.Parameter(torch.randn(rank, out_features) * 0.01)
15
16        # Original linear layer
17        self.linear = nn.Linear(in_features, out_features)
18
19    def forward(self, x):
20        # Compute low-rank adaptation
21        low_rank = torch.mm(x, torch.mm(self.A, self.B)) * self.alpha
22        # Add low-rank adaptation to the original linear transformation
23        return self.linear(x) + low_rank
24
25 # Example usage
26 if __name__ == "__main__":
27     # Define input dimensions
28     input_dim = 8
29     output_dim = 4
30
31     # Create an instance of the LoRALayer
32     lora_layer = LoRALayer(input_dim, output_dim)
33
34     # Sample input
35     x = torch.randn(1, input_dim) # Batch size of 1
36
37     # Forward pass
38     output = lora_layer(x)
39
40     # Output
```

```
41     print("Output:")
42     print(output)
```

🌱 说明

- `LoRALayer` 类继承自 `nn.Module`，表示一个带 LoRA 调整的线性层。
- `A` 和 `B` 是可训练的参数，定义了低秩矩阵的分解部分。
- `forward` 方法中计算了低秩矩阵的输出，并将其与原装的线性输出叠加。
- 在使用示例中，输入是一个大小为 `(1, input_dim)` 的张量，经过 LoRA 调整的层后输出。
- `rank` 参数控制着低秩分解的秩，一般选择较小值来减少参数计算量。
- `alpha` 是一个缩放因子，用来调整低秩适应的贡献。

AdaLoRA (Adaptive LoRA)

LoRA方法用于减少大型语言模型中的参数数量并加速训练。AdaLoRA是这一方法的扩展版本，其关键思想在于通过路径对抗性扰动提升模型泛化。

```
1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import DataLoader, TensorDataset
5
6  class AdaLoRALayer(nn.Module):
7      def __init__(self, input_dim, output_dim, rank):
8          super(AdaLoRALayer, self).__init__()
9          self.input_dim = input_dim
10         self.output_dim = output_dim
11         self.rank = rank
12
13         # Low-rank decomposition components
14         self.U = nn.Parameter(torch.randn(input_dim, rank),
requires_grad=False)
15         self.V = nn.Parameter(torch.randn(rank, output_dim),
requires_grad=True)
16
17     def forward(self, x):
18         # Linear projection
19         low_rank_projection = torch.matmul(self.U, self.V)
20         return torch.matmul(x, low_rank_projection)
21
```

```

22     def update_U(self, grad_output):
23         # Gradient accumulation for U
24         grad_output = grad_output.detach()
25         delta_U = torch.matmul(grad_output.transpose(0, 1), grad_output)
26         self.U += delta_U
27
28     class AdaLoRAModel(nn.Module):
29         def __init__(self, input_dim, hidden_dim, output_dim, rank):
30             super(AdaLoRAModel, self).__init__()
31             self.adalora_layer = AdaLoRALayer(input_dim, hidden_dim, rank)
32             self.output_layer = nn.Linear(hidden_dim, output_dim)
33
34         def forward(self, x):
35             x = self.adalora_layer(x)
36             return self.output_layer(x)
37
38     # Helper function for training
39     def train_model(model, dataloader, criterion, optimizer, num_epochs):
40         for epoch in range(num_epochs):
41             for inputs, labels in dataloader:
42                 optimizer.zero_grad()
43                 outputs = model(inputs)
44                 loss = criterion(outputs, labels)
45                 loss.backward()
46                 optimizer.step()
47                 # Update U matrix in AdaLoRALayer
48                 with torch.no_grad():
49                     model.adalora_layer.update_U(outputs.grad)
50
51     input_dim = 100
52     hidden_dim = 50
53     output_dim = 10
54     rank = 5
55
56     # Synthetic dataset
57     X = torch.randn(1000, input_dim)
58     y = torch.randint(0, output_dim, (1000,))
59     dataset = TensorDataset(X, y)
60     dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
61
62     model = AdaLoRAModel(input_dim, hidden_dim, output_dim, rank)
63     criterion = nn.CrossEntropyLoss()
64     optimizer = optim.Adam(model.parameters(), lr=0.001)
65
66     # Train the model
67     train_model(model, dataloader, criterion, optimizer, num_epochs=10)

```

🌱 说明

上述代码是一个基本的框架，用于展示如何使用低秩方法构建自适应子空间嵌入层和进行简单的优化。关于路径对抗性扰动（Pathwise Adversarial Perturbation）的具体实现和更加详细的理论设置，需要根据具体的研究文献进行深入实现和测试。

QLoRA

实现一个简单的QLoRA（量化低秩近似）的Python代码需要多个复杂的步骤。QLoRA是一种用于压缩和加速大型语言模型推理过程的方法，它结合了低秩逼近和量化技术。以下是实现这一概念的一个基本示例代码，其中包含量化和低秩近似的基本实现步骤。

```
1  import numpy as np
2
3  def quantize_matrix(matrix, num_bits=8):
4      """
5      Quantizes the input matrix to the specified number of bits.
6
7      :param matrix: The matrix to quantize
8      :param num_bits: Number of bits for quantization
9      :return: Quantized matrix
10     """
11     max_val = np.max(matrix)
12     min_val = np.min(matrix)
13
14     # Quantization levels
15     q_levels = 2 ** num_bits
16
17     # Scale and offset
18     scale = (max_val - min_val) / (q_levels - 1)
19     offset = min_val
20
21     # Apply quantization
22     quantized_matrix = np.round((matrix - offset) / scale)
23
24     # De-quantize to approximate original scale
25     return quantized_matrix * scale + offset
26
27 def low_rank_approximation(matrix, rank):
28     """
29     Computes a low-rank approximation of the given matrix.
30
31     :param matrix: The original matrix
32     :param rank: The rank for the low-rank approximation
```

```

33     :return: Approximated matrix with the specified rank
34     """
35     # Singular Value Decomposition
36     U, S, Vt = np.linalg.svd(matrix, full_matrices=False)
37
38     # Retain only the top 'rank' singular values
39     S = np.diag(S[:rank])
40     U = U[:, :rank]
41     Vt = Vt[:rank, :]
42
43     # Compute the low-rank approximation
44     low_rank_matrix = np.dot(U, np.dot(S, Vt))
45
46     return low_rank_matrix
47
48 # Example usage
49 original_matrix = np.random.rand(100, 100)
50
51 # Perform low-rank approximation
52 rank = 10
53 approx_matrix = low_rank_approximation(original_matrix, rank)
54
55 # Quantize the approximated matrix
56 num_bits = 8
57 quantized_matrix = quantize_matrix(approx_matrix, num_bits)
58
59 # Output
60 print("Original Matrix:\n", original_matrix)
61 print("Low-Rank Approximated Matrix:\n", approx_matrix)
62 print("Quantized Matrix:\n", quantized_matrix)

```

🌱 说明

- `quantize_matrix()` 函数：将矩阵量化为指定的位数。
- `low_rank_approximation()` 函数：使用奇异值分解（SVD）对矩阵进行低秩近似。
- 示例使用了随机生成的100x100矩阵进行演示，通过低秩近似减少计算量，通过量化进一步压缩数据。

Dense模型与MOE（Mixture of Experts）

为了实现一个包含密集（Dense）层和专家混合（Mixture of Experts, MOE）机制的模型，我们可以使用TensorFlow和Keras库。密集层是深度学习中最常见的层，而MOE是一种模型组合方法，可以提高模型的训练效率和预测精度。

```

1 import tensorflow as tf
2 from tensorflow.keras.layers import Layer, Dense
3 from tensorflow.keras.models import Model
4
5 # 一个简单的专家混合 (MOE) 层
6 class MOELayer(Layer):
7     def __init__(self, n_experts, units, **kwargs):
8         super(MOELayer, self).__init__(**kwargs)
9         self.n_experts = n_experts
10        self.units = units
11        self.experts = [Dense(units) for _ in range(n_experts)]
12        self.gating_network = Dense(n_experts, activation='softmax')
13
14        def call(self, inputs):
15            # 获取专家网络的输出
16            expert_outputs = [expert(inputs) for expert in self.experts]
17            expert_outputs = tf.stack(expert_outputs, axis=-1)
18
19            # 使用门控网络来为每个专家分配权重
20            gates = self.gating_network(inputs)
21            weighted_expert_outputs = tf.reduce_sum(expert_outputs *
22            tf.expand_dims(gates, axis=-2), axis=-1)
23
24            return weighted_expert_outputs
25
26 # 构建模型函数
27 def build_model(input_shape, n_experts, expert_units, dense_units):
28     inputs = tf.keras.Input(shape=input_shape)
29
30     # 添加MOE层
31     x = MOELayer(n_experts, expert_units)(inputs)
32
33     # 添加一个全连接 (Dense) 层
34     x = Dense(dense_units, activation='relu')(x)
35
36     # 添加输出层
37     outputs = Dense(1, activation='sigmoid')(x)
38
39     model = Model(inputs=inputs, outputs=outputs)
40     return model
41
42 # 设置模型参数
43 input_shape = (10,) # 输入特征的维度
44 n_experts = 3 # 专家的数量
45 expert_units = 16 # 每个专家的神经元数量
46 dense_units = 8 # Dense层的神经元数量

```

```

46
47 # 构建模型
48 model = build_model(input_shape, n_experts, expert_units, dense_units)
49
50 # 编译模型
51 model.compile(optimizer='adam', loss='binary_crossentropy', metrics=
    ['accuracy'])
52
53 # 打印模型架构
54 model.summary()

```

🌱 说明

- `MOELayer` 类实现了一个简单的专家混合层，该层由多个专家（Dense层）和一个门控网络组成。门控网络根据输入数据为每个专家分配权重。
- `build_model` 函数构建并返回一个Keras模型。该模型包括一个MOE层、一个Dense层以及输出层。
- 最后，模型被编译并输出其结构。。

动手实践

动手实现GRPO

https://github.com/aburkov/theLMbook/blob/main/GRPO_From_Scratch_Multi_GPU_DataParallel_Qwen_2_5_1_5B_Instruct.ipynb

从零构建GRPO：基于Qwen2.5-1.5B-Instruct的分布式实现指南



下面关于GRPO的实现都参考上面的英文教程，如果大家英文比较好的话，可以看原文～这一节动手实现需要8张A100，估计大多数同学都没有这个算力资源，所以下面的内容会详细把代码好好给大家介绍一下。展示如何通过GRPO（Group Relative Policy Optimization）方法构建分布式强化学习（RL）流水线，以微调语言模型在数学、逻辑和编程任务中的表现。

这些任务的特点是存在唯一正确答案，可通过简单字符串比对与真实答案进行验证。（这里埋了一个伏笔，GRPO的损失函数的一部分就是从这里来的）

! 请注意下面代码并不是完整可运行代码，是为了展示，提取了核心代码。**完整可运行代码放本章节的最后。**

1. Part 1: 基础设置与导入（Basic Setup and Imports）

1.1 安装依赖库

代码块

```
1 !pip install tf-keras # 为兼容Hugging Face部分功能
2 !pip install flash-attn # 安装FlashAttention2优化注意力计算
3 !pip install wandb # 安装实验追踪工具Weights & Biases
4 !pip install 'accelerate>=0.26.0' # 支持分布式训练的PyTorch加速库
5 !pip install transformers # Hugging Face Transformers API
6 !pip install datasets # Hugging Face数据集工具
```

- **作用：**安装必要库，包括深度学习框架（PyTorch）、模型工具（Hugging Face）、优化组件（FlashAttention2）及实验管理（wandb）。

1.2 导入基础库

代码块

```
1 import random
2 import copy
3 import re
4 import os
5 import numpy as np
6 import wandb
```



作用：

- `random` / `numpy`：数据随机化与数值计算。
- `copy`：对象深拷贝以避免引用污染。
- `re`：正则表达式处理文本。
- `os`：环境变量配置。
- `wandb`：实验日志记录。

1.3 导入深度学习相关库

代码块

```
1 import torch
2 import torch.nn as nn
3 from torch.nn.utils.rnn import pad_sequence
4 from transformers import AutoModelForCausalLM, AutoTokenizer
5 from datasets import load_dataset
```

作用：

- `torch`：构建与训练模型。
- `pad_sequence`：对变长序列进行填充以支持批处理。
- `AutoModelForCausalLM` / `AutoTokenizer`：加载预训练因果语言模型及其分词器。
- `load_dataset`：加载Hugging Face数据集。

1.4 设置随机种子（确保可复现性）

代码块

```
1 def set_random_seed(seed: int = 42):
2     random.seed(seed)
3     np.random.seed(seed)
4     torch.manual_seed(seed)
5     if torch.cuda.is_available():
6         torch.cuda.manual_seed_all(seed)
7     torch.backends.cudnn.deterministic = True
8     torch.backends.cudnn.benchmark = False
9
10 set_random_seed(42)
```

作用：固定Python、NumPy、PyTorch的随机种子，确保实验结果可复现。

关键点：

- `torch.cuda.manual_seed_all`：设置所有GPU的随机种子。
- `cudnn.deterministic`：强制cuDNN使用确定性算法。
- `cudnn.benchmark = False`：禁用自动算法优化以避免随机性。

1.5 配置Weights & Biases环境变量

代码块

```
1 os.environ["WANDB_API_KEY"] = "USE YOUR KEY"
2 os.environ["WANDB_PROJECT"] = "GRPO-Qwen-1.5-Instruct-Multi-GPU"
```

作用：

- `WANDB_API_KEY`：替换为个人wandb账户的API密钥以启用日志同步。

- `WANDB_PROJECT`：指定实验所属的wandb项目名称。

2. Part 2: 数据格式化与答案提取 (Data Formatting and Answer Extraction)

2.1 定义系统提示 (SYSTEM_PROMPT)

代码块

```
1 SYSTEM_PROMPT = """
2 Respond in the following format:
3 <reasoning>
4 ...
5 </reasoning>
6 <answer>
7 ...
8 </answer>
9 """
```

 **作用：**强制模型输出格式化为XML风格的 `<reasoning>`（推理过程）和 `<answer>`（最终答案）标签，便于后续答案提取。（**如果没有按照这个格式输出，则会在损失函数中进行惩罚**）

关键设计：通过结构化输出降低解析复杂度，确保模型生成的答案可被程序化提取。

2.2 从模型输出提取答案 (extract_answer_from_model_output)

代码块

```
1 def extract_answer_from_model_output(text):
2     parts = text.split("<answer>")
3     if len(parts) < 2:
4         return None
5     last_part = parts[-1]
6     if "</answer>" not in last_part:
7         return None
8     answer = last_part.split("</answer>")[0].strip()
9     return None if answer == "..." else answer
```

 **逐行解释：**

- `split("<answer>")`：按 `<answer>` 标签分割文本，获取所有候选答案段。
- `len(parts) < 2`：若未检测到标签，返回 `None`（无有效答案）。

- c. `last_part = parts[-1]`：取最后一个 `<answer>` 标签后的内容（防止中间标签干扰）。
- d. `if "</answer>" not in last_part`：检查闭合标签是否存在。
- e. `split("</answer>")[0].strip()`：提取闭合标签前的内容并去除空格。
- f. `return None if answer == "..."`：过滤占位符（如模型生成的无效 `...`）。

2.3 从数据集提取答案 (extract_answer_from_dataset)

代码块

```
1 def extract_answer_from_dataset(text):  
2     if "####" not in text:  
3         return None  
4     return text.split("####")[1].strip()
```



逐行解释：

- a. `if "####" not in text`：检查GSM8K数据集的分隔符是否存在。
- b. `split("####")[1].strip()`：分割文本并取第二部分（答案），去除空格。

背景：GSM8K数据集使用 `####` 分隔问题与答案，此函数直接提取标准答案。



关键设计原理

1. 格式一致性：

- 通过 `SYSTEM_PROMPT` 强制模型输出结构化文本，确保答案提取逻辑的稳定性。
- 使用闭合标签（如 `<answer>` 和 `</answer>`）避免歧义。

2. 容错机制：

- 处理标签缺失（如未生成 `<answer>` 时返回 `None`）。
- 过滤无效占位符（如 `...`），避免误判模型输出。

3. 数据集适配：

- 针对GSM8K的 `####` 分隔符设计专用提取函数，确保与数据集格式兼容。

代码功能概述

本部分代码完成以下任务：

1. **数据集加载与格式化**: 从Hugging Face加载GSM8K数学数据集, 将原始数据转换为包含系统提示 (`SYSTEM_PROMPT`) 和用户问题的字符串格式。
2. **答案提取**: 利用 `extract_answer_from_dataset` 函数 (定义于Part 2) 从数据集中提取标准答案。
3. **提示构建**: 通过 `build_prompt` 函数将结构化的消息列表 (系统指令+用户问题) 合并为统一的字符串, 确保训练与推理时的输入格式一致。

3. Part 3: 数据集准备 (Dataset Preparation)

3.1 数据集预处理函数 (`prepare_dataset`)

代码块

```
1 def prepare_dataset(split="train"):
2     data = load_dataset('openai/gsm8k', 'main')[split]
3     formatted_data = []
4     for example in data:
5         prompt_str = build_prompt([
6             {"role": "system", "content": SYSTEM_PROMPT},
7             {"role": "user", "content": example["question"]}
8         ])
9         formatted_example = {
10             "prompt": prompt_str,
11             "answer": extract_answer_from_dataset(example["answer"])
12         }
13         formatted_data.append(formatted_example)
14     return formatted_data
```

🌱 逐行解释:

- a. `load_dataset('openai/gsm8k', 'main')[split]`: 从Hugging Face加载GSM8K数据集的指定切分 (训练集或测试集)。
- b. `build_prompt([...])`: 构建包含系统提示和用户问题的完整提示字符串:
 - `role: system`: 插入定义好的 `SYSTEM_PROMPT` (要求模型输出 `<reasoning>` 和 `<answer>` 标签)。
 - `role: user`: 插入当前样本的数学问题 (`example["question"]`)。
- c. `extract_answer_from_dataset(...)`: 从数据集答案字段提取标准答案 (通过 `####` 分隔符分割)。
- d. `formatted_data.append(...)`: 将格式化后的样本 (提示字符串+标准答案) 添加到结果列表。

3.2 提示构建函数 (build_prompt)

代码块

```
1 def build_prompt(messages):
2     return "\n".join([msg["content"].strip() for msg in messages])
```

逐行解释:

- `[msg["content"].strip() for msg in messages]`: 遍历消息列表, 提取每个消息的 `content` 字段并去除首尾空格。
- `"\n".join(...)`: 将所有内容按换行符拼接为单个字符串, 确保提示格式与训练时的输入一致。

关键设计原理

1. 格式一致性保障:

- 系统提示 (`SYSTEM_PROMPT`): 强制模型输出结构化文本 (`<reasoning>` 和 `<answer>` 标签), 便于后续答案提取。
- 提示构建 (`build_prompt`): 将多角色消息 (系统指令+用户问题) 合并为纯文本, 模拟真实对话场景的输入格式。

2. 数据集适配:

- 答案提取: GSM8K数据集的答案通过 `####` 分隔符与问题分离, `extract_answer_from_dataset` 函数直接解析该分隔符。

3. 训练与推理对齐:

- 格式化后的提示字符串 (`prompt_str`) 在训练和推理时保持完全相同的结构, 避免因输入格式差异导致模型性能下降。

4. Part 4: 评估函数 (Evaluation Functions)

4.1 提取末尾数值 (extract_last_number)

代码块

```
1 def extract_last_number(text):
2     text = text.replace('$', '').replace('%', '')
3     pattern = r'(?:(?:^|s|=)\s*(-?\d*\.\d+)?\s*$)'
4     match = re.search(pattern, text)
5     return float(match.group(1)) if match else None
```

逐行解释:

- a. `replace('$', '')`: 移除美元符号和百分号, 避免干扰数值提取。
- b. `pattern`: 正则表达式匹配末尾数值:
 - `(?:^|\s|=)`: 匹配行首、空格或等号 (确保数值独立)。
 - `(-?\d*\.\?\d+)`: 捕获可能带负号的整数或浮点数。
- c. `re.search`: 查找符合模式的末尾数值, 返回浮点数或 `None`。

4.2 提取单个数值 (`extract_single_number`)

代码块

```
1 def extract_single_number(text):
2     numbers = re.findall(r'-?\d*\.\?\d+', text)
3     return float(numbers[0]) if len(numbers) == 1 else None
```

逐行解释:

1. `findall(r'-?\d*\.\?\d+')`: 提取所有数值 (支持负数和小数)。
2. `len(numbers) == 1`: 仅当文本中存在唯一数值时返回, 否则返回 `None`, 避免歧义。

4.3 模型评估主函数 (`evaluate_model`)

代码块

```
1 def evaluate_model(model, tokenizer, eval_examples, device):
2     model.eval()
3     correct = 0
4     total = len(eval_examples)
5     print("\n" + "="*50)
6     print("EVALUATION ON", total, "EXAMPLES")
7     print("="*50)
8
9     for example in eval_examples:
10         full_prompt = example["prompt"]
11         expected = example["answer"]
12         inputs = tokenizer.encode(full_prompt, return_tensors="pt").to(device)
13         with torch.no_grad():
14             outputs = model.generate(
15                 inputs,
16                 max_new_tokens=512,
```

```

17         temperature=0.7,
18         num_return_sequences=1,
19         pad_token_id=tokenizer.pad_token_id,
20         eos_token_id=tokenizer.eos_token_id,
21         forced_eos_token_id=tokenizer.eos_token_id,
22         early_stopping=False,
23     )
24     response = tokenizer.decode(outputs[0], skip_special_tokens=True)
25
26     try:
27         predicted = extract_answer_from_model_output(response)
28         if predicted == expected:
29             is_correct = True
30         else:
31             pred_num = extract_single_number(str(predicted))
32             exp_num = extract_single_number(str(expected))
33             if pred_num is not None and exp_num is not None and pred_num
== exp_num:
34                 is_correct = True
35             else:
36                 pred_num = extract_last_number(str(predicted))
37                 exp_num = extract_last_number(str(expected))
38                 is_correct = (pred_num is not None and exp_num is not None
and
39                             pred_num == exp_num)
40         if is_correct:
41             correct += 1
42
43     print("\nPrompt:")
44     print(full_prompt)
45     print("\nExpected Answer:")
46     print(expected)
47     print("\nExtracted Answer:")
48     print(predicted)
49     print("\nFull Generated Response:")
50     print(response)
51     print("\nCorrect:", "✓" if is_correct else "✗")
52     print("-"*50)
53
54     except Exception as e:
55         print("\nFailed to parse model output for prompt:")
56         print(full_prompt)
57         print("Error:", e)
58         print("-"*50)
59
60     accuracy = (correct / total) * 100
61     print(f"\nAccuracy: {accuracy:.2f}% ({correct}/{total})")

```

```
62     print("="*50)
63     model.train()
64     return accuracy
```

逐行解释：

a. 模型模式切换：

- `model.eval()`：设置模型为评估模式，关闭 dropout 等训练特有操作。

b. 遍历评估样本：

- `full_prompt`：获取当前样本的完整提示（含系统指令和问题）。
- `expected`：从数据集中提取标准答案。

c. 生成模型响应：

- `tokenizer.encode`：将提示文本编码为模型输入张量。
- `model.generate`：生成响应文本，参数包括：
 - `max_new_tokens=512`：最大生成长度。
 - `temperature=0.7`：控制生成随机性。
 - `pad_token_id / eos_token_id`：处理填充和结束标记。

d. 答案提取与校验：

- `extract_answer_from_model_output`：提取模型生成的 `<answer>` 标签内容。
- 三重校验机制：
 1. 精确匹配：直接比对字符串是否完全一致。
 2. 单数值匹配：若双方答案均含唯一数值且相等则视为正确。
 3. 末尾数值匹配：若双方答案末尾数值相等则视为正确（适用于单位不一致场景）。

e. 结果统计与日志：

- 打印每个样本的提示、预期答案、模型输出及校验结果。
- 计算并输出总体准确率（`accuracy`）。

f. 恢复训练模式：

- `model.train()`：评估结束后恢复模型为训练模式。

关键设计原理

a. 多维度答案校验：

- **精确匹配**：确保答案格式严格符合要求（如单位、符号）。
- **数值匹配**：容忍格式差异（如 \$50 vs 50 ），关注数值本身。

b. 容错机制：

- 使用 `try-except` 捕获解析异常，避免因单个样本错误导致评估中断。

c. 评估指标一致性：

- 准确率（Accuracy）作为核心指标，符合分类任务评估标准。

5. Part 5: 奖励函数（Reward Functions）



代码功能概述

本部分代码实现强化学习中的奖励函数设计，包含以下核心组件：

1. **正确性奖励**（`correctness_reward`）：根据答案正确性给予不同奖励（精确匹配：2.0，数值匹配：1.5，错误：0.0）。
2. **格式奖励**（`format_reward`）：通过检测XML标签（`<reasoning>` / `<answer>`）鼓励模型保持结构化输出，每个标签奖励0.2，总上限0.8。
3. **组合奖励**（`combined_reward`）：将上述两个奖励加权求和，形成最终的强化学习反馈信号。

5.1 正确性奖励函数（`correctness_reward`）

代码块

```
1 def correctness_reward(prompts, completions, answer, kwargs):
2     responses = [completion[0]['content'] for completion in completions]
3     extracted = [extract_answer_from_model_output(r) for r in responses]
4     rewards = []
5     for r, a in zip(extracted, answer):
6         if r == a: # 精确匹配
7             rewards.append(2.0)
8         else:
9             r_num = extract_single_number(str(r))
10            a_num = extract_single_number(str(a))
11            if r_num is not None and a_num is not None and r_num == a_num:
12                rewards.append(1.5) # 数值匹配
13            else:
14                rewards.append(0.0) # 错误答案
15     completion_lengths = [len(response.split()) for response in responses]
16     return rewards
```

逐行解释：

- a. `responses = [...]`：从模型输出中提取原始文本内容。
- b. `extracted = [...]`：通过 `extract_answer_from_model_output` 提取 `<answer>` 标签内的答案。
- c. 奖励分配逻辑：
 - 精确匹配 (2.0)：若模型输出与标准答案字符串完全一致。
 - 数值匹配 (1.5)：若双方答案包含唯一数值且相等（如 `$50` vs `50`）。
 - 错误 (0.0)：其他情况。
- d. `completion_lengths`：记录生成文本长度（用于分析生成质量）。

5.2 格式奖励函数 (format_reward)

代码块

```
1 def format_reward(completions, kwargs):
2     responses = [completion[0]['content'] for completion in completions]
3     rewards = []
4     format_scores = []
5     for response in responses:
6         score = 0.0
7         if "<reasoning>" in response: score += 0.2
8         if "</reasoning>" in response: score += 0.2
9         if "<answer>" in response: score += 0.2
10        if "</answer>" in response: score += 0.2
11        rewards.append(score)
12        format_scores.append(score)
13    return rewards
```

逐行解释：

- a. `responses = [...]`：提取模型生成的原始文本。
- b. 标签检测：
 - 每个标签（`<reasoning>`，`</reasoning>`，`<answer>`，`</answer>`）存在时奖励0.2。
 - 总奖励上限0.8，确保格式合规性。
- c. `format_scores`：记录每个样本的格式得分（用于调试或分析）。

5.3 组合奖励函数 (combined_reward)

代码块

```
1 def combined_reward(prompts, completions, answer):
2     correctness_scores = correctness_reward(prompts=prompts,
3     completions=completions, answer=answer)
4     format_scores = format_reward(completions=completions)
5     combined_rewards = []
6     for c_score, f_score in zip(correctness_scores, format_scores):
7         combined_rewards.append(c_score + f_score)
8     return combined_rewards
```



逐行解释：

a. 奖励组合：

- 正确性奖励（0.0-2.0）与格式奖励（0.0-0.8）直接相加。
- 总奖励范围0.0-2.8，确保正确性主导学习方向。

b. 设计原则：

- 正确性优先（权重更高），格式奖励作为辅助信号防止模型遗忘结构化输出。

关键设计原理

1. 分层奖励设计：

- **正确性奖励**：直接关联任务目标（数学问题求解），符合强化学习中“奖励函数应反映最终目标”的原则。
- **格式奖励**：通过奖励塑形（Reward Shaping）维持模型输出的结构化，避免因RL训练破坏预训练的格式能力。

2. 数值匹配容错：

- 允许答案格式差异（如单位、符号），仅关注数值本身，提升评估鲁棒性。

3. 奖励平衡：

- 格式奖励权重较低（0.8 vs 正确性2.0），确保模型优先优化答案正确性，同时保持输出规范。

6. Part 6: 从零实现GRPO



本部分代码是实现GRPO（Group Relative Policy Optimization）算法的核心逻辑，包含以下关键组件：

1. 分布式训练：通过PyTorch的 `DataParallel` 实现多GPU并行计算。

2. 强化学习流水线：

- 生成模型补全（`generate_completions`）。
- 计算对数概率（`compute_log_probs`）。
- 构建奖励函数（`grpo_loss`）。

3. 策略优化：结合PPO的截断目标与KL散度惩罚，更新模型参数。

6.1 选择性对数概率计算（`selective_log_softmax`）

代码块

```
1 def selective_log_softmax(logits, input_ids):
2     log_probs = nn.functional.log_softmax(logits, dim=-1)
3     return log_probs.gather(dim=-1, index=input_ids.unsqueeze(-1)).squeeze(-1)
```

 作用：计算指定 `input_ids` 对应的对数概率。

逐行解释：

- a. `log_softmax`：将logits转换为对数概率分布。
- b. `gather`：提取目标 `input_ids` 对应的对数概率值。

6.2 批量对数概率计算（`compute_log_probs`）

代码块

```
1 def compute_log_probs(model, input_ids, attention_mask, logits_to_keep):
2     logits = model(input_ids=input_ids,
3                     attention_mask=attention_mask).logits[:, :-1, :]
4     input_ids = input_ids[:, -logits_to_keep:]
5     logits = logits[:, -logits_to_keep:, :]
6     return selective_log_softmax(logits, input_ids)
```

 作用：为输入序列的最后 `logits_to_keep` 个token计算对数概率。

逐行解释：

- a. `logits[:, :-1, :]`：排除最后一个token的预测（因为预测的是下一个token）。
- b. `input_ids[:, -logits_to_keep:]`：截取需计算的token ID。

c. 调用 `selective_log_softmax` 完成概率计算。

6.3 补全掩码生成 (`create_completion_mask`)

代码块

```
1 def create_completion_mask(completion_ids, eos_token_id):
2     is_eos = completion_ids == eos_token_id
3     eos_idx = torch.full(...).int().argmax(dim=1)[mask_exists]
4     sequence_indices = torch.arange(...).expand(...)
5     return (sequence_indices <= eos_idx.unsqueeze(1)).int()
```

 作用：生成二进制掩码，忽略EOS（结束符）后的token。

关键逻辑：

- 检测每个序列中第一个EOS的位置。
- 生成掩码：EOS前的token标记为1，之后为0。

6.4 补全生成 (`generate_completions`)

代码块

```
1 def generate_completions(model, tokenizer, prompts, num_generations=4,
2     max_completion_length=32):
3     inputs = tokenizer(prompts, return_tensors="pt", padding=True)
4     prompt_ids = inputs["input_ids"].to(device)
5     prompt_ids = prompt_ids.repeat_interleave(num_generations, dim=0)
6     outputs = model.generate(..., max_new_tokens=max_completion_length, ...)
7     completion_ids = outputs[:, prompt_length:]
8     completion_mask = create_completion_mask(completion_ids,
9     tokenizer.eos_token_id)
10    return prompt_ids, prompt_mask, completion_ids, completion_mask
```

 作用：为每个提示生成多个补全，并构建训练所需的数据结构。

关键步骤：

- `repeat_interleave`：复制提示以生成多组补全。
- `generate`：调用模型生成补全文本。
- `completion_mask`：标记有效token（EOS前）。

6.5 强化学习数据构建 (generate_rollout_data)

代码块

```
1 def generate_rollout_data(model, ref_model, tokenizer, batch_samples,
2   num_generations, max_completion_length):
3     with torch.no_grad():
4         prompt_ids, prompt_mask, completion_ids, completion_mask =
5         generate_completions(...)
6         input_ids = torch.cat([prompt_ids, completion_ids], dim=1)
7         old_log_probs = compute_log_probs(model, ...)
8         ref_log_probs = compute_log_probs(ref_model, ...)
9     return {
10         "input_ids": input_ids,
11         "old_log_probs": old_log_probs,
12         ...
13     }
```



作用：生成包含策略模型与参考模型对数概率的训练数据。

关键逻辑：

- `torch.cat`：合并提示与补全的token ID。
- 分别计算当前策略模型和参考模型的对数概率。

6.6 GRPO损失函数 (grpo_loss)

代码块

```
1 def grpo_loss(model, ref_model, rollout_data, tokenizer, reward_function,
2   beta=0.01, epsilon=0.2):
3     token_log_probs = compute_log_probs(model, ...)
4     ratio = torch.exp(token_log_probs - old_log_probs)
5     rewards = reward_function(...)
6     advantages = (rewards - mean_rewards) / (std_rewards + 1e-4)
7     surr1 = ratio * advantages
8     surr2 = torch.clamp(ratio, 1 - epsilon, 1 + epsilon) * advantages
9     kl = torch.exp(ref_log_probs - token_log_probs) - (...) - 1
10    loss = -((surrogate_loss - beta * kl) * mask).mean()
11    return loss, avg_reward
```



作用：结合PPO截断目标与KL散度惩罚，计算策略更新的损失。

核心公式：

- **优势估计**：标准化奖励以减少方差。
- **PPO目标**： $\min(\text{ratio} * A, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) * A)$ 。
- **KL惩罚**：防止策略偏离参考模型。

6.7 训练循环 (train_with_grpo)

代码块

```
1 def train_with_grpo(model, tokenizer, train_data, ...):
2     model = nn.DataParallel(model, device_ids=device_ids)
3     for iteration in range(num_iterations):
4         ref_model = copy.deepcopy(model.module)
5         optimizer = torch.optim.AdamW(...)
6         for step in range(num_steps):
7             batch_samples = random.sample(train_data, batch_size)
8             rollout_data = generate_rollout_data(...)
9             for grpo_iter in range(mu):
10                loss, avg_reward = grpo_loss(...)
11                loss.backward()
12                optimizer.step()
13     return model.module
```



作用：执行多轮GRPO训练，更新策略模型。

关键流程：

- 分布式训练**：通过 `DataParallel` 跨GPU并行计算。
- 参考模型更新**：每轮迭代复制当前策略作为参考模型。
- 多步优化**：每个批次执行 `mu` 次策略更新以稳定训练。

关键设计原理

1. 分布式并行：

- `DataParallel` 将模型复制到多个GPU，分批次处理数据，提升训练吞吐量。

2. 奖励塑形：

- 通过 `reward_function`（定义于Part 5）结合正确性与格式奖励，引导模型生成高质量答案。

3. KL散度惩罚：

- 防止策略模型在更新过程中过度偏离参考模型，增强训练稳定性。

7. Part 7: 训练设置与执行 (Training Setup and Execution)

🥥 本部分代码整合所有组件完成模型训练全流程，包含以下核心步骤：

1. **模型与分词器初始化**：加载预训练模型 `Qwen2.5-1.5B-Instruct`，配置优化参数（如 `bfloat16`、`FlashAttention2`）。
2. **初始评估**：在微调前评估模型基线准确率。
3. **强化学习训练**：通过自定义的 `train_with_grpo` 函数执行GRPO算法，结合正确性与格式奖励函数。
4. **最终评估与保存**：训练后再次评估模型性能并保存结果。

7.1 模型内存优化 (`optimize_model_memory`)

代码块

```
1 def optimize_model_memory(model):
2     model.train()
3     model.config.use_cache = False
4     if hasattr(model, "enable_input_require_grads"):
5         model.enable_input_require_grads()
6     else:
7         def make_inputs_require_grad(module, input, output):
8             output.requires_grad_(True)
9
10    model.get_input_embeddings().register_forward_hook(make_inputs_require_grad)
11    model.gradient_checkpointing_enable()
12    return model
```

🥥 **作用**：通过禁用KV缓存、启用梯度检查点技术降低显存占用。

关键步骤：

- a. `use_cache=False`：禁用缓存以节省内存。
- b. `enable_input_require_grads`：确保输入嵌入需要梯度。
- c. `gradient_checkpointing_enable`：以计算换内存，减少训练显存需求。

7.2 主执行流程

代码块

```
1 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

```

2 model_name = "Qwen/Qwen2.5-1.5B-Instruct"
3 output_dir = "math_solver_model"
4
5 model = AutoModelForCausalLM.from_pretrained(
6     model_name,
7     torch_dtype=torch.bfloat16,
8     device_map="auto"
9 )
10 tokenizer = AutoTokenizer.from_pretrained(model_name, padding_side="left")
11 tokenizer.pad_token = tokenizer.eos_token
12 model.config.pad_token_id = tokenizer.eos_token_id
13 model.config.eos_token_id = tokenizer.eos_token_id

```



逐行解释：

- a. 设备选择：自动选择GPU或CPU。
- b. 模型加载：使用 `bfloat16` 精度减少显存占用，`device_map="auto"` 自动分配多GPU资源。
- c. 分词器配置：设置左填充（`padding_side="left"`），并将填充符设为EOS标记。

7.3 数据集划分与初始评估

代码块

```

1 all_data = prepare_dataset("train")
2 random.shuffle(all_data)
3 eval_data = all_data[:30]
4 train_data = all_data[30:]
5
6 pre_grpo_accuracy = evaluate_model(model, tokenizer, eval_data, device)
7 print(f"Pre-GRPO Accuracy: {pre_grpo_accuracy:.2f}%")

```



作用：划分训练集与验证集，并输出微调前准确率。

关键逻辑：

- `prepare_dataset`：加载并格式化GSM8K数据集（定义于Part 3）。
- `evaluate_model`：执行初始评估（定义于Part 4）。

7.4 强化学习训练配置

代码块

```
1 training_config = {
2     'num_iterations': 1,
3     'num_steps': 500,
4     'batch_size': 7,
5     'num_generations': 12,
6     'max_completion_length': 400,
7     'beta': 0.04,
8     'learning_rate': 5e-6,
9     'mu': 1,
10    'epsilon': 0.1
11 }
```

超参数说明:

- `num_generations=12` : 每个提示生成12个补全, 增强奖励信号多样性。
- `beta=0.04` : KL散度惩罚系数, 防止策略偏离参考模型。
- `learning_rate=5e-6` : 低学习率确保稳定更新。

7.5 启动GRPO训练

代码块

```
1 model = train_with_grpo(
2     model=model,
3     tokenizer=tokenizer,
4     train_data=train_data,
5     reward_function=combined_reward,
6     device_ids=device_ids,
7     **training_config
8 )
```

作用: 执行分布式GRPO训练 (定义于Part 6)。

关键流程:

- `DataParallel` 分发模型到多GPU。
- 每轮迭代生成补全、计算奖励、更新策略。

7.6 最终评估与模型保存

代码块

```
1 post_grpo_accuracy = evaluate_model(model, tokenizer, eval_data, device)
2 model.save_pretrained("grpo_finetuned_model")
3 tokenizer.save_pretrained("grpo_finetuned_model")
```



作用：对比微调前后准确率，并保存最终模型。

关键逻辑：

- `save_pretrained`：将模型与分词器保存至指定目录，便于后续部署。

关键设计原理

1. 显存优化：

- 通过 `bfloat16` 和梯度检查点技术，在有限显存下支持大模型训练。

2. 分布式训练：

- `DataParallel` 实现多GPU并行，提升训练吞吐量。

3. 奖励函数组合：

- `combined_reward` 结合正确性与格式奖励，平衡任务目标与输出规范。

